Image Classifier Dojo — Refactor Design Doc

Purpose

Top-level entry point and reading guide for the modular design doc. Each section below links to its dedicated file. New readers should follow the sections in order; returning readers should jump directly to the topic they need.

Architectural shape

Dojo is a Hydra + Pydantic configuration-first toolkit for training, evaluating, ensembling, exporting, and inspecting image classification / regression / ordinal / SSL models. Key shape:

- **Config-first.** Hydra composes; Pydantic validates and is the runtime contract.
- **Modular model composition.** `image_input.backbone` (`torchvision` / `timm` / `checkpoint`) + optional `tabular_input` encoder + implicit image-then-tabular embedding concatenation + optional `embedding_adapter` + one-or-more heads. Objectives bind heads to losses, metrics, and weights.
- **Canonical results.** Per-row Parquet via `amplify-db-utils` with a `_metadata.json` sidecar. Configurable partitioning, dictionary encoding, and Arrow list columns for vectors.
- **Self-describing model artifacts.** Checkpoints and exports embed a portable *inference contract* (resolved model, inference pipeline + frozen preprocessing stats, class maps, target schema, `objective_summary`, and the four compatibility hashes), so `dojo infer` / `dojo eval` rebuild a model with no dependency on the producing run's `resolved.yaml`.
- **Four peer output blocks** at the top of the config tree: `training_outputs`, `ensemble_outputs`, `sweep_outputs`, and `eval_outputs` (the home for `dojo infer` / `dojo eval`), all rooted under a single `output_root`.
- **Task selection via `task.type`** (`supervised`, `ssl`, `snapshot_ensemble`) — there are no `train supervised` / `train ssl` subcommands.
- **Command-gated config blocks.** One root schema; the *command* (not `task.type`) selects which blocks are active — `task.type` only chooses the training paradigm within `dojo train`. Per-operation configs are the norm; a loaded lifecycle config is valid and unused blocks warn rather than fail.
- **Representation evaluation** (`representation_eval`) is a top-level block, usable against supervised or SSL encoders, training-integrated or standalone.
- **Ensembling** is prediction-space only in the initial implementation: explicit candidate discovery, candidate manifests, supported selection strategies, and combine modes. Snapshot ensembles share a run directory with their training step.
- **Sweeps** normalize into a top-level `sweep:` block. Functional `mode: grid` supports ordinary hyperparameter sweeps and batch-run-style seed sweeps. Initial execution is manual through `dojo sweep prepare`, selected per-run training, `dojo sweep status`, and `dojo sweep report`.
- **Lightweight base install + extras.** Base install supports config / schema / storage / result reading without Torch.

Design principles

- Pydantic schemas are the runtime contract.
- Strict schema (`extra="forbid"`): deferred features are absent, not stubbed — configuring one fails generic validation, with no reserved slots or runtime `NotImplementedError`.
- Keep task logic separate from model composition.
- Prefer canonical internal representations (single-head normalizes to the multi-head shape).
- Results are first-class artifacts; tall Parquet with a sidecar.
- Avoid generic metadata junk drawers; use explicit columns.
- Keep storage behind a Dojo interface (`amplify-storage-utils` underneath).
- Use `ifcbkit` for IFCB raw-bin handling.
- Make embeddings first-class.
- Optimize for external orchestration — entry points return structured results.

File map

Read in order:

- 01. Goals and Scope — what's in, what's out, what's deferred.
- 02. CLI and Task Types — canonical command surface; `dojo init` / `train` / `infer` / `eval` / `inspect` / `ensemble` / `sweep` / `export`; `task.type` semantics.
- 03. Configuration — canonical config tree; `output_root` and the four peer `*_outputs` blocks; path-template syntax; run-directory collision handling.
- 04. Data and Storage — supported dataset backends (`csv_manifest`, `parquet_manifest`, `parquet_images`, `ifcb_bins`); shared sample contract; `dojo inspect dataset`; storage resolver.
- 05. Models, Training, and Heads — transforms, backbones, freeze policies, heads, objectives, supervised training, optimizer / scheduler / checkpointing, transfer learning.
- 06. Results, Artifacts, and Metadata — canonical result schemas; identifiers and hashes; `_metadata.json` sidecar; partitioning; on-disk artifact layout; logging and diagnostics.
- 07. SSL and Representation Evaluation — Lightly DINOv2 (functional); `representation_eval`; probes, projections, clustering, diagnostics.
- 08. Ensembles — prediction-space ensembling; candidate discovery; manifests; selection strategies; combine modes; `ensemble_outputs:` layout; snapshot-ensemble integration.
- 09. Sweeps and Batch Runs — top-level `sweep:` block; grid sweeps; batch-run-style seed sweeps; deferred Bayesian schema; manual sweep preparation / status / reporting; `sweep_outputs:` block; sweep-id provenance; sweeps as candidate-source feeders.
- 10. Export — TorchScript and ONNX exports; `training_outputs.export` / `ensemble_outputs.export`, `sweep_outputs.artifacts`; `dojo export` command; export metadata; bucket-aware ONNX.
- 11. Dependencies — base install plus optional extras (`train`, `timm`, `ssl`, `ifcb`, `repr_eval`, `onnx`, `all`, `dev`). The `aim` extra is commented out and no `mlflow` / `s3` extra is currently declared (Aim/MLflow logging is deferred; S3 rides on the base `amplify-storage-utils` dependency).
- 12. Validation, Testing, and Preflight — layered validation; `runtime.preflight` controls; strict-schema policy for deferred features; test scope by area; fixture tiers.
- 13. Workplan — priority order for the new `src/dojo` implementation; thin-slice gate; deferred backlog.

Appendices and reference:

- Appendix — Deferred Features — the deferred-feature roadmap; deferred features are absent from the strict schema, plus the P4.8 cleanup milestone.
- Appendix — Repository Structure — proposed `configs/`, `src/dojo/`, and `tests/` tree derived from the canonical decisions above.
- Glossary — config keys, identifiers, hashes, record taxonomies; the single source of truth for terminology.

Tightly-coupled cross-cutting links

Bookmark these pairs:

- `03-configuration.md` & `06-results-artifacts-and-metadata.md` — `output_root`, the `*_outputs` blocks, and path templating connect to result writing and the artifact layout.
- `06-results-artifacts-and-metadata.md` & `08-ensembles.md` — cached result files feed ensembles; ensemble result rows reuse the standard schema with `stage=ensemble_eval`, `ensemble_result_scope`, and the `ensemble_member_id` union column for retained member rows.
- `05-models-training-and-heads.md` & `07-ssl-and-representation-eval.md` — supervised training can schedule representation evaluation; representation eval also runs standalone against checkpoints from either task type.
- `09-sweeps-and-batch-runs.md` & `03-configuration.md` / `06-results-artifacts-and-metadata.md` / `08-ensembles.md` — `sweep_outputs:` schema; `sweep_id` provenance on result rows; sweeps that feed ensemble candidate discovery.
- `12-validation-testing-and-preflight.md` & `appendix-deferred-features.md` — deferred features are absent from the strict schema (generic validation failure, no stubs); the cleanup milestone names its verification obligation.

01. Goals and Scope

Purpose

Defines the scope of the refactor: what the initial implementation must deliver, what is out of scope, and what is deferred. Other files explain how each goal is satisfied; this file just names them.

Core goals

- **Config-first, Hydra + Pydantic architecture.** Configs compose via Hydra and validate via Pydantic. Pydantic is the runtime contract.
- **Canonical CLI command families.** `dojo init` / `dojo train` / `dojo infer` / `dojo eval` / `dojo inspect` / `dojo ensemble` / `dojo sweep` / `dojo export`. Task paradigm selected via `task.type`. See `02-cli-and-task-types.md`.
- **Modular model composition.** `image_input.backbone` + optional `tabular_input` encoder + implicit image-then-tabular embedding concatenation + optional embedding adapter + one-or-more heads.
- **Multi-head, multi-objective supervised training.** Heads define output structure; objectives bind heads to losses, weights, and metrics.
- **Dataset backends:** `csv_manifest`, `parquet_manifest`, `parquet_images`, `ifcb_bins`. `class_folder` is an inspect-only source.
- **First-class embeddings.** Extractable from supervised models, SSL encoders, and transfer-learning checkpoints via `dojo infer embeddings`.
- **Canonical result schemas.** Tall Parquet via `amplify-db-utils`, with `_metadata.json` sidecar. CSV / wide exports are configurable derived outputs.
- **Layered validation and preflight.** Pydantic → `dojo inspect config` → `dojo inspect dataset` / training preflight → runtime validation.
- **Representation evaluation.** Both training-integrated and standalone (`dojo eval representation`). Usable against supervised or SSL encoders.
- **SSL via Lightly.** DINOv2 functional; other SSL methods are deferred.
- **Ensembling.** Prediction-space ensembles with explicit candidate discovery, candidate manifests, supported selection strategies, and combine modes. Snapshot ensembles via `task.type: snapshot_ensemble`.
- **Export.** TorchScript and ONNX via `training_outputs.export`, `ensemble_outputs.export`, sweep-level artifact promotion, or `dojo export`.
- **Lightweight base install plus optional extras.** `train`, `timm`, `ssl`, `ifcb`, `repr_eval`, `onnx`, `all`, `dev`. (`aim` is commented out and `mlflow` / `s3` extras are not currently declared — Aim/MLflow logging is deferred; S3 rides on the base `amplify-storage-utils` dependency.)

Non-goals for the initial implementation

- Maintaining backward compatibility with old listfile dataset formats.
- Porting the old `multilabel` module (it was multi-head multiclass, not multilabel).
- Porting `torchensemble`'s Bagging / Boosting / Fusion / Adversarial / FastGeometric strategies.
- Cross-run ensembling as a distinct algorithm type (it is just `dojo ensemble` with explicit candidate sources).

Deferred-feature backlog

The following are out of scope for the initial implementation but are intentionally preserved as deferred backlog items. Deferred features are **absent from the strict (`extra="forbid"`) schema**, so configuring one fails generic validation at load — there are no reserved config slots, runtime stubs, or per-feature tests. See `appendix-deferred-features.md` and `12-validation-testing-and-preflight.md`.

- Aim logger sink. Only the `local` sink is functional; metrics and figures are recorded locally for the foreseeable future.
- MLflow logger sink same.
- True multilabel support (`multilabel_classification`). Multi-head multiclass is functional and does not depend on this.
- Non-`dino_v2` SSL methods (SimCLR, VICReg, PMSN, original DINO).

- Weight-space ensembles (model soup, greedy soup, uniform soup, SWA, EMA).
- Weighted ensemble combine modes.
- `prediction_trimmed_mean`.
- WebDataset dataset backend.
- Bayesian / AutoML hyperparameter search.
- Registry-based ensemble candidate discovery (the **discovery mechanism**; cross-run ensembling itself is functional via explicit candidate sources).
- HDF / HDF5 derived result exports.
- Tabular-only model schema.
- Expanded tabular encoder families beyond `identity`, `linear`, and `mlp`.

Workplan framing

Use **"initial implementation"**, **"deferred-feature backlog"**, and **"workplan"** language consistently. Do not use "phase 1", "first refactor phase", or "post-Phase-8".

Cross-References

- `02-cli-and-task-types.md` — canonical CLI surface.
- `03-configuration.md` — canonical config tree.
- `11-dependencies.md` — base + extras layout.
- `13-workplan.md` — priority order for the new `src/dojo` implementation.
- `appendix-deferred-features.md` — deferred-feature roadmap (absent from the strict schema).
- `glossary.md` — terminology.

02. CLI and Task Types

Purpose

Defines the canonical CLI command surface, how `task.type` selects training behavior, and the relationship between subcommands and config-first usage. This file is the single source of truth for command names; other files link here rather than re-listing commands.

Command families

Top-level groups:

- `dojo init`
- `dojo train`
- `dojo infer`
- `dojo eval`
- `dojo inspect`
- `dojo ensemble`
- `dojo sweep`
- `dojo export`

Subcommands are config-first shorthands that constrain the target output.

```
dojo init
dojo train                                  # task type comes from task.type
dojo infer
dojo infer predictions
dojo infer embeddings
dojo eval
dojo eval holdout
dojo eval representation
dojo inspect
```

```
dojo inspect config
dojo inspect dataset
dojo inspect backbone
dojo inspect checkpoint
dojo ensemble
dojo ensemble candidates
dojo sweep
dojo sweep prepare
dojo sweep train
dojo sweep status
dojo sweep report
dojo export
```

There are **no** `dojo train supervised`, `dojo train ssl`, or `dojo train-snapshot-ensemble` subcommands. Training paradigm is selected via `task.type` (see below).

Invoking `dojo infer`, `dojo eval`, `dojo inspect`, or `dojo sweep` with no subcommand prints that group's help and exits — they have no default subcommand. `dojo ensemble` is the exception: bare `dojo ensemble` runs an ensemble, with `dojo ensemble candidates` as its only subcommand. `dojo init`, `dojo train`, and `dojo export` are leaf commands.

CLI architecture and execution model

Dojo's CLI is a **Typer** application (git-style subcommands, `--options`, rich help). `dojo init` bootstraps local project files. All other config-aware commands compose configs through the **Hydra Compose API** (`hydra.compose`). Dojo does **not** use `@hydra.main`, and it does not use Hydra's launcher / sweeper plugins. Sweep expansion, runtime-ID generation, output-directory resolution, and run-directory collision handling are all owned by Dojo (see `09-sweeps-and-batch-runs.md`).

Config overrides vs. command options

Every command line carries two distinct kinds of token, separated by a simple lexical rule — **every Hydra override is dash-free, every Typer option starts with `-`**:

- **Config overrides** — dash-free `key=value` and `group=option` tokens, passed through to the Compose API; they land in the composed config tree. Examples: `experiment=ifcb/species_baseline`, `training.batch_size=64`, `model.image_input.backbone.architecture.name=resnet50`, `data=ifcb/species_manifest`.
- **Command options** — Typer `--flags` and positional arguments. They control the command itself and are **not** part of the config tree. Examples: `--check-remote`, `--stats`, `--format json`, `--output`, `--type`, `--checkpoint`.

A command mixes both freely:

```
dojo inspect dataset data=ifcb/species_manifest --stats --format json
#                    [config override]           [command options]
```

Config sources and replay modes

Config-aware commands can start from authored, composed, or resolved config inputs:

- **Config group selectors** — dash-free Hydra selectors such as `experiment=ifcb/species_baseline`. These resolve from the active config search path: explicit `--config-dir` entries, then local `./configs` when present, then packaged read-only Dojo configs.
- **Authored root file** — `--config FILE` loads one human-written root YAML file and then applies CLI overrides. This is useful for project-local configs that are not arranged as Hydra groups.
- **Composed config artifact** — `--config RUN_DIR/config/composed.yaml` replays composed authored intent and still performs fresh runtime resolution: new generated values, output directories, and frozen-stat checks as appropriate for the command.

- **Resolved config artifact** — `--resolved-config RUN_DIR/config/resolved.yaml` loads a fully resolved run artifact. Read-only inspection commands may consume it directly. Mutating commands such as `dojo train` may execute it directly when the referenced run directory is absent, empty, or only contains prepared config artifacts. If the rest of the run folder is not empty, the command requires an explicit override: `--resume` to continue the same run context, or `--clobber` to delete the existing directory contents and run the resolved config in place. To branch instead, copy the composed / resolved config to a new location, edit its output `dir` / `dir_template`, and run that as a fresh config. When output blocks share a physical directory (e.g. the snapshot-ensemble case), `--clobber` clears each resolved physical directory once at startup, so later phases of the same command do not re-clear it.

`--config`, `--resolved-config`, and Hydra group selectors are mutually exclusive as root config sources, though ordinary value overrides may still be applied where the command permits them.

Command I/O is options, not config

Per-invocation inputs and outputs are command options, never config keys: `--checkpoint`, `--model`, `--output`, `--type`, `--ensemble-manifest`. They do not appear in the canonical root shape (`03-configuration.md`).

Commands select which config blocks apply

Dojo uses **one** root config schema for every command. The **command**, not `task.type`, decides which blocks are active: `dojo train` consumes `model` / `training` / `optimizer` / `scheduler` / `objectives` / `checkpointing` (with `task.type` selecting the training paradigm among `supervised` / `ssl` / `snapshot_ensemble`); `dojo infer` and `dojo eval holdout` consume `data` / `runtime` / `storage` / `eval_outputs` and take the model and its preprocessing from the artifact's embedded inference contract (`06-results-artifacts-and-metadata.md`); `dojo eval representation` additionally activates `model` / `transforms` / `representation_eval` because it builds an encoder and configured probes; `dojo ensemble` consumes `ensemble` / `ensemble_outputs` and, when member inference is possible or required (`inference_as_needed` / `force_inference`), also consumes `data` / `runtime` / `storage` for the target dataset and execution context; `dojo sweep` consumes `sweep` / `sweep_outputs`. `inference` and `eval` are **not** `task.type` values — they are commands, exactly like `ensemble` and `export`.

Per-operation configs are the documented norm, but a single "loaded" lifecycle config carrying blocks for several commands is valid: each command consumes its slice and **warns** about blocks it does not use rather than failing. When a config carries more than the active command needs, `config_hash` is computed over that command's active block-set only, so editing an unused block never changes the job's identity (`06-results-artifacts-and-metadata.md`). A command's **active block-set** is the blocks it consumes (enumerated above) minus the global `config_hash` exclusions — runtime-resolved values, output paths, `output_root`, and the `*_outputs` blocks. So an `infer` / `eval holdout` job hashes `data`; a `train` job hashes its full model / training block-set; an `eval representation` job additionally includes `model` / `transforms` / `representation_eval`.

Two ways a checkpoint enters a command

- **Building a model for the run** → config. A checkpoint that initializes the model (transfer learning, representation eval against an encoder) is `model.image_input.backbone.weights.source: checkpoint` + `model.image_input.backbone.weights.uri` — composed, validated, and recorded in the run's config provenance. See `05-models-training-and-heads.md`.
- **Consuming a complete model artifact** → command option. `dojo export`, `dojo inspect checkpoint`, `dojo infer`, and `dojo eval` take a finished model as `--checkpoint` (a `.ckpt`) or `--model` (an exported `.pt` / `.onnx`, with `--export-config` for an ONNX metadata sidecar). The model and its input pipeline self-describe from the artifact's embedded inference contract (`06-results-artifacts-and-metadata.md`), so no `resolved.yaml` is required.

Sweeps are prepared by `dojo sweep`

There is no `-m` / `--multirun` flag. `dojo train` executes one concrete training run and rejects authored / composed configs with an active `sweep.grid`. Use `dojo sweep prepare` to compose a config whose `sweep:` block defines axes (`sweep.mode: grid` with a non-empty `sweep.grid`) and write the sweep directory, manifest, and per-run resolved configs.

```
dojo sweep prepare experiment=ifcb/baseline \
  'sweep.grid.model.image_input.backbone.architecture.name=[efficientnet_b0,efficientnet_b1]'
```

Initial manual sweep workflow:

```
dojo sweep prepare experiment=ifcb/baseline \
  'sweep.grid.optimizer.lr=[1e-4,3e-4]'
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --index 0
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --run-id RUN_ID
dojo sweep status ./runs/ifcb/sweep_results/SWEEP_ID
dojo sweep status ./runs/ifcb/sweep_results/SWEEP_ID --index 0
dojo sweep report ./runs/ifcb/sweep_results/SWEEP_ID
```

`dojo sweep train` is a convenience wrapper for training jobs in a prepared sweep. It reads the sweep manifest, selects one job by `--index` or `--run-id`, and runs the same internal training path as `dojo train --resolved-config JOB_DIR/config/resolved.yaml`. The lower-level `dojo train --resolved-config ...` path remains valid.

`dojo sweep status` reads the sweep manifest and per-run status files. With no selector, it lists every sweep index and `run_id`; with `--index` or `--run-id`, it reports one job. `dojo sweep report` reads the same manifest and writes sweep-level metrics, figures, and promoted artifacts according to `sweep_outputs`.

`SWEEP_DIR` is the canonical target for `dojo sweep train`, `status`, and `report`. For config-parity with other commands, they may also accept `--resolved-config SWEEP_DIR/config/resolved.yaml`. The standalone `sweep_manifest.json` path is an internal artifact, not a top-level CLI input in the initial contract. See `09-sweeps-and-batch-runs.md` for the sweep block, manifest, and authored -> resolved pipeline.

dojo init

`dojo init` bootstraps an editable local project from packaged Dojo config templates. It is additive by default: missing files are created and existing files are skipped. It writes nothing when `--dry-run` is used, and overwrites existing files only with `--clobber`.

```
dojo init
dojo init ./my-dojo-project --minimal --supervised --data
dojo init --ssl --sweep
dojo init --all
dojo init experiment=ifcb/species_baseline
dojo init --config ./my_experiment.yaml
```

The optional positional argument is the project directory, default `.`. Configs are written under `<project>/configs`. `--data` materializes a small packaged fixture dataset under `<project>/example-data` and writes local data config entries that point at the materialized files, so the starter experiment can run without separate dataset setup.

Initial options:

- `--minimal` — copy only selected starter roots and their referenced config dependency closure.
- `--supervised` — include supervised training templates and starter experiments.
- `--ssl` — include SSL / DINOv2 templates and starter experiments.
- `--sweep` — include grid-sweep templates and starter experiments.
- `--data` — materialize the small packaged fixture dataset and matching local data config.
- `--all` — copy the whole packaged config tree and materialize the fixture dataset (it includes `--data`).
- `--config FILE` — use a concrete authored YAML file as a materialization root, copying any referenced packaged configs needed by that file.
- `--dry-run` — report create / skip / overwrite actions without writing.
- `--clobber` — overwrite existing files.

Scope flags are additive. If no scope flag or explicit materialization root is provided, `dojo init` defaults to `--minimal --supervised`. `--all` is mutually exclusive with `--minimal`, scope flags, and explicit materialization roots. Dash-free config selectors such as `experiment=ifcb/species_baseline` are materialization roots for `dojo init`: Dojo copies that selected packaged config and any packaged configs it references into the local project.

Task types

`task.type` is the axis that determines what `dojo train` does. Supported types in the initial implementation:

- `supervised`
- `ssl`
- `snapshot_ensemble`

`supervised`

Standard supervised training, single- or multi-head, with optional tabular input encoder, implicit input concatenation, and embedding adapter. See `05-models-training-and-heads.md`.

`ssl`

Self-supervised training. The functional method is `dino_v2` via Lightly. Other SSL methods (SimCLR, VICReg, PMSN, original DINO) are deferred and not schema values; authoring them fails validation — see `appendix-deferred-features.md`.

`snapshot_ensemble`

Single-command convenience that orchestrates two steps against one shared run directory:

1. Supervised training with a snapshot-cycle scheduler (cosine warm restarts producing one checkpoint per cycle).
2. Ensembling the just-finished snapshot checkpoints through the ensemble pipeline.

Snapshot checkpoints land in `<training_outputs.dir>/checkpoints/`. Ensemble artifacts land under `ensemble_outputs/` sub-directories (default `ensemble_results/` and `ensemble_figures/`, controlled by `ensemble_outputs.results.dir` and `ensemble_outputs.figures.dir`).

A `task.type: snapshot_ensemble` config must include a `training:` block, a snapshot-capable `scheduler:` block (a top-level peer of `training:`, e.g. `cosine_warm_restarts`), and an `ensemble:` block (selection strategy + combine modes). Pydantic validation enforces all three.

Canonical invocation:

```
dojo train experiment=ifcb/snapshot_experiment
```

where the experiment config sets `task.type: snapshot_ensemble`.

Re-running ensembling against a historical training run is not a `task.type: snapshot_ensemble` invocation. It is a regular `dojo ensemble` invocation with a `run_dir` or `run_dir_glob` candidate source pointing at the historical run directory.

Example `task.type: snapshot_ensemble` config:

```yaml
task:
  type: snapshot_ensemble

training:
  max_epochs: 300

scheduler:
  name: cosine_warm_restarts
  first_cycle_epochs: 50
  cycle_mult: 1.0
  max_lr: 1.0e-4
  min_lr: 1.0e-6
  warmup_epochs: 5

checkpointing:
  save_cycle_snapshots:
```

```yaml
    enabled: true
    at_cycle_end: true

ensemble:
  selection:
    strategy: all          # candidates are the run's cycle-end snapshots (implicit source)
  inference:
    combine:
      classification: probabilities_mean
```

`dojo inspect config`

Replaces the old `dojo validate-config`. Behavior:

- compose configs;
- apply CLI overrides;
- run schema validation;
- render output-path templates;
- show expected output folder structure;
- warn about run/sweep directory collisions;
- show enabled outputs;
- optionally emit machine-readable JSON for CI/tests.

Remote-artifact validation strength

Default: **schema-and-local feasibility only**. Resolve paths, validate format of locally-accessible artifacts; do **not** require network access to remote URIs. CI-safe and offline-friendly.

Opt-in `--check-remote` command option performs HEAD requests / etag checks against remote URIs (S3 manifests, checkpoints, etc.) and reports availability and size. It is a command option only — there is no `inspect.check_remote` config key. Without `--check-remote`, missing-remote-artifact errors surface at runtime, not at inspect time. This is intentional.

`dojo inspect config` is not a dataset-stats producer. It never computes missing frozen dataset statistics and never updates the stats cache. When a config requires frozen stats (`normalize: {mode: dataset}`, tabular imputation / normalization, target transforms, cached lengths, or count-dependent weighting), inspect config validates the configured cache shape and any locally available cache metadata. Without `--check-remote`, remote cache contents are not fetched; missing or stale remote cache contents surface when run resolution / preflight actually consumes them. Use `dojo inspect dataset` with the needed aspect flags, or `--stats`, to create or refresh those values.

`dojo inspect dataset`

Replaces the old `dojo tools make-manifest`. Read-only by default. May write canonical CSV / Parquet manifests when an output path is explicitly configured. Class-folder manifest generation is folded in: `class_folder` is an inspect-dataset source, **not** a train / eval / infer backend.

Inspect outputs are not normal run outputs and do not require a run directory.

With no aspect flag it reports missing targets per target and split, including `missing_count`, `valid_count`, `drop_sample_count`, `mask_objective_count`, and whether the configured `missing_policy` would fail validation (default policy: `error`). For multi-head configs this report is per target, because `mask_objective` can keep a partially labeled sample usable for other objectives.

Aspect flags scope **both what is computed and the I/O cost paid**, so a header-level histogram never triggers a full pixel decode. Each aspect is either a **frozen** value (deterministic, written to the dataset stats cache, consumed at config resolution, and hashed) or an **advisory** report (human-facing, may be sampled, never cached or hashed). Frozen fit statistics are computed on the `train` split; structural properties are computed across all splits. The stats cache is defined in `04-data-and-storage.md`.

Tier 0 — manifest only (no image I/O):

Flag	Computes	Kind
`--targets`	per-target missing / valid / drop / mask / validation-failure counts (default)	advisory
`--class-counts`	per-class counts + frequencies	frozen
`--class-map`	resolved index <-> label mapping	frozen
`--target-stats`	regression / ordinal target distribution + fitted transform stats (standardize mean / std, Box-Cox lambda)	frozen
`--tabular-stats`	numeric tabular normalization stats + imputation fill values; categorical vocabularies	frozen
`--imbalance`	imbalance ratio, empty-class, non-contiguous-index checks	advisory

Tier 1 — image headers only (`imagesize` / lazy open, no pixel decode):

Flag	Computes	Kind
`--dimensions`	per-sample native width / height (→ aspect ratio, long side), cached as the primitive for bucket design and assignment	frozen (structural)
`--bucket-histogram`	aspect-ratio + long-side distribution; suggested bucket boundaries — derived from cached `--dimensions` (reads headers only if absent)	advisory
`--bit-depth`	resolve `input_bit_depth`; flag heterogeneous depths	frozen value + advisory warning

Tier 2 — full pixel decode / full byte read (opt-in, expensive):

Flag	Computes	Kind
`--normalization`	per-channel mean / std (streaming) for `normalize: {mode: dataset}`	frozen
`--content-hash`	true `dataset_content_hash` over image bytes plus declared tabular feature values when present; folds into any full pass for free (see `04-data-and-storage.md`)	frozen (verification)

Backend-specific:

Flag	Computes	Kind
`--bin-lengths`	ROI counts per bin → `bin_lengths` cache (`ifcb_bins` only)	frozen

Compound and control flags:

- `--stats` — compute the **cheap** frozen aspects (Tier 0 manifest + Tier 1 header level): tabular and target stats, imputation, categorical vocabularies, `class-map` / `class-counts`, `dimensions`, `bit-depth`, and (`ifcb_bins`) `bin-lengths`. Results are displayed (per `--format`) and, when `data.stats_cache_uri` is configured, written to that cache; with no cache configured it is display-only. The decode-tier frozen aspects are **not** run by `--stats` alone because they need a full pixel pass: add `--normalization` (the producer for `normalize: {mode: dataset}`) or use `--all`. A bare `--stats` reads no pixels and never writes `dataset_content_hash`; that hash is recorded only when a full pass actually runs (`--normalization` / `--content-hash` / `--all`, see `04-data-and-storage.md`).
- `--report` — run all advisory aspects, print / write a human report, write no cache.
- `--all` — every aspect, including the decode tier (warns: incurs a full-decode pass; this is what also records `dataset_content_hash`).
- `--split train|val|all` — override split selection; otherwise fit statistics default to `train` and structural properties to `all`.
- `--sample N|FRACTION` — estimate from a subsample. Permitted for advisory aspects; for frozen aspects it marks the result `estimated: true` so a sampled statistic is never silently frozen into the contract.
- `--format text|chart|json` — advisory output rendering. `chart` (default in an interactive terminal) draws in-terminal bar charts / histograms for distributions (`--bucket-histogram`, `--class-counts`, `--imbalance`, target / tabular distributions); `text` is plain tabular; `json` is machine-readable. Frozen values are displayed too, and — when `data.stats_cache_uri` is configured — also written to the cache regardless of `--format`.
- `--output PATH` — write a canonical manifest (incl. `class_folder` scan). A command option, not a config key.

When tiers stack (e.g. `--normalization --bit-depth --bucket-histogram`), images are opened once and all requested aspects are collected in that single pass; the decode tier subsumes the header tier.

Examples:

```
# fast, header-only bucket advisory (sub-second with --sample)
dojo inspect dataset data=ifcb/species_manifest --bucket-histogram

# write the frozen stats cache (data.stats_cache_uri) consumed at training time
# (add --normalization for dataset normalization mean/std; bare --stats reads no pixels)
dojo inspect dataset data=ifcb/species_manifest --normalization --stats

# canonical manifest from a class-folder source
dojo inspect dataset data=ifcb/species_manifest \
  --output ./inspect_outputs/species_manifest.parquet
```

`dojo inspect backbone`

Reports per-module shape, parameter count, trainable/frozen status, and output embedding dim for the configured backbone with freeze policy applied. Useful for picking `named_modules_trainable` / freeze module names. Example:

```
dojo inspect backbone experiment=ifcb/baseline
```

This composes the experiment config, then inspects `model.image_input.backbone` with `training.freeze.backbone` applied.

```
dojo inspect backbone \
  model.image_input.backbone.architecture.source=torchvision \
  model.image_input.backbone.architecture.name=resnet50
```

```
dojo inspect backbone \
  model.image_input.backbone.architecture.source=torchvision \
  model.image_input.backbone.architecture.name=resnet50 \
  training.freeze.backbone.policy=after_module_trainable \
  training.freeze.backbone.module=layer3 \
  training.freeze.backbone.inclusive=true
```

dojo inspect checkpoint

Reports the checkpoint's embedded inference contract — compatibility hashes, head / preprocessing configuration (see "Portable inference contract", 06-results-artifacts-and-metadata.md) — and the checkpoint-hash filename match. This command requires the Torch stack (image_classifier_dojo[train]) for .ckpt deserialization; base installs do not inspect Lightning checkpoints.

dojo infer

dojo infer predictions writes per-sample prediction rows (canonical result schema, stage=infer); dojo infer embeddings extracts embeddings. Both take the model as a --checkpoint (.ckpt) or --model (.pt / .onnx) artifact and rebuild the model + input pipeline from its embedded inference contract; they never re-author transforms: (the artifact's inference_pipeline is authoritative). The dataset to run on is the only required addition: a data=<group> selector, an authored data: block, or the --input PATH shorthand (backend inferred from the file, default columns, split=all). data.targets are optional for inference.

Per-invocation output selection is command options, not config: --output, --format parquet|csv, --heads, --embeddings <kind>. Durable output configuration (directory, partitioning) lives in eval_outputs (03-configuration.md).

dojo infer embeddings is the canonical embedding-extraction command; the legacy dojo eval embeddings name is removed.

dojo eval

dojo eval scores a finished model against a **labeled** dataset, outside any training run — e.g. when fresh labeled data arrives and you want metrics without retraining.

- dojo eval holdout — evaluate against a holdout split (stage=holdout_eval). Takes the model as --checkpoint / --model and rebuilds everything from the inference contract; it **requires** data.targets and computes metrics from the artifact's objective_summary plus target_schema (no objectives: block needed).
- dojo eval representation — standalone representation evaluation (stage=representation_eval). This **builds an encoder** from model.image_input.backbone.weights.source: checkpoint and attaches the probes / projections / clustering configured under representation_eval:, so it activates model / transforms / representation_eval rather than consuming a finished-model artifact. See 07-ssl-and-representation-eval.md.

Results, metrics, and an always-written **eval manifest** (model source + hashes, dataset identity + split, metric summary) land in eval_outputs. By default an eval run is standalone with its own run_id; it can instead co-locate beside the source run by templating eval_outputs.dir_template with {source_run_dir} (03-configuration.md). The manifest lets a later dojo ensemble or report step consume eval runs as a candidate / result source.

dojo eval knn and dojo eval linear-probe are not primary commands in the initial implementation; that functionality lives inside representation_eval config sub-blocks accessed through dojo eval representation or scheduled during training.

dojo ensemble

- dojo ensemble — ensemble evaluation / inference against a candidate manifest or candidate-discovery sources.
- dojo ensemble candidates — artifact-inspection / manifest-writing command. Does not initialize experiment logging.

Cross-run ensembling is not a distinct mode. It works via dojo ensemble with explicit candidate sources such as run_dir and run_dir_glob. See 08-ensembles.md.

dojo sweep

- dojo sweep prepare — compose an authored sweep config, expand sweep.grid, write the sweep directory, immutable manifest, and per-run resolved configs.
- dojo sweep train — execute one prepared training job selected by --index or --run-id.

- `dojo sweep status` — inspect all prepared jobs, or one job selected by `--index` / `--run-id`, from the manifest plus per-run status files.
- `dojo sweep report` — after required jobs are done, write sweep-level metrics, figures, and exports according to `sweep_outputs`.

Automated local sequential and Slurm execution are deferred; the initial `sweep.execution.mode` is `manual`.

`dojo export`

Explicit export from a checkpoint, manifest, or already-existing run output. Artifact types: `torchscript` and `onnx`. See `10-export.md`.

Config-first usage

Config-aware execution and inspection commands accept dash-free Hydra config overrides; command I/O is expressed as `--options` (see "CLI architecture and execution model" above). Examples:

```
dojo inspect config experiment=ifcb/species_baseline training.batch_size=64
dojo inspect dataset data=ifcb/species_manifest --output ./inspect_outputs/species_manifest.parquet
dojo train experiment=ifcb/species_baseline
dojo infer embeddings experiment=ifcb/species_baseline --checkpoint ./runs/baseline/checkpoints/best.ckpt
dojo eval representation experiment=ifcb/dinov2_repr_eval
dojo ensemble candidates experiment=ifcb/ensemble_candidates ensemble_outputs.manifests.dir=./shared_manifes
dojo ensemble \
  experiment=ifcb/ensemble_search \
  ensemble.candidates.sources.0.type=manifest \
  ensemble.candidates.sources.0.manifest_uri=./shared_manifests/ifcb_candidates.json
dojo sweep prepare experiment=ifcb/baseline \
  'sweep.grid.optimizer.lr=[1e-4,3e-4]'
```

Cross-References

- `03-configuration.md` — config-key surface every command consumes.
- `04-data-and-storage.md` — dataset backends and `dojo inspect dataset` behavior.
- `05-models-training-and-heads.md` — `task.type: supervised` and `task.type: snapshot_ensemble` training internals.
- `07-ssl-and-representation-eval.md` — `task.type: ssl`, `dojo eval representation`.
- `08-ensembles.md` — `dojo ensemble` / `dojo ensemble candidates` and the snapshot-ensemble orchestration step.
- `09-sweeps-and-batch-runs.md` — `dojo sweep prepare`, `dojo sweep train`, `dojo sweep status`, and `dojo sweep report`.
- `10-export.md` — `dojo export`, training / ensemble export blocks, and sweep-level artifact promotion.
- `12-validation-testing-and-preflight.md` — `dojo inspect config` validation tiers and preflight checks.
- `glossary.md` — task type and CLI vocabulary.

03. Configuration

Purpose

Defines the canonical config tree: top-level groups, the peer `*_outputs` blocks, path-resolution semantics, and run-config artifacts. This file is the canonical schema source referenced by every other file that mentions a config key.

Schema source of truth

Hydra composes configs from `configs/` groups via the Compose API (Dojo does not use `@hydra.main`). Pydantic validates the composed config and is the runtime contract. Dash-free `key=value` CLI **config overrides** apply during

composition, before Pydantic validation; dash-prefixed **command options** (`--checkpoint`, `--output`, etc.) are not part of the config tree. See `02-cli-and-task-types.md` for the overrides-vs-options model.

Strict schema, no reserved slots

The Pydantic config models are strict: every model sets `extra="forbid"`, and enum / discriminated-union fields list only **implemented** values. An unrecognized key or an unimplemented value therefore fails generic validation at config load — exactly as a typo would. There are no reserved-but-inert config slots and no runtime `NotImplementedError` stubs: a deferred feature is simply absent from the schema until it is built, and its validation error names the offending key/value without advertising it as planned. The deferred roadmap lives only in `appendix-deferred-features.md` and `13-workplan.md`.

This strictness concerns *unknown* keys and values and is orthogonal to command-gating: a loaded lifecycle config may legitimately carry blocks that are inactive for the current command (those warn, not fail — see `02-cli-and-task-types.md`), but every block and value it carries must be a recognized, implemented one. The one place curated messages remain is invalid combinations of *implemented* features (incompatible head / loss pairs, bad objective references), which are real current contract.

Config search path

Config group selectors such as `experiment=ifcb/species_baseline` resolve through this search path:

1. explicit command `--config-dir` entries, in the order provided;
2. local `./configs`, when present;
3. packaged read-only Dojo configs bundled as package resources.

Earlier entries shadow later entries. Packaged configs make pip-installed Dojo usable before a user creates a local config tree. `dojo init` materializes packaged configs into an editable local project: either a scope-selected starter set, the complete packaged config tree, or the dependency closure of a selected config root.

`--config FILE` bypasses group lookup and loads a concrete authored or composed root config file. `--resolved-config FILE` loads a saved resolved config artifact and is intentionally explicit because resolved configs contain generated values and resolved output paths. See `02-cli-and-task-types.md` for command-level replay modes.

Canonical root shape

```yaml
experiment:
task:
runtime:
storage:

data:
transforms:

model:
  image_input:
    name:
    backbone:
  tabular_input:
    name:
  embedding_adapter:
  heads:

objectives:

ssl:
representation_eval:
```

```
training:
optimizer:
scheduler:
checkpointing:

ensemble:

sweep:

output_root:

training_outputs:
  dir:
  dir_template:
  logging:
  results:
  export:
  metrics:
  figures:

ensemble_outputs:
  dir:
  dir_template:
  results:
    member_results:
      mode:
  export:
  metrics:
  figures:
  members:
  manifests:

sweep_outputs:
  dir:
  dir_template:
  enabled:
  collect:
  artifacts:
  metrics:
  figures:

eval_outputs:
  dir:
  dir_template:
  results:
  metrics:
  figures:
```

Notes:

- `runtime` holds process behavior (`seed`, `precision`, `num_workers`, `fast_dev_run`, `autobatch`, `preflight`, `run_id`, `sweep_id`). `seed` lives under `runtime`, not at top level.
- `storage` is top-level, peer to `runtime`. It does **not** live under `runtime`.
- `optimizer`, `scheduler`, `checkpointing` are top-level peers of `training`.
- `transforms` owns input preprocessing: the image pipeline plus tabular preprocessing. Config compilation derives resolved inference-time preprocessing from it.
- `model.image_input` holds the required image model-input config. `model.image_input.name` names the image

input stream and defaults to `image`.

- `model.image_input.backbone` holds the image backbone config. `model.image_input.backbone.architecture` describes the module shape, and `model.image_input.backbone.weights` describes initialization. `model.image_input.back` is the architecture selector (`resnet50`, `vit_small_patch16_224`, etc.) and must not be reused as the input-stream name.
- `model.tabular_input` holds optional tabular model-input config: input-stream name, selected logical tabular features, and encoder. `model.tabular_input.name` defaults to `tabular`. Tabular imputation, normalization, and categorical encodings live under `transforms.tabular`, not under `model.tabular_input`.
- There is no `model.fusion` config block. When both image and tabular inputs are enabled, Dojo concatenates their embeddings implicitly in canonical input order: image first, tabular second. With one enabled input, no concatenation node is included in the graph. In the initial implementation, image input is required and tabular input is optional; tabular-only schema is deferred.
- The old top-level `outputs:` block is gone. `training_outputs` is its replacement.
- `logging` lives under `training_outputs.logging`. Only model-training runs initialize experiment logging; `dojo ensemble` and `dojo ensemble candidates` do not.
- `ensemble` is the algorithmic block (selection / combine / candidate config); `ensemble_outputs` is the corresponding output block.
- `ensemble_outputs.results.member_results.mode` controls whether selected member prediction rows are materialized into `ensemble_results/`; the ensemble manifest is written regardless.
- `sweep` is the algorithmic block for sweep generation / search (`mode: grid` or deferred `mode: bayesian`). Manual execution is the initial contract (`sweep.execution.mode: manual`); automated runners are deferred. `sweep_outputs` is the corresponding sweep-level reporting output block.
- `eval_outputs` is the output block for `dojo infer`, `dojo eval`, and standalone `dojo eval representation`. It is standalone by default (own `run_id`); set `eval_outputs.dir_template` with `{source_run_dir}` to co-locate beside the source run. An `infer` / `eval` run always writes `eval_manifest.json` (model source, dataset identity, metric summary) and, like `dojo ensemble`, does not initialize experiment logging.

Minimal supervised example

```yaml
experiment:
  name: ifcb_species_baseline

task:
  type: supervised

runtime:
  seed: 123
  run_id: "{coolname:noseed}"
  precision: bf16-mixed
  num_workers: 8
  fast_dev_run: false

storage:
  local_cache_dir: ./.cache/dojo

data:
  backend: parquet_manifest
  manifest_uri: s3://datasets/ifcb/species_manifest.parquet
  image_uri_column: image_uri
  split_column: split
  sample_id_column: roi_id
  targets:
    species:
      column: species_idx
      type: multiclass_classification
      class_names: s3://datasets/ifcb/species_classes.json
```

```yaml
      missing_policy: error

model:
  image_input:
    name: image
    backbone:
      architecture:
        source: torchvision
        name: resnet50
        output_dim: auto
      weights:
        source: library
        name: DEFAULT
  tabular_input:
    enabled: false
  embedding_adapter:
    enabled: false
  heads:
    species:
      type: multiclass_classification
      target: species
      num_classes: 42
      network:
        type: linear

objectives:
  species:
    head: species
    loss: cross_entropy
    metrics: [accuracy, f1_macro, f1_per_class]
    weight: 1.0

training:
  max_epochs: 50
  batch_size: 64
  freeze:
    backbone:
      policy: none

optimizer:
  name: adamw
  lr: 0.0003
  weight_decay: 0.01

scheduler:
  name: cosine
  warmup_epochs: 3

checkpointing:
  monitor: val/species/f1_macro
  mode: max
  save_top_k: 3

output_root: ./runs

training_outputs:
```

```
dir_template: "{experiment.name}/{runtime.run_id}"
```

Output paths and directory resolution

`output_root` is a single filepath string used as the base for rendered `training_outputs.dir_template`, `ensemble_outputs.dir_template`, `sweep_outputs.dir_template`, and `eval_outputs.dir_template`, and for bare-relative top-level `*_outputs.dir` values.

Each `*_outputs` block has its own concrete `dir` or `dir_template`. When `dir_template` is used, the template resolves to the corresponding `*_outputs.dir` value under `output_root`.

For any `dir` key (top-level or sub-block):

- Absolute paths (`/folder`) are used as provided.
- `./folder` is resolved relative to process CWD.
- Bare-relative paths are resolved against the parent object's resolved `dir`. For top-level `*_outputs.dir`, the parent base is `output_root`. For sub-block values such as `results.dir: folder` under `training_outputs`, the parent base is `training_outputs.dir`.

Path template syntax

`dir_template` values use Dojo-owned `{...}` tokens, resolved by Dojo (not OmegaConf `${...}`). The token set is **open**: a token is either a dotted path into the resolved config or one of a small fixed set of special tokens.

- **Config-path tokens** — any dotted path into the resolved config; the token renders the resolved value at that path. Examples: `{experiment.name}`, `{runtime.run_id}`, `{runtime.sweep_id}`, `{training.batch_size}`, `{optimizer.lr}`, `{model.image_input.backbone.architecture.name}`.
- **Special tokens** (not config paths):
  - `{coolname:<source>}` — a coolname generated from an explicit seed source, usually a hash such as `{coolname:config_hash}`, `{coolname:model_hash}`, `{coolname:ensemble_hash}`, or `{coolname:sweep_hash}`. `{coolname:noseed}` generates a fresh unseeded name. Bare `{coolname}` is deterministic from `runtime.seed` and should not be used for unique run directories. See `06-results-artifacts-and-metadata.md`.
  - `{timestamp}` — the run's start time as a filesystem-safe string (e.g. `2026-06-26_14-30-05`).
  - `{job_num}` — Dojo's per-run sweep index (`sweep.active_run.index`, see `09-sweeps-and-batch-runs.md`), the per-run position within a sweep. Not Hydra's `hydra.job.num` (Dojo does not use `@hydra.main`).
  - `{ensemble_id}` — the generated selected-ensemble id, available after candidate discovery and selection (see `08-ensembles.md`).
  - `{source_run_dir}` — the resolved run directory of the model artifact passed to `dojo infer` / `dojo eval` (the `--checkpoint` / `--model` source). Available only when that artifact lives inside a Dojo run directory; co-locating eval output errors clearly for export-only or remote artifacts that have no source run directory.
  - `{dataset_id}` — the evaluation dataset's `dataset_id`, falling back to a short `dataset_hash` form when the dataset does not self-name.

Format specifiers A token may carry a `:spec` suffix applied to its resolved value:

- `:slug` — a Dojo **custom** formatter that produces a filesystem-safe token: characters in `[a-zA-Z0-9-_.]` are kept as-is (so `{optimizer.lr:slug}` renders `0.0003` unchanged and `{model.image_input.backbone.architecture.name:slug}` keeps `resnet50`), spaces become `-`, and every other character becomes `_`. Uses the `python-slugify` internally.
- Any standard Python format spec works normally, e.g. `{training.batch_size:03}` → `032` and `{optimizer.lr:.0e}` → `3e-04`.

For the special `{coolname:<source>}` token, the suffix names the coolname seed source rather than a value formatter.

Template rendering has two phases. ID templates that create generated runtime or artifact values (`runtime.run_id`, `runtime.sweep_id`, `config_id`, `model_id`, `ensemble_id`) render during ID generation; if they use `{coolname:<hash>}`, the referenced hash is computed first and the generated ID does not feed back into that hash. Output/path templates render after ID generation, so generated `run_id` / `sweep_id` / `ensemble_id` values are available. Use this syntax over OmegaConf `${...}` for string composition in configs, typically output paths. See `09-sweeps-and-batch-runs.md` for the unified runtime ID and output resolution order.

For ensemble runs that use `{ensemble_id}`, `ensemble_outputs.dir_template` resolves after candidate discovery, compatibility validation, selection, and selected-ensemble identity generation. The discovered candidate audit and selected member list are execution artifacts recorded in the ensemble manifest, not authored-config entries.

Snapshot ensemble directory sharing

For `task.type: snapshot_ensemble`, both `training_outputs.dir_template` and `ensemble_outputs.dir_template` typically resolve to the same directory. The recommended pattern is to default `ensemble_outputs.dir_template` to match `training_outputs.dir_template`.

Run directory collisions

A run refuses to start when its resolved output directory already contains content beyond prepared `config/` artifacts. The guard is **uniform** — it applies to every run whether the config was freshly composed or replayed from `--resolved-config`. Freshly composed runs rarely trip it because `runtime.run_id` renders a unique value per run (e.g. `{coolname:noseed}`), so each resolves to its own directory; replaying a *fixed* resolved config deliberately targets the same directory and is the common collision case.

Two command options override the guard (see `02-cli-and-task-types.md`):

- `--clobber` — delete the resolved directory contents and run in place. Available to any run.
- `--resume` — continue the same run context. Meaningful only when replaying a `--resolved-config` run with partial artifacts to resume.

To branch a run, copy its `config/composed.yaml` to a new location, edit the output `dir` / `dir_template`, and run that as a fresh config.

When multiple output blocks resolve to the same physical directory (e.g. the snapshot-ensemble case), `--clobber` clears each resolved physical directory **once** at command startup; later phases of the same command must not re-clear directories the earlier phases just wrote.

Sweep output reporting

`sweep_outputs.enabled` controls sweep-level reporting only. Default: `true`. When `false`, Dojo still prepares the sweep and concrete jobs may still run, but `dojo sweep report` skips sweep-level collection, summary metrics, aggregate figures, and artifact promotion.

`sweep_outputs.collect` is a list of metric / artifact collection specs used by `dojo sweep report`. Each item names the metric or artifact to collect from every concrete run and the per-run source to read. For metric specs, optional `mode` is `min` or `max` and controls sweep-level ranking / best-run selection for that collected metric:

```yaml
sweep_outputs:
  enabled: true
  collect:
    - metric: val/species/f1_macro
      source: best
      mode: max
```

Initial `source` values:

- `best` — collect the value associated with the run's best checkpoint.
- `last` — collect the final recorded value for the run.
- `all` — collect all recorded values for that metric across epochs / steps.

Sweep reporting uses the persisted sweep manifest written during sweep preparation as its run index. The manifest records each concrete run's resolved output directories and resolved config artifact paths. The reporter reads those per-run resolved configs and metric artifacts from the recorded locations rather than discovering runs by scanning directories.

Run config artifacts

Every run writes its resolved configuration into `config/` under the resolved output directory:

```
config/composed.yaml      # after config composition and CLI overrides
config/resolved.yaml      # fully resolved with generated values + defaults
config/resolved.json
config/cli.txt            # invoked command and overrides
config/overrides.txt
config/sweep_values.txt   # sweep members only
```

Generated runtime values (`run_id`, `sweep_id`, etc.) appear in the resolved-config artifacts.

Template + sweep example

```yaml
runtime:
  run_id: "{coolname:noseed}"
  sweep_id: "{coolname:sweep_hash}"

output_root: ./runs

model:
  image_input:
    backbone:
      architecture:
        source: torchvision
        name: efficientnet_b0
      weights:
        source: library

training:
  batch_size: 32

sweep:
  mode: grid
  execution:
    mode: manual
  grid:
    model.image_input.backbone.architecture.name: [efficientnet_b0, resnet50]
    training.batch_size: [32, 64]

training_outputs:
  dir_template: >-
    {experiment.name}/sweep_runs/{model.image_input.backbone.architecture.name:slug}/bs{training.batch_size

sweep_outputs:
  dir_template: "{experiment.name}/sweep_results/{runtime.sweep_id}"
  enabled: true
  collect:
    - metric: val/species/f1_macro
      source: best
      mode: max
```

Cross-References

- `02-cli-and-task-types.md` — `dojo inspect config` validates and renders this schema; `task.type` semantics.
- `06-results-artifacts-and-metadata.md` — uses `output_root` / `*_outputs` resolution rules and consumes the `results:` sub-block.

- `04-data-and-storage.md` — `data:` and `storage:` blocks.
- `05-models-training-and-heads.md` — `model:`, `transforms:`, `training:`, `optimizer:`, `scheduler:`, `checkpointing:`, `objectives:`.
- `07-ssl-and-representation-eval.md` — `ssl:` and `representation_eval:`.
- `08-ensembles.md` — `ensemble:` and `ensemble_outputs:`.
- `09-sweeps-and-batch-runs.md` — `sweep:`, `sweep_outputs:`, and Hydra integration.
- `glossary.md` — config-key vocabulary.

04. Data and Storage

Purpose

Defines supported dataset backends, the shared sample contract, manifest schemas, IFCB-specific behavior, the storage resolver, and the `dojo inspect dataset` command. The `data:` and `storage:` config blocks live at the top of the config tree (see `03-configuration.md`).

Supported dataset backends

For train / eval / infer:

- `csv_manifest`
- `parquet_manifest`
- `parquet_images` (Parquet with inlined image bytes)
- `ifcb_bins`

`class_folder` is an **`dojo inspect dataset` source only** — it is not a train / eval / infer backend. Use `dojo inspect dataset` to materialize a canonical CSV/Parquet manifest from a class-folder layout.

Old listfile dataset formats are not maintained or ported.

WebDataset is deferred — see `appendix-deferred-features.md`.

Shared sample contract

Every sample carries:

- `sample_id` (stable per-row identifier)
- `uri` (resolved image URI)
- `split` (`train`, `val`, `test`, `unlabeled`, `holdout`)
- optional `bin_id` / `bin_uri` for IFCB-derived samples
- target columns (one or more) per the head/objective configuration
- optional tabular feature columns
- optional source-extra passthrough columns

CSV / Parquet manifests

Example:

```yaml
data:
  backend: parquet_manifest
  manifest_uri: s3://datasets/ifcb/species_manifest.parquet
  stats_cache_uri: s3://datasets/ifcb/species_manifest.stats.json  # colocated with the manifest; omit for
  sample_id_column: roi_id
  image_uri_column: image_uri
  split_column: split
  source_extra_columns: [cruise_id, cast_id, instrument_id]
  tabular_feature_columns:
    - depth_m
    - temperature_c
```

```
    - salinity_psu
  targets:
    species:
      label_index_column: species_idx
      label_name_column: species_name
      type: multiclass_classification
      missing_policy: error
    biovolume:
      column: biovolume_um3
      type: regression
      missing_policy: drop_sample
```

`csv_manifest` mirrors this shape against a CSV file. `parquet_images` inlines image bytes in the same Parquet file as the manifest — preferred for test fixtures because it avoids path-resolution combinatorics.

`parquet_images` uses the same `dataset_hash` / `dataset_content_hash` contract as every other backend. Embedded image bytes are not folded into the cheap `dataset_hash`; they belong to `dataset_content_hash`. For tabular data, the available logical tabular column names / physical bindings are part of `dataset_hash`, while the per-row tabular values are part of `dataset_content_hash`.

Split assignment

Each sample's `split` (`train` / `val` / `test` / `unlabeled` / `holdout`) comes from one of two mutually exclusive sources, exactly one of which must be configured:

- `split_column` — read the split from a manifest column. The column values must already be the canonical split names above.
- `split_from_filename` — derive the split from the source file each row was read from. This covers manifest sets that shard split into separate files (a common Hugging Face / sharded-Parquet layout) and carry no split column.

`split_from_filename` maps each canonical split name to a filename glob matched against the basename of the file a row came from:

```
data:
  backend: parquet_images
  manifest_uri: ./datasets/nes-plankton/data
  file_pattern: "*.parquet"
  split_from_filename:
    train: "train-*.parquet"
    val: "validation-*.parquet"
  sample_id_column: ifcb_roi_pid
  images:
    column: image
  targets:
    species:
      label_index_column: label
      label_name_column: classname
      type: multiclass_classification
```

Rules:

- The mapping keys are canonical split names, so a `validation-*` file is assigned `split: val` — the split vocabulary is normalized at config time, not inherited from filenames.
- Patterns are matched against the file basename. A file matching no pattern contributes no labeled split and its rows are excluded; a file matching more than one pattern is a configuration error.
- `split_from_filename` only assigns the split; `file_pattern` still governs which files are discovered under `manifest_uri`. Patterns should be a subset of what `file_pattern` admits.

Tabular features

`data.tabular_feature_columns` declares the available logical tabular features in the dataset. The simple list form is shorthand for numeric features where each entry is both the logical feature name and the physical manifest column:

```yaml
data:
  tabular_feature_columns:
    - depth_m
    - temperature_c
```

P3.6 adds the explicit object form for categorical features. In object form, the mapping key is the logical feature name, `column` binds it to the physical manifest column, and `type` is `numeric` or `categorical`:

```yaml
data:
  tabular_feature_columns:
    depth_m:
      column: depth_m
      type: numeric
    instrument:
      column: instrument_id
      type: categorical
```

String-list entries resolve to object entries with `type: numeric`. Categorical feature values are canonicalized to strings before vocabulary fitting and inference-time lookup.

This block is dataset schema, not model input order and not preprocessing. `transforms.tabular` owns tabular preprocessing such as imputation, normalization, train-only augmentation, categorical encoding, and future value transforms. `model.tabular_input.columns` selects the logical tabular features consumed by a model and fixes tensor order; every selected feature must exist in `data.tabular_feature_columns`. If `model.tabular_input.columns` is omitted while tabular input is enabled, it resolves to all declared tabular features in data order.

Targets

`data.targets` declares logical data targets (named keys). Heads reference these targets by name; objectives bind heads to losses, metrics, and weights. See `05-models-training-and-heads.md` for the target → head → objective reference chain. Per-target fields:

- `label_index_column` — manifest column holding each sample's integer class index.
- `label_name_column` — manifest column holding each sample's readable class name. At least one of `label_index_column` / `label_name_column` is required. When only names are present, class indices are assigned to the distinct names alphabetically; when both are present the name column supplies the index → name mapping recorded in result metadata.
- `type` — target type (`multiclass_classification`; `regression`, `ordinal_classification`, etc. are deferred).
- `missing_policy` — label-missing behavior for this target:
  - `error` (default) — missing labels for this target in active supervised / eval splits fail preflight.
  - `drop_sample` — remove samples missing this target from the run when the target is required by an enabled objective or eval scorer.
  - `mask_objective` — keep the sample, but exclude it from losses and metrics for objectives / scorers that use this target.
- `transform` — optional target transform for regression / ordinal targets (e.g. `log1p`, `log1p_standardize`). Authored as the functional form; resolved configs add any fitted statistics (e.g. standardize mean / std) frozen at fit time. This is the single home for target transforms — objectives carry none.

`missing_policy` applies only to target labels. Missing tabular feature values are handled separately by `transforms.tabular` imputation / categorical missing-token rules, so a missing feature does not drop or mask a supervised target.

IFCB bins

```yaml
data:
  backend: ifcb_bins
  manifest_uri: s3://datasets/ifcb/bin_manifest.parquet
  bin_id_column: bin_id
  bin_uri_column: bin_uri
  split_column: split
  exclude_patterns: [bad, skip, beads, temp, data_temp]
  shuffle_buffer_size: 1000
  targets:
    species:
      label_index_column: species_idx
      label_name_column: species_name
      type: multiclass_classification
      missing_policy: error
```

IFCB-specific behavior to preserve or port through `ifcbkit`:

- blacklist / exclude filtering;
- old-schema handling;
- shuffle buffer;
- stable ROI IDs;
- optional estimated bin length (exact ROI counts come from the dataset stats cache via `--bin-lengths`).

Dojo does not reimplement IFCB raw-bin parsing — `ifcbkit` is the canonical interface.

Storage

Top-level `storage:` configures the storage resolver (peer to `runtime`, **not** under `runtime`). Storage uses `amplify-storage-utils` underneath. Dataset / training / export code should not leak `amplify-storage-utils` APIs directly — go through the Dojo storage interface.

Minimal example:

```yaml
storage:
  local_cache_dir: ./.cache/dojo
```

S3 capability comes through the base `amplify-storage-utils` dependency (installed as a git reference). There is no separate `s3` optional extra in the current `pyproject.toml`; see `11-dependencies.md`.

`dojo inspect dataset`

Read-only by default. May write canonical CSV / Parquet manifests when an output path is explicitly configured. Inspect outputs do not require a run directory.

Behavior:

- Resolves the configured backend.
- For `class_folder` source: scans the directory tree and produces a canonical manifest.
- Reports missing targets per target and split, including how many samples would fail validation, be dropped, or be masked from an objective.
- Default missing-target policy is `error` for all heads / head counts.
- Optionally writes a CSV or Parquet manifest:

```
dojo inspect dataset \
  data=ifcb/species_manifest \
  --output ./inspect_outputs/species_manifest.parquet
```

The full aspect-flag surface (`--stats`, `--normalization`, `--bucket-histogram`, `--bit-depth`, …) is documented in `02-cli-and-task-types.md`.

Dataset stats cache

Several resolved values are dataset-derived statistics computed once and frozen: image normalization mean / std (`normalize: {mode: dataset}`), numeric tabular normalization stats and imputation fill values, categorical tabular vocabularies, fitted target transform statistics, per-class counts, the resolved class map, the resolved `input_bit_depth`, per-sample native dimensions, and `ifcb_bins` bin lengths. `dojo inspect dataset --stats` is the **producer**. To stay cheap it computes only the manifest- and header-level frozen aspects — per-class counts and class map, fitted target / tabular stats, imputation fill values, categorical vocabularies, per-sample dimensions, `input_bit_depth`, and `ifcb_bins` bin lengths — fitting on the `train` split (structural properties such as bit depth and bin lengths across all splits) and writing them to a stats cache. Image normalization mean / std is a **decode-tier** frozen aspect: it is not part of a bare `--stats` and is produced by `--stats --normalization` (or `--all`), which reads every image pixel.

The cache location is `data.stats_cache_uri`, configured in the data config and conventionally **colocated with the manifest**. `dojo inspect dataset` always **displays** the computed aspects to the terminal (per `--format`) and additionally writes the frozen aspects to `data.stats_cache_uri` when that field is set; with no `stats_cache_uri` configured, `--stats` is display-only and writes nothing.

The cache is keyed by `dataset_hash`. When `dataset_hash_provenance` is strong (`manifest_content` or `uri_etag`), this auto-invalidates stale cache entries when the manifest identity changes. When provenance falls back to `uri_only`, Dojo records that weakness and cannot promise automatic stale-cache detection; production configs should use content-derived or etag-derived dataset identity for frozen stats.

Config resolution **consumes** the stats cache; it does not produce or repair it. `normalize: {mode: dataset}`, tabular imputation / normalization / categorical vocabularies, target transforms, cached `ifcb_bins` bin lengths, the count-dependent losses (`weighted_cross_entropy`, `class_balanced_effective_number`), and the `class_balanced` sampler read their frozen values from the cache instead of recomputing per run. Every dataset-derived frozen value — including `ifcb_bins` bin lengths — lives in this one cache; there is no separate per-aspect cache file.

Resolved values are materialized into the resolved config and hashed by content (`06-results-artifacts-and-metadata.md`); the cache is a production and reuse mechanism, not the hash input. Missing required cache aspects, dataset-hash mismatches, estimated values in non-sampled runs, and weak `uri_only` provenance where strict freshness is required are configuration errors with an actionable fix: run `dojo inspect dataset` with the needed frozen aspect flags, or `--stats`, to write an updated cache before training / evaluation / inference.

`dataset_hash` itself stays cheap and usually available without reading image pixels or tabular cell payloads — manifest identity / canonical manifest projection, or URI + size + etag / last-modified when available — so it remains a stable identity for the cache key, result rows, and ensemble compatibility. Because a `--normalization` / `--content-hash` pass already reads sample content, `dojo inspect dataset` additionally records a separate **`dataset_content_hash`** (a true hash over image bytes and, when present, tabular feature values) for integrity / drift verification. It is recorded alongside, never folded into, `dataset_hash`: the cheap identity must not change value depending on whether a full pass happened to run. `dataset_content_hash` catches in-place pixel or tabular-value mutation that manifest-level identity cannot see.

Stats cache format

The stats cache is a JSON document per dataset, written by `dojo inspect dataset --stats` to `data.stats_cache_uri`. Small aggregates live inline; per-sample arrays (dimensions, bin lengths) are too large for JSON and are written as Parquet sidecars referenced by URI.

```
{
  "schema_version": "1.0.0",
  "dataset_hash": "sha256:1a2b…",
  "dataset_hash_provenance": "manifest_content",
  "dataset_content_hash": "sha256:9f8e…",
  "created_by": "dojo inspect dataset",
  "dojo_version": "0.1.0",
  "splits_present": ["train", "val", "test"],
  "aspects": {
    "normalization": {
```

```
      "split": "train", "estimated": false,
      "image_mode": "grayscale_repeat3",
      "mean": [0.42, 0.42, 0.42], "std": [0.18, 0.18, 0.18]
    },
    "bit_depth": { "split": "all", "value": 8, "heterogeneous": false },
    "class_counts": {
      "split": "train", "estimated": false,
      "per_head": {
        "species": { "num_classes": 42, "total": 50000,
                     "counts": { "0": 1203, "1": 88, "2": 940 } }
      }
    },
    "target_stats": {
      "split": "train",
      "per_target": {
        "biovolume": { "transform": "log1p_standardize",
                       "fit": { "mean": 2.31, "std": 0.74 } }
      }
    },
    "tabular_stats": {
      "split": "train",
      "per_column": {
        "depth_m": {
          "type": "numeric",
          "mean": 48.2,
          "std": 31.7,
          "impute": 45.0
        },
        "instrument": {
          "type": "categorical",
          "encoding": "one_hot",
          "observed_categories": ["IFCB101", "IFCB102"],
          "vocabulary": ["IFCB101", "IFCB102", "__MISSING__", "__UNKNOWN__"],
          "missing_token": "__MISSING__",
          "unknown_token": "__UNKNOWN__"
        }
      }
    },
    "dimensions": {
      "split": "all",
      "parquet_uri": "stats/dimensions.parquet",
      "columns": ["sample_id", "native_width_px", "native_height_px"]
    },
    "bin_lengths": {
      "split": "all",
      "parquet_uri": "stats/bin_lengths.parquet",
      "columns": ["bin_id", "roi_count"]
    }
  }
}
```


Rules:

- Every aspect records the `split` it was computed on and an `estimated` flag (`true` when produced under `--sample`); resolution refuses to freeze an `estimated` value into a non-sampled run.
- `dataset_hash` is the cache key. Resolution compares it against the current dataset and refuses the cache on mismatch. It does not recompute inside config resolution; run `dojo inspect dataset` to refresh the cache.

- Aspects are independent: a cache may hold only the aspects that have been run. If a required aspect is missing, resolution fails with a message naming the missing aspect and the inspect command that produces it.
- Inline aggregates (`normalization`, `bit_depth`, `class_counts`, `target_stats`, `tabular_stats`) are the values hashed by content into the compatibility hashes; `tabular_stats` includes both numeric tabular statistics and categorical vocabularies. The per-sample Parquet sidecars (`dimensions`, `bin_lengths`) are derivation inputs (bucket assignment, sampler lengths), not hash inputs.

Aspect-bucket assignment

When `aspect_bucket` is enabled, every sample's bucket is a deterministic function of its native dimensions and the resolved bucket scheme. Per-sample dimensions are the cached primitive (`dojo inspect dataset --dimensions`), so candidate bucket schemes are evaluated from cache without re-reading images, and at run setup the resolved `aspect_bucket` assignment is materialized as a working-manifest column — computed from cached dimensions × the scheme, with no image I/O at run time. Changing the bucket scheme re-derives the column from the same cached dimensions; it never requires a rescan.

Batches must be tensor-stackable (identical H × W), so bucketing requires batching **within** a bucket. The `batch_aspect_buckets` sampler groups samples by the `aspect_bucket` column to yield size-homogeneous batches (see `05-models-training-and-heads.md`). The bucket scheme is part of `preprocessing_hash` via `inference_pipeline`, so assignments are reproducible.

Preflight in `dojo train`

Training runs the dataset preflight checks listed in `12-validation-testing-and-preflight.md` (e.g. `empty_train_classes`, `non_contiguous_class_indices`, `imbalance_ratio_gt`).

Cross-References

- `02-cli-and-task-types.md` — `dojo inspect dataset` and related commands.
- `03-configuration.md` — placement of `data:` and `storage:` blocks.
- `05-models-training-and-heads.md` — target / head / objective reference chain, tabular input composition.
- `06-results-artifacts-and-metadata.md` — `source_extra_json` and `tabular_features_json` on `sample_metadata` rows.
- `11-dependencies.md` — `ifcb` optional extra; S3 via the base `amplify-storage-utils` dependency.
- `12-validation-testing-and-preflight.md` — preflight checks.
- `appendix-deferred-features.md` — WebDataset.
- `glossary.md` — `split`, `sample_id`, dataset-backend vocabulary.

05. Models, Training, and Heads

Purpose

Defines the model composition pipeline (transforms → backbone → optional tabular input encoder → implicit input concatenation → optional embedding adapter → heads), the head / objective contract, the supervised training LightningModule, optimizer / scheduler / checkpointing config, and supervised transfer learning. This file absorbs what the original draft split between transforms, backbones, heads, objectives, and supervised training.

Transforms and preprocessing

Avoid hard-coding domain-specific transform modules where YAML composition can express the behavior.

In this section, `transforms.pipeline` is the **image** pipeline. Tabular preprocessing uses the separate `transforms.tabular` block defined in "Tabular input and implicit concatenation" below.

```
Python transform modules = reusable operations
YAML configs             = experiment/domain-specific recipes
```

Canonical image transform step names:

```
letterbox
aspect_bucket
foreground_crop
grayscale
normalize
crop
blur
noise
rotate
horizontal_flip
vertical_flip
```

Authored configs use these canonical `name` values. Resolved configs keep these same names and only materialize defaults / frozen parameters.

Pipeline example:

```yaml
transforms:
  image_mode: grayscale_repeat3

  pipeline:
    - name: foreground_crop
      enabled: true
      method: threshold_bbox
      expand_margin_fraction: 0.15

    - name: aspect_bucket
      bucket_by:
        - aspect_ratio
        - native_long_side
      buckets:
        - name: small_square
          min_aspect: 0.75
          max_aspect: 1.33
          max_native_long_side: 96
          canvas_size: [96, 96]
        - name: standard_square
          min_aspect: 0.75
          max_aspect: 1.33
          min_native_long_side: 97
          canvas_size: [224, 224]
        - name: wide
          min_aspect: 1.33
          max_aspect: 3.0
          canvas_size: [224, 448]

    - name: rotate
      mode: multiples_of_90
      p: 0.5
      train_only: true

    - name: horizontal_flip
      p: 0.5
      train_only: true

    - name: normalize
      mode: dataset
```

```
mean: [0.42, 0.42, 0.42]
std: [0.18, 0.18, 0.18]
```

The `aspect_bucket` transform buckets by aspect ratio and/or native size (`bucket_by`), supporting both aspect-ratio buckets (preserve morphology for elongated organisms) and size-aware buckets (avoid artificially upscaling tiny ROIs). Scale-related fields are recorded as `sample_metadata` columns (`native_width_px`, `native_height_px`, `resize_width_px`, `resize_height_px`, `aspect_bucket`, `microns_per_pixel`) — see `06-results-artifacts-and-metadata.md`.

Bucketed batching (`batch_aspect_buckets`)

`aspect_bucket` produces variable canvas sizes *across* buckets but a fixed size *within* a bucket. Tensors in a batch must stack to identical H × W, so a bucketed run batches **within** a single bucket rather than across — otherwise the whole point of bucketing (avoiding letterbox padding waste) is lost.

The `batch_aspect_buckets` sampler is the consumer that makes this work: it groups samples by their `aspect_bucket` assignment and emits size-homogeneous batches. That assignment is a deterministic function of each sample's native dimensions and the resolved bucket scheme, materialized as a working-manifest column at run setup from cached dimensions — so the sampler is a column lookup with **no per-batch image I/O**, and changing the bucket scheme only re-derives the column (see `04-data-and-storage.md`). The chain reads: the `aspect_bucket` transform assigns each sample an `aspect_bucket` column value, and the `batch_aspect_buckets` sampler groups batches by it.

Class-balanced sampling

The `class_balanced` sampler weights sampling toward under-represented classes using the frozen per-class counts from the dataset stats cache (`dojo inspect dataset --class-counts`, `04-data-and-storage.md`) — the same counts the weighted losses consume — so it never re-tallies the dataset at run start. Sampler selection (`class_balanced`, `batch_aspect_buckets`, `weighted`, or unsampled) is a data-loading concern. Class-balancing and bucketed batching compose with a precedence rule: a batch must stay within one `aspect_bucket`, so bucket grouping is the outer constraint and class weighting applies within each bucket.

Train-only vs. always-on steps

`transforms.pipeline` is a single ordered list so interleaving is explicit (deterministic ops and augmentation can alternate, e.g. crop → augment → resize → augment → normalize). Each step may set:

- `enabled` — global on/off (default `true`).
- `train_only` — when `true`, the step runs during `train` only and is dropped for every non-train stage (`val`, `predict`, export). Default `false` (always-on). Stochastic augmentation (`rotate`, flips, blur, noise) sets `train_only: true`; deterministic preprocessing (foreground crop, bucketed resize, normalize) leaves it `false`.

A step is augmentation by **stochastic intent**, marked per step — not by module identity. A `crop` may be a deterministic center crop (always-on) or a random crop (`train_only`), and `rotate` may be a fixed rotation or a random rotation; the flag and parameters, not the name, draw the line.

`image_mode` is a load-time channel-layout policy, not a pipeline step: it is singular, always-on, and defines the channel **contract** (count / layout) the pipeline and backbone assume. Values: `rgb` (3-channel color), `grayscale` (1-channel — requires a backbone that accepts `in_chans=1`, e.g. timm; torchvision ImageNet models expect 3 and need `grayscale_repeat3` instead), `grayscale_repeat3` (decode 1-channel, broadcast to 3 channels for an RGB-pretrained backbone). It stays a top-level `transforms` field.

The `grayscale` pipeline module is **orthogonal** to `image_mode` and is not a duplicate of it. `image_mode` sets the channel container; the `grayscale` module operates on pixel **content** — desaturating color to luminance — while leaving the channel count to `image_mode`. They compose: `image_mode: rgb` plus a `grayscale` step (always-on) gives deterministic luminance carried in 3 channels, and `image_mode: rgb` plus a `grayscale` step with `train_only: true` and a probability is random-grayscale augmentation. For inherently single-channel sources (e.g. IFCB), `image_mode: grayscale_repeat3` already yields desaturated content and the `grayscale` module is unnecessary.

Config compilation materializes a derived `inference_pipeline`: the ordered subset of `pipeline` where each step is `enabled` and not `train_only`, with parameters baked in. It is the pipeline used by every non-train

stage and by exported models, and it is the sole input the `preprocessing_hash` extractor reads for transform steps (`06-results-artifacts-and-metadata.md`). `inference_pipeline` is **resolved-only**: validation rejects it in authored configs and it must not be hand-edited. Resolved configs therefore carry both the full training `pipeline` and the derived `inference_pipeline`; SSL multi-view augmentation is configured separately under `ssl:` (`07-ssl-and-representation-eval.md`) and is not part of this pipeline. Both authored `pipeline` steps and resolved `inference_pipeline` steps use the same canonical step names; resolution never rewrites alternate spellings because alternate spellings are invalid.

Pixel value range and bit depth

The transform builder follows a fixed value-range convention, so range is never an independent config knob: **decode → scale to `[0.0, 1.0]` → `normalize`**. Raw integer pixels are scaled to floats in `[0, 1]`, then the `normalize` step applies `mean` / `std`. The model-facing range is therefore an emergent property of `normalize`, not a separate setting — `mean: 0.5, std: 0.5` yields `[-1, 1]`, ImageNet stats yield roughly `[-2, 2.6]`, and so on. Normalization must match the backbone's pretraining (ImageNet stats for torchvision / timm ImageNet weights, DINOv2's expected stats for DINOv2); there is deliberately no `pixel_range` enum. When `normalize: {mode: dataset}`, the mean / std are produced once by `dojo inspect dataset --stats --normalization` (the decode-tier pass; bare `--stats` does not read pixels), frozen into the resolved config, and read from the dataset stats cache at resolution (`04-data-and-storage.md`).

The scale-to-`[0, 1]` divisor depends on source bit depth, set with `transforms.input_bit_depth`:

```yaml
transforms:
  image_mode: grayscale_repeat3
  input_bit_depth: auto   # auto | 8 | 12 | 16 → divide by 255 / 4095 / 65535
```

Default `auto`, which resolves at config-compile time from the storage dtype (`uint8` → 8, `uint16` → 16) plus format metadata when present (e.g. TIFF `BitsPerSample`, which catches 12-bit data). The resolved value is **materialized as a concrete integer** in the resolved config — decided once, frozen, and never a per-image runtime decision, so the same raw sample always scales identically. `dojo inspect dataset --bit-depth` performs this resolution and flags heterogeneous depths (`04-data-and-storage.md`).

`auto` cannot disambiguate the one genuinely ambiguous case: a 12- (or 10-, 14-) bit image stored in a 16-bit container with no bit-depth metadata reads as `uint16` (max 65535) though its true maximum is 4095, and dividing by the wrong number silently rescales every pixel. High-bit-depth IFCB / microscopy sources of that kind must set `input_bit_depth` explicitly; content-based guessing (per-image or dataset-wide max scans) is rejected because it makes scaling data-dependent and breaks on unseen inference images.

`input_bit_depth` is a load-time decode policy — singular and always-on, like `image_mode` — and is part of the input contract, so it contributes to `preprocessing_hash` as its **resolved integer**, never as the literal `auto`.

Backbones

Contract

```python
class Backbone(nn.Module):
    output_dim: int

    def forward_features(self, x: Tensor) -> Tensor:
        ...
```

Returned tensor is usually `batch_size x embedding_dim`.

`model.image_input.name` names the image input stream and defaults to `image`. `model.image_input.backbone` holds the image backbone config. `model.image_input.backbone.architecture` describes the module shape; `model.image_input.backbone.weights` describes how that module is initialized.

This keeps the input-stream name separate from `model.image_input.backbone.architecture.name`, which is the backbone architecture selector (`resnet50`, `vit_small_patch16_224`, etc.) and is used by config templates such as `{model.image_input.backbone.architecture.name:slug}`.

Architecture and weights sources

`model.image_input.backbone.architecture.source`:

- `torchvision`
- `timm`

`model.image_input.backbone.weights.source`:

- `none` — initialize from the architecture provider's default random initialization.
- `library` — initialize from provider-native pretrained weights (`weights.name` for torchvision, provider default for timm unless a specific name is supported).
- `checkpoint` — initialize from a Dojo checkpoint / exported encoder using `weights.uri`, `weights.key`, and `weights.strict`.

Authored configs may use `weights.name: DEFAULT` as a Dojo convenience alias for provider-native library weights. For torchvision, Dojo maps it to torchvision's native `DEFAULT` weight enum for the selected architecture, then stores the concrete resolved enum/name. For timm, Dojo maps it to timm's default pretrained configuration for the selected architecture (`pretrained=True` behavior), then stores the resolved pretrained config identity (for example the resolved tag / config name / HF Hub id when available). Resolved configs, saved config artifacts, and provenance never store the moving `DEFAULT` alias.

`timm` is **functional** in the initial implementation, gated by the `timm` optional extra. The schema accepts `architecture.source: timm` regardless of install; the runtime raises a clear error if `timm` is not installed.

`architecture.source: lightly` is **not** introduced. Lightly remains an SSL framework implementation detail; SSL configs select it via `ssl.framework: lightly` while the backbone architecture is still selected through `torchvision` or `timm`. DINOv2 (Lightly) is allowed to use timm ViT backbones internally and through the public `architecture.source: timm` selector.

Examples

Torchvision:

```yaml
model:
  image_input:
    name: image
    backbone:
      architecture:
        source: torchvision
        name: resnet50
        output_dim: auto
      weights:
        source: library
        name: DEFAULT        # resolved config stores the concrete name

training:
  freeze:
    backbone:
      policy: none
```

timm:

```yaml
model:
  image_input:
    name: image
    backbone:
      architecture:
        source: timm
        name: vit_small_patch16_224
        output_dim: auto
```

```yaml
      weights:
        source: library
        name: default

training:
  freeze:
    backbone:
      policy: last_n_blocks_trainable
      n: 2
```

Checkpoint:

```yaml
model:
  image_input:
    name: image
    backbone:
      architecture:
        source: timm
        name: vit_small_patch14_dinov2
        output_dim: auto
      weights:
        source: checkpoint
        uri: s3://bucket/runs/ssl_dino_v2/exports/encoder.pt
        key: encoder_state_dict
        strict: false

training:
  freeze:
    backbone:
      policy: last_n_blocks_trainable
      n: 4
```

`backbone.architecture.output_dim: auto` is the default and should usually not be changed manually.

Freeze policies

Freeze policies live under `training.freeze.backbone`, because they control trainability and optimizer membership rather than model architecture. They are accepted for SSL and supervised training.

Named in terms of what remains trainable:

```
none
all
last_n_blocks_trainable
named_modules_trainable
named_modules_frozen
after_module_trainable
```

`dojo inspect backbone` reports per-module shape, parameter count, and trainable / frozen status with a freeze policy applied. Use it to pick module names. See `02-cli-and-task-types.md`.

Supervised transfer learning from SSL

Transfer learning from an SSL-pretrained encoder is **not a new task type**. It is plain `task.type: supervised` with `model.image_input.backbone.weights.source: checkpoint` and `weights.uri` pointing at the SSL encoder export. Freeze policy, embedding adapter, and heads are configured exactly as for any supervised run.

`weights.strict: false` maps to PyTorch's `load_state_dict(..., strict=False)` and governs **backbone** keys only. Heads in the new run come from the new `heads:` block; nothing from the source checkpoint's head ever appears in the new model.

When `weights.strict: false`, Dojo logs missing and unexpected keys clearly.

Frozen feature extractor recipe:

```yaml
model:
  image_input:
    name: image
    backbone:
      architecture:
        source: torchvision
        name: resnet50
        output_dim: auto
      weights:
        source: checkpoint
        uri: s3://bucket/runs/ssl_dino_v2/exports/encoder.pt
        key: encoder_state_dict
        strict: false

  embedding_adapter:
    enabled: true
    type: mlp
    hidden_dims: [512]
    output_dim: 256
    activation: gelu
    dropout: 0.1

training:
  freeze:
    backbone:
      policy: all
```

For a true linear-probe evaluation, leave `embedding_adapter` disabled and let the head's `network: linear` consume the raw backbone embedding.

Tabular input and implicit concatenation

Tabular features may be useful model inputs (size descriptors, depth, temperature, instrument id, etc.). Tabular preprocessing is configured under `transforms.tabular`; `model.tabular_input` declares the tabular input stream, the selected logical feature columns, and the tabular encoder. There is no `model.fusion` config block: when both image and tabular inputs are enabled, Dojo concatenates their embeddings implicitly in canonical input order, image first and tabular second.

```yaml
transforms:
  tabular:
    add_missing_indicator: true
    augmentations:
      - name: random_missing
        columns: [depth_m, temperature_c]
        p: 0.05
    columns:
      depth_m:
        imputation:
          strategy: median              # statistic can come from the stats cache
        normalization:
          type: mean_std                # resolved config adds mean/std from the stats cache
      temperature_c:
        imputation:
          strategy: median
```

```
      normalization:
        type: mean_std
    salinity_psu:
      imputation:
        strategy: constant
        fill_value: 35.0
      normalization:
        type: identity
    instrument:
      encoding:
        type: one_hot
        unknown_policy: use_unknown_token
        unknown_token: "__UNKNOWN__"
        missing_policy: use_missing_token
        missing_token: "__MISSING__"

model:
  image_input:
    name: image          # default input-stream name
    backbone:
      architecture:
        source: torchvision
        name: resnet50
      weights:
        source: library

  tabular_input:
    enabled: true
    name: tabular        # default input-stream name
    columns: [depth_m, temperature_c, salinity_psu, instrument]
    encoder:
      type: mlp
      hidden_dims: [64, 64]
      output_dim: 64

  embedding_adapter:
    enabled: true
    type: mlp
    hidden_dims: [512]
    output_dim: 256
    activation: gelu
    dropout: 0.1
```

`model.tabular_input.name` names the tabular input stream and defaults to `tabular`. The initial implementation has at most two model inputs: `model.image_input.name` (`image` by default) and `model.tabular_input.name` (`tabular` by default). Image input is required and tabular input is optional; tabular-only model schema is deferred to P4.13.

`model.tabular_input.columns` names logical tabular features declared in `data.tabular_feature_columns`, after `transforms.tabular` preprocessing. It selects the subset consumed by the model and fixes tensor order. If omitted while tabular input is enabled, config resolution expands it to all declared tabular features in data order. Resolved configs materialize the final ordered tabular input contract, including missing-indicator columns and the derived `model.tabular_input.encoder.input_dim`.

If only image input is enabled, the backbone embedding flows directly to the optional `embedding_adapter`. If image and tabular inputs are enabled, Dojo concatenates the two embeddings along the feature dimension in fixed order: image embedding first, tabular embedding second. This concatenation is identity-like and non-parametric: it learns no weights and has no authored config. Shared learned capacity after concatenation belongs in `embedding_adapter`.

Model flow:

```
image → backbone → image_embedding
tabular features → tabular_encoder → tabular_embedding
image_embedding + tabular_embedding → implicit concat → fused_input_embedding
  ↓ optional embedding_adapter → head_input_embedding
  ↓ head(s)
```

Exported model artifacts must include tabular feature names, ordering, input-stream names, encodings, normalization specs / statistics, imputation fill values, and implicit concatenation order (see `10-export.md`).

Tabular missing values

Tabular feature columns may be missing per sample (sensor dropout, unresolved joins, ROIs without co-located measurements). Because the tabular encoder needs a finite numeric tensor, missing feature values are **imputed**, not dropped. This is distinct from `data.targets.<t>.missing_policy` (`04-data-and-storage.md`), which governs missing **labels**: a missing label may drop a sample, but a missing feature is filled so the sample can still produce a prediction at inference time.

`transforms.tabular` configures train-only augmentation, numeric imputation / normalization, and categorical encoding:

- `augmentations` — optional train-only tabular perturbations. There is no per-augmentation `train_only` flag; this block is train-only by definition and is dropped from non-train stages and export inference metadata.
- initial augmentation: `random_missing`, which masks observed values to missing before imputation. It accepts `columns` and per-value probability `p`; masked values then flow through the same imputation and missing-indicator path as real missing values.
- per-column `imputation.strategy` — `mean`, `median`, or `most_frequent` (computed on the train split by `dojo inspect dataset --stats` and frozen), or `constant` with an explicit `fill_value`.
- authored configs may provide defaults plus per-column overrides, but resolved configs materialize the concrete strategy / fill value for every selected feature.
- per-column `normalization.type` — `identity` leaves the imputed value unchanged; `mean_std` standardizes as `(x - mean) / std` using train-split statistics from the stats cache. Authored configs name the normalization type; resolved configs materialize the frozen `mean` / `std` values for `mean_std`.
- `mean_std` is the canonical enum spelling. Do not use `mean-std`.
- Tabular `normalization` is scalar numeric scaling, not a general ordered transform chain. If broader value transforms are added later (`log1p`, clipping, winsorization, etc.), they should get an explicit field rather than being folded into `normalization`.
- `add_missing_indicator` — when `true`, append one synthetic binary feature per configured numeric column marking whether the original value was present (`0`) or missing-and-imputed (`1`), letting the model use missingness as a signal. The indicator set is fixed by config (all configured numeric columns), not by which columns happen to contain nulls in a given split, so the tabular input width stays reproducible across datasets. Categorical missingness is represented by the configured missing token in the one-hot vocabulary, so no extra missing-indicator column is emitted for categorical features. Because missing indicators widen the tabular encoder input, they are part of the model architecture contract (`model_config_hash`) as well as the input contract.

Statistic-based fill values, tabular normalization statistics, and categorical vocabularies are computed on the train split only by the dataset-stats cache producer and frozen, exactly like dataset image normalization mean/std. The frozen fill values, per-column strategy, categorical encodings / vocabularies, and normalization statistics are resolved preprocessing state: persisted in the config artifact, exported with portable models (`10-export.md`), and contributing to `preprocessing_hash` by value (`06-results-artifacts-and-metadata.md`).

Tabular `augmentations` remain in the resolved training config for reproducibility and contribute to `config_hash`, but they are excluded from the resolved inference preprocessing contract, export metadata, and `preprocessing_hash`.

Tabular categorical features

P3.6 initial categorical support is deliberately narrow: categorical features are transformed into one-hot numeric vectors before they reach `model.tabular_input.encoder`. Learned categorical embeddings, embedding bags, TabTransformer-style models, and other categorical-specialized encoder families are deferred with the expanded tabular encoder backlog (`appendix-deferred-features.md` P4.14).

Feature type is dataset schema. `data.tabular_feature_columns` declares each logical feature as `type: numeric` or `type: categorical`; the string-list shorthand remains numeric-only (see `04-data-and-storage.md`). A categorical feature may configure:

```yaml
transforms:
  tabular:
    columns:
      instrument:
        encoding:
          type: one_hot
          unknown_policy: use_unknown_token  # error | use_unknown_token
          unknown_token: "__UNKNOWN__"
          missing_policy: use_missing_token  # error | use_missing_token
          missing_token: "__MISSING__"
```

Rules:

- `encoding.type: one_hot` is the only initial categorical encoding.
- Categorical values are canonicalized to strings before fitting or lookup.
- `dojo inspect dataset --stats` fits categorical vocabularies on the training split and writes them to the dataset stats cache. Config resolution reads the frozen vocabulary; infer / eval / export never refit it.
- The resolved vocabulary contains observed train-split categories in deterministic sorted order, followed by `missing_token` when `missing_policy: use_missing_token`, then `unknown_token` when `unknown_policy: use_unknown_token`. Reserved token strings must not collide with observed category strings.
- `missing_policy: error` rejects null categorical values. `use_missing_token` maps nulls to the configured missing token.
- `unknown_policy: error` rejects inference-time categories not present in the frozen vocabulary. `use_unknown_token` maps them to the configured unknown token.
- Numeric imputation / normalization fields are invalid on categorical features, and categorical `encoding` is invalid on numeric features.
- Tensor order is deterministic from `model.tabular_input.columns`: each selected numeric feature emits its scalar value plus any configured numeric missing indicator; each selected categorical feature emits one one-hot vector in resolved vocabulary order. Config resolution materializes the expanded tabular input contract and `model.tabular_input.encoder.input_dim`.

Simple network specs

The same small set of simple network specs appears in several model sub-blocks. Keep the names consistent, but do not add a `network:` wrapper outside heads:

- `model.tabular_input.encoder.type`
- `model.embedding_adapter.type`
- `model.heads.<head>.network.type`

Initial values:

Type	Meaning	Initial locations
`identity`	No learned module; output is the input feature vector unchanged.	`model.tabular_input.encoder.type`

Type	Meaning	Initial locations
`linear`	One learned affine projection. No hidden layers. For heads, this means the head-specific final projection only.	`model.tabular_input.encoder.type`, `model.embedding_adapter.type`, `model.heads.<head>.network.type`
`mlp`	One or more hidden layers before the output projection; supports nonlinear feature interactions.	`model.tabular_input.encoder.type`, `model.embedding_adapter.type`, `model.heads.<head>.network.type`

For `tabular_input.encoder` and `embedding_adapter`, `linear` and `mlp` require an explicit `output_dim`. For head networks, the head type determines the final output shape, so `network` does not set `output_dim`; it only chooses whether hidden layers exist before the head-specific projection.

Heads, objectives, and the reference chain

```
objective -> head -> data target -> physical column
```

Heads define **output structure** (type, num_classes, network spec, the logical `target` they read). Objectives define **training intent** (loss, metrics, weight). Heads do **not** own loss config.

Head types

```
multiclass_classification
binary_classification
multilabel_classification          # deferred - see appendix P4.2 (not a schema member)
regression
ordinal_classification             # head predicts a discrete ordered bin
distributional_regression          # deferred - see appendix P4.9
count_regression                   # deferred - see appendix P4.9
```

`multilabel_classification` is for true multi-hot multilabel problems. It is **deferred and not a member of the head-type union** in the initial implementation, so authoring it fails schema validation as an unknown head type (see `appendix-deferred-features.md` P4.2). The old `multilabel` module (which was actually multi-head multiclass) is not ported. Multi-head multiclass uses one `multiclass_classification` head per target. The old module is preserved under `dojo_deprecated` for reference.

`distributional_regression` and `count_regression` are **deferred** head types in the initial implementation: they are not members of the head-type union, so authoring either fails schema validation as an unknown head type. Neither has a result record type yet (`06-results-artifacts-and-metadata.md` defines no `distributional_output` / `count_output`), and their dedicated losses (`gaussian_nll`, `negative_binomial_nll`, `poisson_nll`) are deferred with them. Plain `regression` is the only functional regression head type. See `appendix-deferred-features.md` P4.9.

Required head fields and network defaults

Authored configs must provide:

```
type               # one of the head types above
target             # logical data target key from data.targets
```

`network` is optional in authored configs. If omitted, config compilation injects the head type's default network spec. Resolved configs, saved config artifacts, and runtime objects always include `network`.

Type-specific required fields:

```
multiclass_classification / binary_classification / multilabel_classification:
  num_classes

regression:
```

```
  output_dim     (default: 1)

ordinal_classification:
  num_classes
  ordinal        # optional; encoding / decoding, defaulted (see "Ordinal encoding and decoding")

distributional_regression:
  distribution   (e.g. gaussian, negative_binomial)

count_regression:
  output_dim     (default: 1)
```

Initial supported head network types:

Head type	Allowed `network.type`	Default `network.type`
`multiclass_classification`	`linear`, `mlp`	`linear`
`binary_classification`	`linear`, `mlp`	`linear`
`multilabel_classification` (deferred, P4.2)	`linear`, `mlp`	`linear`
`regression`	`linear`, `mlp`	`linear`
`ordinal_classification`	`linear`, `mlp`	`linear`
`distributional_regression` (deferred, P4.9)	`linear`, `mlp`	`linear`
`count_regression` (deferred, P4.9)	`linear`, `mlp`	`linear`

`network.type: linear` means there are no hidden layers between the head input embedding and the final head-specific projection. The head type still determines output shape and interpretation: classification heads emit logits, regression heads emit continuous values, ordinal heads emit ordinal logits / bins, distributional regression heads emit distribution parameters, and count regression heads emit count/rate parameters.

`network.type: mlp` adds "MultiLayer Perceptron" hidden layers before the same head-specific final projection. Initial MLP head-network config:

```yaml
network:
  type: mlp
  hidden_dims: [512]      # required; one or more hidden layer widths
  activation: gelu        # default: gelu
  dropout: 0.0            # default: 0.0
```

Head MLP networks do not set `output_dim`; the head type and its type-specific fields determine the final projection size.

Pydantic validation ensures the resolved `target` exists in `data.targets` and has a compatible dtype.

Ordinal naming and result columns

- Head type: `ordinal_classification` (the head predicts a discrete ordered bin, not a continuous value).
- Supported ordinal losses: `coral`, `corn`, `ordinal_cross_entropy`.
- Result `record_type` values: `ordinal_output` for native ordinal heads; `ordinal_probe_prediction` for ordinal probes.
- `ordinal_logits` (the `num_classes - 1` cumulative threshold logits) is populated only for cumulative encodings (`coral`, `corn`) and is null for `ordinal_cross_entropy`. `probabilities` (per-bin) is always populated: by differencing decoded cumulative probabilities for `coral` / `corn`, or directly by softmax over per-bin logits for `ordinal_cross_entropy`.

See `06-results-artifacts-and-metadata.md`.

Ordinal encoding and decoding

Order semantics are carried by the head (`type: ordinal_classification`, `num_classes`, and the ordered class labels). How those ordered classes are turned into output units, and how outputs map back to a class, is configured on the head under `ordinal`:

```yaml
model:
  heads:
    size_category:
      type: ordinal_classification
      target: size_category
      num_classes: 4
      ordinal:
        encoding: coral        # coral | corn | ordinal_cross_entropy
        decoding: threshold    # threshold | expected_rank | argmax
      network:
        type: linear
```

- `encoding` determines the output tensor. `coral` and `corn` emit `num_classes - 1` cumulative threshold logits ("is the class beyond bin k?"); `ordinal_cross_entropy` emits `num_classes` per-bin logits.
- `decoding` maps outputs to a predicted class: `threshold` counts how many cumulative thresholds clear 0.5 (CORAL / CORN); `expected_rank` takes the probability-weighted rank; `argmax` takes the most probable bin (per-bin form).

`ordinal` is optional in authored configs. Config compilation injects a default: `encoding` follows the configured ordinal loss when an objective is present (else `coral`), and `decoding` defaults per encoding (`coral` / `corn` → `threshold`, `ordinal_cross_entropy` → `argmax`). Resolved configs, saved config artifacts, and runtime objects always carry `ordinal` explicitly.

Encoding / decoding live on the head, not on the loss, because they define output structure and interpretation that must survive without an `objectives` block — exported portable models and cached-result ensembles still have to interpret `ordinal_logits` at inference time. They therefore contribute to `target_schema_hash` and are written into export metadata (`10-export.md`). When an objective is present, config validation checks that its ordinal loss is consistent with the head's `ordinal.encoding`; this is a cross-field check, not the head owning loss config.

Single- vs. multi-head

There are no separate single-head / multi-head model classes. A single-head model is a special case with exactly one head and one objective. Internally, configs normalize to the canonical multi-head / multi-objective shape.

Reference-chain example

```yaml
data:
  targets:
    species:
      column: species_idx
      type: multiclass_classification

model:
  heads:
    species:
      type: multiclass_classification
      target: species
      num_classes: 42
      network:
        type: linear

objectives:
  species:
```

```yaml
    head: species
    loss: cross_entropy
```

Multi-head example

```yaml
data:
  targets:
    species:
      column: species_idx
      type: multiclass_classification
      missing_policy: error
    life_stage:
      column: life_stage_idx
      type: ordinal_classification
      missing_policy: error
    biovolume:
      column: biovolume_um3
      type: regression
      transform: log1p_standardize
      missing_policy: drop_sample

model:
  tabular_input:
    enabled: true
    name: tabular
    columns: [depth_m, temperature_c, salinity_psu]
    encoder:
      type: mlp
      hidden_dims: [64, 64]
      output_dim: 64
  heads:
    species:
      type: multiclass_classification
      target: species
      num_classes: 42
      network:
        type: linear
    life_stage:
      type: ordinal_classification
      target: life_stage
      num_classes: 5
      ordinal:
        encoding: ordinal_cross_entropy
        decoding: argmax
      network:
        type: linear
    biovolume:
      type: regression
      target: biovolume
      output_dim: 1
      network:
        type: linear

objectives:
  species:
    head: species
```

```
    loss: cross_entropy
    metrics: [f1_macro, f1_per_class]
    weight: 1.0
  life_stage:
    head: life_stage
    loss: ordinal_cross_entropy
    metrics: [mae, quadratic_weighted_kappa]
    weight: 0.5
  biovolume:
    head: biovolume
    loss:
      type: huber
      delta: 1.0
    metrics: [mae, rmse, r2]
    weight: 0.25
```

Compatible losses

Classification: `cross_entropy`, `weighted_cross_entropy`, `class_balanced_effective_number`, `focal`, `label_smoothing_cros`

Regression: `mse`, `mae`, `huber`, `smooth_l1`, `quantile`. (`gaussian_nll`, `poisson_nll`, and `negative_binomial_nll` are deferred with the `distributional_regression` / `count_regression` heads — see `appendix-deferred-features.md` P4.9.)

Ordinal: `coral`, `corn`, `ordinal_cross_entropy`.

Count-dependent losses (`weighted_cross_entropy`, `class_balanced_effective_number`) derive their per-class weights from the frozen train-split class counts in the dataset stats cache (`dojo inspect dataset --class-counts`, `04-data-and-storage.md`). The resolved weights are frozen into config, not recomputed per run.

Metric registry

Metric names are canonical and aliases are not accepted. Prefer TorchMetrics semantics and TorchMetrics-style option names. Dojo owns the canonical defaults it materializes; those defaults do not have to rely on whatever defaults a TorchMetrics release happens to choose. Config validation checks metric compatibility through a small registry keyed by head type; the implementation may keep the registry central or merge per-head metric specs, but validation, logging, and `objective_summary` serialization must consume one canonical registry.

Each metric registry entry defines:

```
canonical name
compatible head types
TorchMetrics class / factory and Dojo default params
allowed authored params
prediction / target space used for computation
logging output name(s)
whether the metric emits one scalar or labeled per-class values
```

Initial metric names:

Head type	Metric	TorchMetrics semantics / default params	Output name
`multiclass_classificatio`	`accuracy`	multiclass top-1 accuracy, `top_k: 1`	`accuracy`
`multiclass_classificatio`	`f1_macro`	multiclass F1, `average: "macro"`	`f1_macro`
`multiclass_classificatio`	`f1_per_class`	multiclass F1, `average: null` / per-class output	`f1_per_class/{label}`

Head type	Metric	TorchMetrics semantics / default params	Output name
`regression`	`mae`	mean absolute error on external target units	`mae`
`regression`	`rmse`	square root of mean squared error on external target units	`rmse`
`regression`	`r2`	coefficient of determination on external target units	`r2`
`ordinal_classification`	`mae`	decoded ordinal class index	`mae`
`ordinal_classification`	`quadratic_weighted_kappa`	decoded ordinal class index	`quadratic_weighted_kappa`

Regression metrics use external target units after inverse target transforms. Ordinal metrics use the decoded ordinal class index unless a future registry entry explicitly says otherwise. Missing labels are handled before metric updates by the target's missing policy; metrics receive only valid examples for their objective.

Resolved configs and `objective_summary` materialize registry defaults into each metric spec. For example, authored `f1_macro` resolves to:

```yaml
name: f1_macro
params:
  average: "macro"
output: f1_macro
```

Target transforms

Common transforms (regression / ordinal heads): `identity`, `standardize`, `log1p`, `log1p_standardize`, `power` / Box-Cox / Yeo-Johnson. See `06-results-artifacts-and-metadata.md` for the external vs. internal column convention.

Target transforms are configured once, on the data target (`data.targets.<t>.transform`) — not on the objective. The functional form is authored; config compilation freezes any fitted statistics (standardize mean / std, Box-Cox lambda) into the resolved transform — the same authored-vs-resolved pattern used for normalization and imputation stats. The inverse is applied to predictions at inference to recover external units, so the resolved transform must survive without an `objectives` block: it is carried in export metadata and contributes to `target_schema_hash` by value.

Total loss

For each objective, the task module builds a valid-label mask from the referenced target's `missing_policy`:

- `error` means preflight should already have rejected missing labels.
- `drop_sample` means missing rows for that target are removed before batching when that target is required by an enabled objective / scorer.
- `mask_objective` keeps the sample in the batch but excludes rows with missing labels for that target from that objective's loss and metrics.

Objective losses are reduced over valid rows only. If an objective has zero valid rows in a batch, that objective contributes no loss for that batch. The total loss is the weighted sum of available objective losses, with no automatic renormalization of weights:

```
total_loss = sum( objective_i.weight * objective_i.loss for objectives with valid rows )
```

If an enabled objective has zero valid labels across an active training or eval split, preflight fails before training / scoring starts.

Objective shorthand

For simple configs, an objective name may imply the head name:

```yaml
objectives:
  species:
    loss:
      type: focal
      gamma: 2.0
    weight: 1.0
```

resolves to:

```yaml
objectives:
  species:
    head: species
    loss:
      type: focal
      gamma: 2.0
    weight: 1.0
```

Pydantic validation

Validation enforces:

- every objective references an existing head;
- loss is compatible with the referenced head type;
- metrics are compatible with the referenced head type;
- target transforms are compatible with the head type;
- objective weights are non-negative;
- at least one objective is enabled for supervised training;
- for classification / ordinal heads, the head's `num_classes` matches the resolved class map for the referenced `data.targets.<target>` (the ordered labels from `class_names`, or the `--class-counts` class count); a mismatch is a config error.

Supervised training

LightningModule responsibilities

Owns:

- forward-pass orchestration;
- `training_step`, `validation_step`, `test_step`;
- optimizer / scheduler creation;
- per-objective loss computation;
- weighted total-loss sum;
- metric updates derived from objective definitions.

Does **not** own dataset path logic, experiment-tracker specifics, artifact layout, snapshot bundling, export, or result-file serialization.

```python
class SupervisedTaskModule(L.LightningModule):
    def __init__(
        self,
        model_config: SupervisedModelConfig,
        training_config: TrainingConfig,
        objectives_config: ObjectiveCollectionConfig,
        optimizer_config: OptimizerConfig,
        scheduler_config: SchedulerConfig | None = None,
    ):
        super().__init__()
```

```
        self.save_hyperparameters()
        self.model = build_supervised_model(
            model_config,
            freeze_config=training_config.freeze,
        )
        self.objectives = build_objectives(objectives_config)
        self.metrics = build_metrics(objectives_config)
```

Constructor arguments are serializable Pydantic configs so Lightning hyperparameter checkpoints stay clean.

Model composition

```
backbone = build_backbone(cfg.model.image_input.backbone.architecture)
initialize_backbone_weights(backbone, cfg.model.image_input.backbone.weights)
apply_freeze_policy(backbone, cfg.training.freeze.backbone)

tabular_encoder = (
    build_tabular_encoder(cfg.model.tabular_input)
    if cfg.model.tabular_input.enabled
    else None
)

model_input_order = [cfg.model.image_input.name]
model_embedding_dim = backbone.output_dim
if tabular_encoder:
    model_input_order.append(cfg.model.tabular_input.name)
    model_embedding_dim += tabular_encoder.output_dim

# SupervisedModel concatenates enabled input embeddings in model_input_order
# before applying the optional embedding adapter.
embedding_adapter = (
    build_embedding_adapter(
        cfg.model.embedding_adapter,
        input_dim=model_embedding_dim,
    )
    if cfg.model.embedding_adapter.enabled
    else None
)

heads = build_heads(
    cfg.model.heads,
    input_dim=embedding_adapter.output_dim if embedding_adapter else model_embedding_dim,
)

model = SupervisedModel(
    backbone=backbone,
    tabular_encoder=tabular_encoder,
    embedding_adapter=embedding_adapter,
    heads=heads,
)
```

Optimizer, scheduler, checkpointing

`optimizer`, `scheduler`, and `checkpointing` are top-level config groups peer to `training`. Checkpointing supports best-k, last, epoch, step, and snapshot checkpoints.

P1/P2 executable baseline:

```yaml
optimizer:
  name: adamw
  lr: 0.0003
  weight_decay: 0.01

scheduler:
  name: cosine              # none | cosine | cosine_warm_restarts | step
  warmup_epochs: 3

checkpointing:
  monitor: val/species/f1_macro
  mode: max                 # min | max
  save_top_k: 3
  save_last: true
```

Initial optimizer support is `adamw`. Initial scheduler support is `none`, epoch-based `cosine`, `cosine_warm_restarts` for snapshot ensembles, and `step`; warmup values are epoch counts unless a scheduler explicitly states otherwise. `checkpointing.monitor` must match a logged metric name from the objective / metric registry. If the monitored metric is never logged, checkpoint setup fails with a config/runtime validation error rather than silently saving no best checkpoint.

Snapshot-cycle checkpointing for `task.type: snapshot_ensemble`:

```yaml
checkpointing:
  monitor: val/species/f1_macro
  mode: max
  save_top_k: 3
  save_last: true
  save_cycle_snapshots:
    enabled: true
    at_cycle_end: true
```

The cosine-warm-restarts scheduler is the recommended snapshot-cycle scheduler. See `02-cli-and-task-types.md` for the `task.type: snapshot_ensemble` flow and `08-ensembles.md` for the candidate-selection step.

Portable `.pt` / `.onnx` files are **not** automatic training artifacts — they live under `exports/` and are produced by explicit export configuration (`training_outputs.export`) or `dojo export`. See `10-export.md`.

Inference-contract embedding

The task module's `on_save_checkpoint` hook writes a portable **inference contract** into `checkpoint["dojo_inference_contrac` the buildable `model_config`, the resolved `inference_pipeline` with frozen preprocessing stats, per-head class maps, the target schema with frozen target transforms, the resolved `objective_summary` used for holdout scoring, and the four compatibility hashes. It is kept **out** of Lightning `hyper_parameters` (so the constructor configs stay clean) and makes a `.ckpt` as self-describing as an export: `dojo infer` / `dojo eval` rebuild the model and its input pipeline from the artifact alone, with no dependency on the producing run's `resolved.yaml`. See `06-results-artifacts-and-metadata.md`.

Early stopping

Early stopping is a training-loop behavior under `training:`, not `runtime:`:

```yaml
training:
  early_stopping:
    enabled: true
    monitor: val/loss
    mode: min
    patience: 10
```

Representation evaluation scheduling

A supervised run may schedule representation evaluation against its own encoder during training. The same `representation_eval` config is used standalone via `dojo eval representation`. See `07-ssl-and-representation-eval.md`.

Cross-References

- `02-cli-and-task-types.md` — `task.type: supervised`, `task.type: snapshot_ensemble`, `dojo inspect backbone`.
- `03-configuration.md` — placement of `model:`, `transforms:`, `training:`, `optimizer:`, `scheduler:`, `checkpointing:`, `objectives:`.
- `04-data-and-storage.md` — `data.targets` referenced by heads.
- `06-results-artifacts-and-metadata.md` — record types (`classification_output`, `regression_output`, `ordinal_output`, `sample_metadata`), external vs. internal target/prediction columns, embedding kinds.
- `07-ssl-and-representation-eval.md` — supervised encoders fed into representation eval; SSL transfer-learning recipe.
- `08-ensembles.md` — snapshot-cycle scheduler feeds snapshot ensembles.
- `10-export.md` — exports/ artifacts and their metadata.
- `11-dependencies.md` — `train` and `timm` optional extras.
- `glossary.md` — head-type and backbone-source vocabulary.

06. Results, Artifacts, and Metadata

Purpose

Defines the canonical result schemas, identifiers and hashes, sidecar `_metadata.json` shape, on-disk artifact layout under each `*_outputs.dir`, result partitioning, and logging/diagnostics behavior. This file is the authoritative reference for everything that gets written to disk other than checkpoints.

Results backend

- Dojo owns canonical result schemas and `_metadata.json`.
- Result writing uses `amplify-db-utils` for partitioned DuckDB / Parquet output, schema registration, filtered reads, bulk reads, and object-store-compatible paths.
- Explicit `pyarrow.Schema` definitions back Dojo result tables, especially for Arrow list / vector columns.
- Only per-sample records belong in result Parquet. Per-class metrics, run-level metrics, and plots belong in `metrics/` and `figures/`.

Result writer config

```yaml
training_outputs:
  results:
    enabled: true
    backend: amplify_db_utils
    dir: results            # relative to training_outputs.dir
    format: parquet
    partition_by: [stage, record_type, epoch]
    dictionary_encode:
      enabled: true
      columns:
        - split
        - stage
        - record_type
        - ensemble_result_scope
        - head_name
        - embedding_kind
        - prediction_label
```

```
      - resize_width_px
      - resize_height_px
  write_metadata_json: true
```

Defaults: `training_outputs.results.dir` → `<training_outputs.dir>/results/`. `ensemble_outputs.results.dir` → `<ensemble_outputs.dir>/ensemble_results/`.

Arrow list columns

Vector columns:

```
embedding
logits
probabilities
ordinal_logits
```

Dictionary-encoded columns

```
split
stage
record_type
ensemble_result_scope
head_name
embedding_kind
prediction_label
resize_width_px
resize_height_px
```

IDs and hashes

Identity and content-equality are tracked separately:

- A **`*_hash`** is derived deterministically from object content.
- A **`*_id`** is either manually set or generated from an explicit coolname template. Hash-paired IDs use hash-seeded coolnames by default.

Coolname template forms:

Form	Seed behavior	Deterministic?	Intended use
`{coolname:<source>}`	Seed from the named resolved value, usually a hash such as `config_hash`, `model_hash`, `ensemble_hash`, or `sweep_hash`	yes	hash-derived IDs
`{coolname}`	Seed from resolved `runtime.seed`	yes	rare/debug; collision-prone for run IDs
`{coolname:noseed}`	Fresh unseeded coolname	no	invocation IDs such as `run_id`

The seed material includes the source label as well as the source value (for example `model_hash:<hash>`), so identical raw hash strings in different domains do not imply identical names. A hash-seeded coolname can only be rendered after that hash has been computed; generated IDs are excluded from the source hash and do not feed back into it.

Explicit exceptions:

- `run_id` may be generated from a fresh `{coolname:noseed}` template token. There is no `run_hash`.
- `dataset_id` is only set when the dataset self-names; no generated fallback.
- `sweep_id` may be manually set, rendered from a coolname template, or fall back to `{coolname:sweep_hash}` when unset.

Full hashes are recorded by default. The only standard truncation is the first 6 hex characters embedded in checkpoint filenames.

Canonicalizer

Lives in `dojo.utils.artifact_hashing`. One JSON normalization recipe used by every hash function:

- canonical UTF-8 JSON;
- sorted object keys;
- no insignificant whitespace;
- lists preserved in source order;
- floats rounded to **12 significant decimal digits** before serialization (well within float64 precision of ~15.95, absorbs roundtrip noise);
- `NaN`, `+Infinity`, `-Infinity` serialize as the literal strings `"NaN"`, `"Infinity"`, `"-Infinity"`;
- bytes/binary inputs hashed directly without JSON wrapping.

Each `*_hash` function selects a specific subset of fields from the source object before canonicalization.

Per-field table

Field	Kind	Derivation	Default when not set
`run_id`	id only	manual, else template render — e.g. `{coolname:noseed}` (fresh unseeded) or a composed pattern like `{experiment.name}-{timestamp}-{job_num}`	`{coolname:noseed}`
`config_id`	id (paired with `config_hash`)	manual, else `{coolname:config_hash}`	`{coolname:config_hash}`
`config_hash`	hash	canonical hash of resolved config, **excluding** runtime-resolved values, output paths, `output_root`, and all `*_outputs` blocks. When a loaded config carries blocks the active command does not use, the hash covers only that command's active block-set (see `02-cli-and-task-types.md`).	always derived
`dataset_id`	id only	the dataset's self-name when the manifest provides one	null (no generated fallback)

Field	Kind	Derivation	Default when not set
`dataset_hash`	hash	cheap identity (never reads image pixels or tabular cell payloads): manifest identity / canonical manifest projection or URI + size + etag/last-modified, plus backend type and declared logical tabular column names / bindings. Basis recorded in `dataset_hash_provenance` (`manifest_content` / `uri_etag` / `uri_only` fallback). `uri_only` is weak identity and does not reliably auto-invalidate stale stats caches.	always derived
`dataset_content_hash`	hash	true hash over sample content: all image bytes plus tabular feature values for declared tabular columns when present; recorded separately when a content pass runs (`dojo inspect dataset --content-hash` / `--normalization`). Integrity / drift verification only — **not** the cache key, identity, or a compatibility hash	derived when a content pass runs, else null
`checkpoint_hash`	hash	SHA-256 of the `.ckpt` file bytes	always derived
`model_id`	id (paired with `model_hash`)	manual on export, else `{coolname:model_hash}` after export bytes are hashed	`{coolname:model_hash}`
`model_hash`	hash	SHA-256 of the exported `.pt` / `.onnx` file bytes	always derived
`ensemble_id`	id (paired with `ensemble_hash`)	manual on ensemble, else `{coolname:ensemble_hash}` after selected-ensemble identity is hashed	`{coolname:ensemble_hash}`
`ensemble_hash`	hash	canonical hash of the selected-ensemble identity block (selected members + combine + selection). Candidate-audit metadata in the manifest is excluded.	always derived

Field	Kind	Derivation	Default when not set
`sweep_id`	id (paired with `sweep_hash`)	manual, else template render; falls back to `{coolname:sweep_hash}` when unset	`{coolname:sweep_hash}`
`sweep_hash`	hash	canonical hash of sweep definition (base config + sweep axes + value lists), excluding runtime-resolved values and output paths	always derived
`ensemble_member_id`	union column	for member-level rows / partitioning; member's `checkpoint_hash` (checkpoint member) or `model_id` (exported model member)	derived per-row

There is no `run_hash`. There is no `checkpoint_id`; checkpoints are identified by `checkpoint_hash` plus their filename.

Checkpoint filename convention

`{stem}.{first6_hex}.{ext}`

Examples:

```
loss-1.23_epoch-003_f1score-88.7ff91a.ckpt
last.a1b2c3.ckpt
snapshot_cycle-02_epoch-100.0f0f0f.ckpt
```

Compatibility hashes

For `target_schema_hash`, `class_mapping_hash`, `model_config_hash`, and `preprocessing_hash`, both the hash and the exact canonical source sub-block are stored in `_metadata.json`. Fast path: compare hashes. Slow path: when hashes differ, diff source sub-blocks and present a human-readable mismatch report.

Compatibility hashes are computed from the **resolved config** after defaults and generated schema values are injected, but before run-local paths and runtime identifiers matter. The extractor must build a minimal canonical JSON object containing only the fields listed below. Do not hash whole config branches by reference.

For prediction-space ensembles, not every compatibility hash is a cross-member equality requirement. `target_schema_hash` and `class_mapping_hash` usually must agree for a shared head, while `model_config_hash` and `preprocessing_hash` primarily validate a member's own checkpoints, exports, cached rows, and metadata drift. See `08-ensembles.md`.

What each hash validates:

- `target_schema_hash` validates output-task structure: head names, head types, target types, output dimensions, `num_classes`, ordinal encoding / decoding, distributional head shape, and target transforms. It answers whether result rows, checkpoints, or models are comparable for the same prediction-task shape.
- `class_mapping_hash` validates label-index semantics for discrete heads. Two models may both output 42 logits, but they are incompatible if index 7 names different classes. It answers whether each logit / probability position means the same label across artifacts.
- `model_config_hash` validates the architecture contract governing checkpoint loadability and the inference-time forward function: image-input backbone architecture, tabular-input encoder shape, embedding adapter shape, and head network shapes and activations. It answers whether checkpoints and exports load under the same model definition and compute the same function. It excludes initialization (library or checkpoint weights), trainability (freeze policy), and training-only regularization (dropout) — none change tensor shapes or inference outputs.

- `preprocessing_hash` validates the input contract: image mode, bit-depth scaling, resize / bucketing, normalization, foreground crop, grayscale handling, inference pipeline, tabular feature ordering, encodings, and normalization stats. It answers whether the same raw sample would be transformed into the same model input tensor.

Common exclusions for all compatibility hashes:

```
experiment
task
runtime
storage
training
optimizer
scheduler
checkpointing
objectives
ensemble
sweep
ssl
representation_eval
output_root
training_outputs
ensemble_outputs
sweep_outputs
eval_outputs
logging
metrics
figures
export destinations
run_id / sweep_id / config_id
local cache paths
```

`target_schema_hash` source fields:

```
version: "1"
heads:
  <head_name>:
    type
    target
    target_type                       # from data.targets[<target>].type
    target_transform                  # resolved data.targets[<target>].transform, by value (type + frozen s
    output_dim                        # regression/count/distributional heads
    num_classes                       # classification/ordinal heads
    distribution                      # distributional_regression only (deferred head; see appendix P4.9)
    ordinal:                          # ordinal_classification heads only; from model.heads[<head>].ordinal
      encoding                        # coral | corn | ordinal_cross_entropy
      decoding                        # threshold | expected_rank | argmax
```

The `heads` object is keyed by resolved head name and sorted by key during canonicalization. `target_schema_hash` excludes loss type, loss params, objective weights, metric selections, optimizer settings, and checkpoint monitor fields. Ordinal `encoding` / `decoding` are read from the resolved head (`model.heads.<head>.ordinal`), not from the objective loss, so the loss exclusion holds even though the encoding determines output structure. The target transform is read only from `data.targets.<target>.transform` (its single home; objectives carry no target transform) and is hashed by value, including any resolved fit statistics.

`class_mapping_hash` source fields:

```
version: "1"
heads:
```

```
  <head_name>:
    target
    ordered_labels:
      - index: 0
        label: <string>
      - index: 1
        label: <string>
```

Include only heads with discrete classes: `multiclass_classification`, `binary_classification`, `multilabel_classification`, and `ordinal_classification` when ordinal bins have configured class names. `classes` is the resolved ordered class names for the target — from its `label_name_column`, or upstream dataset class metadata — recorded in `_metadata.json` alongside `class_mapping`. The resolved ordered class content is the hash input, not any source URI.

`model_config_hash` source fields:

```
version: "1"
model:
  image_input:
    backbone:
      architecture:
        source                        # torchvision / timm
        name
        output_dim
        params                        # architecture params that change module shape / forward behavior
  tabular_input:
    enabled
    columns                          # selected logical features in tensor order
    encoder                          # type, input_dim, output_dim, hidden_dims, activation
  embedding_adapter:
    enabled
    type
    hidden_dims
    output_dim
    activation
  heads:
    <head_name>:
      type
      target
      network                        # type, hidden_dims, activation (excludes dropout)
      num_classes
      output_dim
      distribution
```

For checkpoint-initialized models, include only the effective backbone architecture (the module the checkpoint instantiates), not the checkpoint-loading plumbing. `weights.source`, `weights.uri`, `weights.key`, and `weights.strict` govern which upstream weights initialize the backbone at build time, not the resulting architecture or inference function; that initialization provenance lives in `config_hash`, the trained bytes in `checkpoint_hash`, and export artifacts in `model_hash`. `model_config_hash` excludes optimizer, scheduler, training loop settings, objective loss/metric choices, checkpoint save policy, output paths, and runtime identifiers. It also excludes fields that change neither tensor shapes nor the inference forward function: `model.image_input.backbone.weights` (initialization), `training.freeze` (trainability), and `dropout` (training-only regularization). `activation` is retained because it changes inference outputs even though it is stateless.

`model.tabular_input.columns` is retained because it maps tabular encoder input positions to feature semantics; changing the selected features or their order changes the inference function even when the tensor width is unchanged. Available-but-unused tabular features from `data.tabular_feature_columns` are not part of `model_config_hash`.

`preprocessing_hash` source fields:

```
version: "1"
transforms:
  image_mode
  input_bit_depth                  # resolved integer (auto resolves to 8/12/16); [0,1] scaling divisor
  inference_pipeline               # resolved inference steps, ordered, with parameters
tabular_preprocessing:
  selected_columns                 # resolved model.tabular_input.columns after default expansion
  encodings                        # per selected categorical feature: one_hot policy, frozen vocabulary, t
  imputation                       # per-column strategy, frozen fill values, missing-indicator set
  normalization                    # per-column type plus resolved mean/std for mean_std
```

Only inference-time preprocessing belongs in `preprocessing_hash`. Manifest column-name bindings (`image_uri_column`, `sample_id_column`, `split_column`) and the storage `backend` are excluded: they locate a sample, they do not transform it, so they must not make otherwise-identical input contracts hash differently. Image transform steps come solely from the resolved `inference_pipeline` (`05-models-training-and-heads.md`), which excludes `train_only` augmentation by construction; the full training `pipeline` is not hashed, and normalization, resize / bucket, and foreground-crop parameters are the parameters of their steps inside `inference_pipeline`, not separate fields. The `inference_pipeline` uses the same canonical transform step names accepted in authored configs.

Tabular preprocessing comes from resolved `transforms.tabular` for the selected model-input features only, excluding train-only `transforms.tabular.augmentations`. `data.tabular_feature_columns` is dataset schema: adding an unused available feature does not change the `preprocessing_hash`. Tabular augmentations affect training behavior and therefore `config_hash`, but they are not part of the export inference contract or `preprocessing_hash`. Resolved dataset statistics such as normalization mean/std, numeric tabular normalization stats, frozen tabular imputation fill values, and categorical vocabularies are included by value, not by the URI from which they were loaded. These resolved statistics are produced by `dojo inspect dataset` and cached (`04-data-and-storage.md`) — `--stats` for the manifest/header-level stats, `--stats --normalization` for dataset normalization mean/std — but the hash always uses their resolved content, never the cache location. The `imputation` entry captures the per-column fill strategy, the frozen fill values, and the missing-indicator set. The `normalization` entry captures the per-column normalization type (`identity` or `mean_std`) and the frozen train-split `mean` / `std` values for `mean_std`; `identity` carries no fitted statistics. When `add_missing_indicator` is enabled the resulting encoder input width is additionally reflected in `model_config_hash` via the resolved `model.tabular_input.encoder.input_dim` and any downstream resolved concatenated / adapter dimensions (see `05-models-training-and-heads.md`). For categorical features, the `encodings` entry captures the resolved `one_hot` encoding contract by value: canonical feature name, source type, resolved vocabulary in tensor order, `missing_policy`, `missing_token`, `unknown_policy`, `unknown_token`, and the expanded tabular input positions. This ensures exported models and cached-result consumers agree on how raw category values become numeric model inputs. Adding, removing, or reordering a category changes `preprocessing_hash`; the resulting input width also changes `model_config_hash` through the resolved tabular encoder input dimension.

The Pydantic schema should keep these field lists close to the relevant models, for example with compatibility-hash extractor methods or field metadata. The documentation above is the execution contract those extractors must satisfy.

Portable inference contract

`dojo infer` and `dojo eval` rebuild a model and its exact input pipeline from the model artifact alone — they do **not** require the producing run's `resolved.yaml`. `dojo inspect checkpoint` also reads this contract from `.ckpt` files, but `.ckpt` deserialization requires the Torch stack (`11-dependencies.md`). Everything these consumers need is bundled as a single **inference contract** object, defined once and serialized identically by both artifact kinds:

- **Checkpoints** embed it under a dedicated `checkpoint["dojo_inference_contract"]` key, written by the task module's `on_save_checkpoint` hook. It is **not** folded into Lightning `hyper_parameters`, which stay limited to the constructor configs (see `05-models-training-and-heads.md`).
- **Exports** write it as their export metadata (`10-export.md`); the export metadata *is* the inference contract.

Contract fields:

```
schema_version
model_config            # buildable resolved model: backbone architecture, tabular encoder, embedding adapte
inference_pipeline      # resolved, ordered inference transforms with frozen parameters
```

```
preprocessing_stats     # frozen normalization mean/std, resolved input_bit_depth, bucket scheme, tabular no
class_maps              # ordered index -> label per discrete head (resolved label content, not a URI)
target_schema           # head types, targets, num_classes/output_dim, ordinal encoding/decoding, target tra
objective_summary       # objective/head loss and metric metadata needed for holdout eval scoring
compatibility           # target_schema_hash / class_mapping_hash / model_config_hash / preprocessing_hash p
provenance              # config_hash, source run_id, dojo_version
```

It deliberately **excludes** training-dataset identity (`manifest_uri`, `dataset_hash`, per-class counts): those describe the data a model was trained on, not what is needed to run it on new data. `dojo inspect checkpoint` reads this contract to report embedded hashes and head / preprocessing configuration. `objective_summary` is descriptive and metric-driving metadata; it is not a compatibility-hash input.

`objective_summary`

`objective_summary` is the resolved scoring contract for supervised artifacts. It lets `dojo eval holdout` compute the same loss / metric family the model was trained and validated with, without requiring an `objectives:` block in the eval config. It is not used to rebuild the model.

Shape:

```yaml
objective_summary:
  schema_version: 1
  total_loss:
    reduction: weighted_sum
  objectives:
    species:
      head: species
      target: species
      weight: 1.0
      loss:
        type: cross_entropy
        params: {}
      metrics:
        - name: accuracy
          params:
            top_k: 1
          output: accuracy
        - name: f1_macro
          params:
            average: "macro"
          output: f1_macro
        - name: f1_per_class
          params:
            average: null
          output: "f1_per_class/{label}"
```

Rules:

- `objectives` contains the enabled resolved objectives, keyed by objective name. Disabled authored objectives are omitted from the portable contract.
- `head` must name a head in `target_schema`; `target` must match that head's target. This duplication is a validation guard and makes scorer setup straightforward, but `target_schema` remains authoritative for head type, output dimensions, ordinal encoding / decoding, and target transforms.
- `loss` is the canonical resolved loss spec: string shorthand is expanded to `{type, params}` and any resolved class / sample weighting values needed to reproduce evaluation loss are stored by value, not by dataset-stat URI.
- `metrics` is the ordered list of canonical resolved metric specs from the metric registry (`05-models-training-and-heads.md`). Aliases are not accepted. Registry defaults such as averaging mode, top-k values, thresholds, and per-class behavior are materialized under `params`; output / logging names are materialized under `output`.

- `weight` is included so holdout eval can report both per-objective losses and the weighted total loss using the same weighted-sum rule as training.
- Classification and ordinal label names are read from `class_maps`; target inverse transforms and internal-vs-external unit conventions are read from `target_schema`, not duplicated here.
- Pure embedding artifacts with no supervised objectives serialize `objectives: {}` and `total_loss: null`; `dojo eval holdout` raises a validation error if no objective covers the requested labeled target.

Result rows

Common provenance columns

All supervised / representation-evaluation result records share:

```
sample_id
uri
split
stage
record_type
run_id
config_id
config_hash
dataset_id
dataset_hash
epoch
global_step
checkpoint_hash
model_id
model_hash
sweep_id            # when produced as part of a sweep
sweep_hash
schema_version
```

`epoch` and `global_step` may be null for static sample metadata. `checkpoint_hash` identifies the producing checkpoint. `model_id` / `model_hash` are populated when the row was produced by an exported model artifact.

Ensemble result records additionally carry:

```
ensemble_id
ensemble_hash
ensemble_result_scope    # member | ensemble
ensemble_member_id       # populated only for member rows
```

Rows written by the ensemble pipeline use `stage=ensemble_eval`. Cached source rows keep their original stage in their source result dataset; if they are transcribed into `ensemble_outputs.results.dir`, they are rewritten as ensemble-produced rows with `stage=ensemble_eval`.

For representation-evaluation rows, add `evaluation_name` to identify the specific probe / retrieval / clustering / projection / diagnostic pass.

`split` vs. `stage`

- `split` — source dataset split (`train`, `val`, `test`, `unlabeled`, `holdout`).
- `stage` — process that produced the row (`train_validation`, `holdout_eval`, `infer`, `representation_eval`, `ensemble_eval`).

`record_type` values

Supervised:

`sample_metadata`

```
embedding
classification_output
regression_output
ordinal_output
```

Representation-evaluation:

```
embedding
nearest_neighbor
knn_prediction
classification_probe_prediction
regression_probe_prediction
ordinal_probe_prediction
cluster_assignment
projection
outlier_score
diagnostic
```

`sample_metadata` columns

First-class sample metadata columns:

```
native_width_px
native_height_px
resize_width_px
resize_height_px
microns_per_pixel
aspect_bucket
bin_id
bin_uri
roi_number
```

Plus optional single-column JSON blobs:

- `source_extra_json` — present only when `data.source_extra_columns` is configured; only appears on `record_type=sample_metadata`.
- `tabular_features_json` — single JSON-string column for configured tabular features.

`embedding` columns

```
embedding_kind = image_embedding | tabular_embedding | fused_input_embedding | head_input_embedding
embedding
embedding_dim
embedding_model_name        # representation-eval only
```

Default `embedding_kind`:

- supervised → `head_input_embedding`
- SSL → `image_embedding`

Classification output

```
head_name
prediction_index
prediction_label
prediction_confidence
logits
probabilities
```

The row is scoped by `head_name`; the head → target link lives in `_metadata.json` (each head's `target`). Ground-truth columns (`target_index` / `target_name`) are reserved for P2 (see workplan P2.5); P1 writes predictions only.

Regression output

External vs. internal columns:

- `target` and `prediction_value` are external (original units). `prediction_value` is the post-inverse-transform model output.
- `target_internal` and `prediction_value_internal` are model-space values (post `target_transform`).
- When no `target_transform` is configured, writers may either populate both columns identically or leave the `_internal` columns null; the behavior is recorded in `_metadata.json`.
- `prediction_uncertainty` is in external units by default; if the head defines uncertainty in transformed space, a parallel `prediction_uncertainty_internal` column may be added.

Columns:

```
head_name
target
prediction_value
target_internal
prediction_value_internal
prediction_uncertainty
```

Ordinal output

```
head_name
target
prediction_index
prediction_label
prediction_confidence
ordinal_logits         # cumulative threshold logits; null for non-cumulative encodings
probabilities          # per-bin probabilities, always populated
```

`ordinal_logits` holds the `num_classes - 1` cumulative threshold logits and is populated only for cumulative encodings (`coral`, `corn`); it is null for `ordinal_cross_entropy`, which has no cumulative parameterization. `probabilities` always holds the `num_classes` per-bin probabilities: for `coral` / `corn` the writer derives them by differencing cumulative probabilities decoded from `ordinal_logits`; for `ordinal_cross_entropy` they are the softmax over the per-bin logits directly. Consumers therefore always see per-bin probabilities regardless of `ordinal.encoding`. Because `ordinal_logits` is encoding-specific, the `ordinal_logits_mean` ensemble combine mode applies only to cumulative-encoding members; per-bin members combine via `ordinal_probabilities_mean` (see `08-ensembles.md`).

Native ordinal heads write `record_type=ordinal_output`. Ordinal probes write `record_type=ordinal_probe_prediction`.

Representation-evaluation record specifics

- `nearest_neighbor`: `query_sample_id`, `neighbor_sample_id`, `neighbor_rank`, `distance`, `distance_metric`, `neighbor_label` (nullable when unlabeled).
- `knn_prediction`: classification-style head columns plus `k` and `distance_metric`.
- Probe predictions: same column shape as the matching native output record. `head_name` should identify the probe (e.g. `linear_probe_species`). `probe_model_type` documents the probe class (e.g. `ridge`, `linear_classifier`, `ordinal_logistic_regression`).
- `cluster_assignment`: `cluster_method`, `cluster_id`, `cluster_distance`, `cluster_probability` (optional, soft assignments only).
- `projection`: `projection_method`, `projection_x`, `projection_y`, `projection_z` (null for 2D).
- `outlier_score`: `outlier_method`, `outlier_score`, `outlier_rank`, `is_outlier` (optional).
- `diagnostic`: `diagnostic_name`, `diagnostic_value`, `diagnostic_scope` (`sample`, `batch`, `epoch`, `dataset`). Aggregate diagnostics may leave `sample_id` null.

`_metadata.json` sidecar

Structure: top-level `record_types` keys (not a top-level `heads` key). Stores semantic / provenance metadata; **not** low-level Parquet encoding details. Per-record-type sections name `target_transform`, class mappings, and head

mappings.


```
{
  "schema_name": "dojo.supervised_results",
  "schema_version": "1.0.0",
  "created_by": "dojo",
  "run_id": "ifcb-green-river",
  "compatibility": {
    "target_schema_hash": "sha256:...",
    "target_schema_source": {
      "version": "1",
      "heads": {}
    },
    "class_mapping_hash": "sha256:...",
    "class_mapping_source": {
      "version": "1",
      "heads": {}
    },
    "model_config_hash": "sha256:...",
    "model_config_source": {
      "version": "1",
      "model": {}
    },
    "preprocessing_hash": "sha256:...",
    "preprocessing_source": {
      "version": "1",
      "transforms": {},
      "tabular_preprocessing": {}
    }
  },
  "record_types": {
    "sample_metadata": {
      "description": "One row per evaluated sample.",
      "source_extra_json": {
        "columns": ["cruise_id", "cast_id", "instrument_id"]
      },
      "tabular_features_json": {
        "columns": ["depth_m", "temperature_c", "salinity_psu"]
      }
    },
    "classification_output": {
      "heads": {
        "species": {
          "target": "species",
          "classes": ["A", "B", "C"],
          "class_mapping": {"0": "A", "1": "B", "2": "C"}
        }
      }
    },
    "regression_output": {
      "heads": {
        "biovolume": {
          "target": "biovolume",
          "target_transform": "log1p"
        }
      }
```

```
    }
  }
}
```

The `compatibility` block is required for model-produced result sidecars, checkpoint-adjacent metadata, and exported-model metadata. A value may be `null` only when the artifact genuinely cannot supply that compatibility dimension, such as cached sample metadata without a producing model. Consumers compare `*_hash` values first and diff the paired `*_source` objects when hashes differ.

Result partitioning

`training_outputs.results.partition_by`, `ensemble_outputs.results.partition_by`, and `eval_outputs.results.partiti` are configurable lists. Partition field options include:

```
stage
epoch
ensemble_result_scope
ensemble_member_id
record_type
sweep_id
```

Include `sweep_id` automatically when a row is produced as part of a sweep. `ensemble_result_scope` distinguishes combined ensemble rows (`ensemble`) from retained member-level rows (`member`). `ensemble_member_id` is a union column whose value is the member's `checkpoint_hash` (checkpoint members) or `model_id` (exported model members). It is populated only for `ensemble_result_scope=member`, so it is not a default partition key for mixed member / ensemble result datasets.

Examples:

```yaml
training_outputs:
  results:
    partition_by: [stage, record_type]
```

```yaml
training_outputs:
  results:
    partition_by: [stage, epoch, record_type]
```

```yaml
ensemble_outputs:
  results:
    partition_by: [stage, record_type, ensemble_result_scope]
```

`ensemble_member_id` may be added as an opt-in partition key for member-only analysis outputs, but mixed ensemble/member outputs should partition by `ensemble_result_scope` first or leave `ensemble_member_id` as a filter column.

Artifact layout

Each `*_outputs` block owns a stable set of sub-directories under its resolved `dir`. Whether each is written depends on the corresponding sub-block being enabled in config.

- `metrics/` — numeric aggregates only: per-split metric JSON and confusion-matrix data (JSON or CSV). No rendered images.
- `figures/` — **configurable output block**, not just a directory. Config specifies which kinds of plots to render (training curves, confusion-matrix heatmaps, UMAP/t-SNE/PCA scatter, retrieval panels, calibration, ensemble comparison) and plot styling. Output files are PNG / SVG / HTML.
- There is no generic training-run `data/` folder; input-dataset information lives in configs and resolved-config artifacts.
- Ensemble candidate manifests live under `ensemble_outputs.manifests.dir`, default `{ensemble_outputs.dir}/ensemble_` Manifests are JSON, not Parquet.

Per-block layouts:

```
training_outputs.dir/
  config/
  checkpoints/
  exports/
  metrics/
  figures/
  results/

ensemble_outputs.dir/
  config/
  exports/
  metrics/
  ensemble_figures/
  ensemble_results/
  ensemble_manifests/
  ensemble_members/         # optional; only when ensemble materializes members locally

sweep_outputs.dir/
  config/
    composed.yaml
    resolved.yaml
    resolved.json
    sweep_manifest.json
    cli.txt
    overrides.txt
  exports/
  metrics/
  figures/

eval_outputs.dir/
  config/
  results/                  # stage=infer | holdout_eval | representation_eval rows
  metrics/
  figures/
  eval_manifest.json        # always written: model source + dataset identity + metric summary
```

`eval_outputs/` is written by `dojo infer`, `dojo eval`, and standalone `dojo eval representation`. `eval_manifest.json` is always written and records the model source (artifact URI + `checkpoint_hash` / `model_hash` and the inference-contract compatibility hashes), the evaluation dataset (`dataset_id` / `dataset_hash` + split), a metric summary, and the result-rows location — mirroring the ensemble manifest so a later `dojo ensemble` or report step can consume eval runs. Like `dojo ensemble`, an `infer` / `eval` run does **not** initialize experiment logging.

For `task.type: snapshot_ensemble` with `ensemble_outputs.dir_template` defaulted to the training value, both blocks resolve to the same path and the directory holds the union of both layouts. Training rows are written under `results/`; ensemble rows are written under `ensemble_results/`. Row `stage` still records provenance (`train_validation` vs. `ensemble_eval`), but separate result directories are the primary collision-avoidance boundary. Metrics files include the producing block in their filenames when namespacing is needed.

`ensemble_members/` is only used when `dojo ensemble` materializes member artifacts locally. Local member files may be symlinked; remote member files may be cached through `storage` config and then symlinked.

Logging and diagnostics

Experiment logging applies only to model-training runs. `dojo ensemble` and `dojo ensemble candidates` do not initialize experiment logging.

Sinks:

- `local` — functional. **The only functional sink for the foreseeable future**; metrics and figures are recorded locally.
- `aim` — deferred; **not a registered sink type**. See `appendix-deferred-features.md`.
- `mlflow` — deferred; **not a registered sink type**. See `appendix-deferred-features.md`.

Multi-sink composition is supported via `CompositeExperimentLogger`, but `local` is the only registered sink, so functional configurations use `local` alone. There is no artificial cap on sink count. A configuration naming `aim` or `mlflow` fails schema validation at load (unknown sink type) per `12-validation-testing-and-preflight.md`.

`logging` lives under `training_outputs.logging`:

```yaml
training_outputs:
  logging:
    sinks:
      - type: local
```

The logger abstraction:

```python
class ExperimentLogger:
    def log_config(self, cfg) -> None: ...
    def log_metrics(self, metrics, step=None) -> None: ...
    def log_artifact(self, local_path, artifact_path) -> None: ...
    def close(self) -> None: ...
```

Artifacts are produced according to result / export / checkpoint configuration; logger sinks may register or upload them after local creation. Logger sinks should play nicely with Lightning's logging system.

Cross-References

- `03-configuration.md` — `output_root` and `*_outputs` resolution, `results:` / `metrics:` / `figures:` / `export:` sub-block placement.
- `02-cli-and-task-types.md` — which commands write what kinds of results.
- `04-data-and-storage.md` — `source_extra_columns` and tabular feature configuration drive the sidecar contents.
- `05-models-training-and-heads.md` — heads / objectives / target transforms inform record types and sidecar `record_types` blocks.
- `07-ssl-and-representation-eval.md` — representation-evaluation record types and `evaluation_name`.
- `08-ensembles.md` — `ensemble_result_scope`, `ensemble_member_id`, `stage=ensemble_eval` rows, namespacing in shared directories, manifest JSON files.
- `09-sweeps-and-batch-runs.md` — `sweep_id` / `sweep_hash` provenance columns and `sweep_outputs/` layout.
- `10-export.md` — `exports/` sub-directory and export metadata.
- `12-validation-testing-and-preflight.md` — strict-schema deferral policy covering the Aim and MLflow sinks.
- `appendix-deferred-features.md` — deferred Aim and MLflow sinks; HDF / `.h5` result exports.
- `glossary.md` — `split`, `stage`, `record_type`, `embedding_kind`, `head_name`, identifier / hash vocabulary.

07. SSL and Representation Evaluation

Purpose

Defines self-supervised training (functional: DINOv2 via Lightly) and the `representation_eval` config group, which replaces the old `ssl_eval`. Representation evaluation is **not SSL-only** — it works against any task that produces image embeddings, including supervised models.

SSL

Framework

Use Lightly only for SSL method implementations. Do not depend directly on Meta DINOv2 repositories. The initial SSL task focuses on DINOv2-style training using Lightly components.

`ssl.framework: lightly` selects the Lightly framework. The backbone architecture for an SSL run is still selected via `model.image_input.backbone.architecture.source: timm | torchvision`. Checkpoint initialization uses `model.image_input.backbone.weights.source: checkpoint`. There is no `model.image_input.backbone.architecture.sour` `lightly`.

DINOv2 typically uses a timm ViT backbone. See the Lightly DINOv2 example at https://docs.lightly.ai/self-supervised-learning/examples/dinov2.html.

Functional method

```yaml
ssl:
  method: dino_v2
  framework: lightly
```

`dino_v2` through Lightly is functional in the initial implementation.

Deferred SSL methods

SimCLR, VICReg, PMSN, and the original DINO method are intentionally out of scope for the initial implementation. They are **not** `ssl.method` schema values, so authoring `ssl.method: simclr | vicreg | pmsn | dino` fails generic validation as an out-of-enum value. See `appendix-deferred-features.md`.

Model structure

```
image views
  ↓
backbone encoder (timm/torchvision/checkpoint)
  ↓
projection head
  ↓
DINOv2-style SSL loss
```

The SSL task module wraps the encoder + projection head, applies the SSL-specific multi-view transform, and produces canonical results (embeddings, SSL diagnostics) through the same result writer as supervised runs.

SSL training example

```yaml
task:
  type: ssl

ssl:
  method: dino_v2
  framework: lightly
  image_size: 224
  projection_dim: 65536

model:
  image_input:
    backbone:
      architecture:
        source: timm
        name: vit_small_patch14_dinov2
      weights:
```

```
        source: none

representation_eval:
  enabled: true
  name: ssl_epoch_eval
  schedule:
    mode: every_n_epochs
    every_n_epochs: 5
    include_fit_end: true
  dataset:
    splits:
      reference: train
      query: val
    seed: 123
  embeddings:
    enabled: true
    kinds: [image_embedding]
    cache: true
  projections:
    enabled: true
    methods:
      - type: umap
      - type: tsne
  clustering:
    enabled: true
    methods:
      - type: hdbscan
  probes:
    classification:
      enabled: true
      targets: [species]
    regression:
      enabled: true
      targets: [biovolume]
    ordinal:
      enabled: true
      targets: [quality_grade]
```

SSL encoder export

SSL training produces standard `.ckpt` training artifacts and may export a portable encoder via `training_outputs.export` or `dojo export`. The exported encoder is the standard input for supervised transfer learning (see `05-models-training-and-heads`

Representation evaluation

`representation_eval` is a top-level config group usable with any task that produces image embeddings, including `task.type: supervised` and `task.type: ssl`. A supervised run may schedule representation evaluation against its own encoder during training. The standalone `dojo eval representation` command works against checkpoints from either task type.

Canonical config shape

`representation_eval` is one schema with optional sub-blocks. Disabled sub-blocks are ignored; enabled sub-blocks validate their required fields and write canonical result rows.

```
representation_eval:
  enabled: true
```

```yaml
name: default_repr_eval

schedule:
  mode: fit_end        # disabled | fit_end | every_n_epochs | every_n_steps | fractional_epoch
  every_n_epochs: null
  every_n_steps: null
  fractional_epoch: null
  include_fit_end: true

dataset:
  splits:
    reference: train
    query: val
  max_reference_samples: null
  max_query_samples: null
  include_reference_outputs: false
  require_labels: auto
  seed: 123

embeddings:
  enabled: true
  kinds: [image_embedding]
  batch_size: null
  cache: true

diagnostics:
  enabled: true
  metrics: [embedding_norm, per_dimension_std, effective_rank]

nearest_neighbors:
  enabled: false
  k: [1, 5, 10]
  distance: cosine
  write_neighbors: true
  write_knn_predictions: true

probes:
  classification:
    enabled: false
    targets: []
    model: linear_classifier
  regression:
    enabled: false
    targets: []
    model: ridge
  ordinal:
    enabled: false
    targets: []
    model: ordinal_logistic_regression

projections:
  enabled: false
  methods:
    - type: pca
      n_components: 2
    - type: umap
```

```
      n_components: 2
    - type: tsne
      n_components: 2

clustering:
  enabled: false
  methods:
    - type: kmeans
      n_clusters: auto
    - type: mini_batch_kmeans
      n_clusters: auto
    - type: hdbscan
    - type: agglomerative
      n_clusters: auto

outliers:
  enabled: false
  methods:
    - type: local_outlier_factor
    - type: isolation_forest
```

`representation_eval.name` becomes `evaluation_name` on result rows and disambiguates multiple scheduled / standalone evaluations in the same run. `dataset.seed` controls deterministic subsampling and any representation-eval algorithm with a random state.

Supported components

- Embedding extraction.
- Embedding diagnostics (collapse checks, variance, effective rank, augmentation consistency, etc.).
- Dimensionality reduction: PCA, UMAP, t-SNE.
- Clustering: kmeans, mini_batch_kmeans, HDBSCAN, agglomerative (optional).
- Nearest-neighbor retrieval (labeled or unlabeled).
- Probes:
  - classification (linear, multinomial-logistic);
  - regression (ridge);
  - ordinal (ordinal logistic regression).
- Outlier / novelty scoring.

UMAP, t-SNE, and HDBSCAN are gated by the `repr_eval` optional extra but are **not deferred** — they are functional when the extra is installed. Regression and ordinal probes are functional. Supervised fine-tuning from an SSL pretrained backbone is supervised transfer learning, not a representation-evaluation mode — see `05-models-training-and-heads.md`.

Reference/query split semantics

Representation evaluation has two logical sample sets:

- **reference** — fit set for learned evaluation artifacts: probe models, k-NN reference indexes, projection fit steps when the method supports transform, clustering fit steps, and outlier/density reference models.
- **query** — scored / predicted / assigned set that produces the primary output rows.

Defaults: `reference=train`, `query=val`. Learned evaluation artifacts fit on the reference split only, then score the query split. Set `include_reference_outputs: true` to also write predictions, projections, cluster assignments, or outlier scores for the reference samples. Methods that do not support a clean fit/transform split, such as t-SNE and HDBSCAN, fit on the union of reference + query embeddings for visualization / clustering, but query rows remain the primary reported outputs unless `include_reference_outputs` is enabled.

Subsampling is deterministic per split from `representation_eval.dataset.seed`. When subsampling is enabled, written rows and sidecar metadata record the sample counts, requested limits, and seed.

Label-required vs. label-free evaluation

record_type	Requires labels	Notes
`embedding`	No	Any image.
`nearest_neighbor`	No	Retrieval / supporting record for `knn_prediction`.
`knn_prediction`	Yes	k-NN classification.
`classification_probe_prediction`	Yes	Linear / multinomial probes.
`regression_probe_prediction`	Yes	Ridge / similar probes.
`ordinal_probe_prediction`	Yes	Ordinal probes.
`cluster_assignment`	No	Labels optional for ARI / NMI / purity later.
`projection`	No	PCA / UMAP / t-SNE coords; labels optional for coloring.
`outlier_score`	No	Density / distance / novelty.
`diagnostic`	No	Collapse / variance / augmentation consistency.

Probe `head_name` should identify the probe (e.g. `linear_probe_species`, `ridge_probe_biovolume`). `probe_model_type` documents the probe class (e.g. `ridge`, `linear_classifier`, `ordinal_logistic_regression`). Probe predictions reuse the matching native head record column shape but with the probe-specific `record_type`. See `06-results-artifacts-and-metadata.md`.

Validation contract

Pydantic validation enforces:

- `enabled: false` disables the whole block; enabled sub-blocks validate independently.
- `schedule.mode: disabled` is allowed only for standalone configs or when the block is present but intentionally inactive during training.
- `schedule.mode: every_n_epochs` requires `every_n_epochs`; `every_n_steps` requires `every_n_steps`; `fractional_epoch` requires `fractional_epoch` in `(0, 1]`.
- `dataset.splits.reference` and `dataset.splits.query` must be valid dataset splits for the active command.
- `max_reference_samples` and `max_query_samples`, when set, must be positive integers.
- Probe `targets` must reference `data.targets`, and the probe family must match the target type: classification probes for classification targets, regression probes for regression targets, and ordinal probes for `ordinal_classification` targets.
- `nearest_neighbors.write_knn_predictions: true` requires classification labels for the reference and query samples selected by the active splits.
- `require_labels: auto` derives the requirement from enabled components; `true` errors if any selected sample lacks required labels; `false` allows label-free components only and rejects enabled probes / k-NN predictions.
- Standalone `dojo eval representation` must configure an encoder through `model.image_input.backbone.weights.sourc checkpoint` or another buildable checkpoint-backed model config. Training-integrated evaluation uses the current run encoder.
- UMAP, t-SNE, HDBSCAN, sklearn probes, clustering, and outlier methods are gated by the `repr_eval` optional extra. Missing extras raise a clear runtime dependency error, not a deferred-feature error.

Scheduling

Training-integrated evaluations are schedulable by:

```
disabled
fit_end
every_n_epochs
every_n_steps
fractional_epoch
```

`fractional_epoch: 0.10` means approximately every 10% of an epoch. `include_fit_end: true` adds one final evaluation at training end even when the main schedule is epoch-, step-, or fractional-epoch-based.

Expensive evaluations support subsampling:

```yaml
representation_eval:
  dataset:
    max_reference_samples: 50000
    max_query_samples: 10000
```

Execution and result mapping

Both standalone and training-integrated paths use the same evaluator:

```
build / load encoder
extract embeddings for reference and query splits
optionally cache embeddings for downstream components
run diagnostics
build nearest-neighbor index and optional k-NN predictions
fit probes on reference embeddings and predict query embeddings
fit / transform projections
fit / assign clusters
fit / score outliers
write canonical result rows and metrics / figures
```

Standalone vs. training-integrated

Same evaluator code, two entry points:

- training-integrated callbacks (scheduled inside a `dojo train` run), writing into that run's `training_outputs`;
- standalone CLI: `dojo eval representation`, writing rows / metrics and an `eval_manifest.json` into `eval_outputs` (`03-configuration.md`).

Standalone example:

```
dojo eval representation \
  experiment=ifcb/dinov2_repr_eval \
  model.image_input.backbone.weights.source=checkpoint \
  model.image_input.backbone.weights.uri=./runs/dinov2/checkpoints/best.ckpt \
  representation_eval.dataset.splits.query=holdout
```

Result mapping:

Config block	Result rows
`embeddings`	`record_type=embedding`
`diagnostics`	`record_type=diagnostic`
`nearest_neighbors.write_neighbors`	`record_type=nearest_neighbor`
`nearest_neighbors.write_knn_predictions`	`record_type=knn_prediction`
`probes.classification`	`record_type=classification_probe_prediction`
`probes.regression`	`record_type=regression_probe_prediction`
`probes.ordinal`	`record_type=ordinal_probe_prediction`
`projections`	`record_type=projection`
`clustering`	`record_type=cluster_assignment`
`outliers`	`record_type=outlier_score`

Training-integrated callbacks write rows, metrics, and figures under the current `training_outputs` directory. Standalone `dojo eval representation` writes rows / metrics / figures and `eval_manifest.json` under `eval_outputs`.

Diagnostics

Useful `diagnostic_name` values for the `diagnostic` record type:

```
embedding_norm_mean
embedding_norm_std
per_dimension_std
effective_rank
covariance_condition
pairwise_cosine_mean
pairwise_cosine_std
augmentation_consistency_cosine
```

`diagnostic_scope` values: `sample`, `batch`, `epoch`, `dataset`.

Cross-References

- `02-cli-and-task-types.md` — `task.type: ssl` and `dojo eval representation`.
- `03-configuration.md` — placement of `ssl:` and `representation_eval:`.
- `05-models-training-and-heads.md` — backbone sources used by SSL, supervised transfer learning from SSL exports, supervised representation-eval scheduling.
- `06-results-artifacts-and-metadata.md` — representation-evaluation record types and `evaluation_name`.
- `11-dependencies.md` — `ssl` and `repr_eval` optional extras.
- `appendix-deferred-features.md` — non-DINOv2 SSL methods.
- `glossary.md` — record-type vocabulary.

08. Ensembles

Purpose

Defines prediction-space ensembling: candidate discovery, candidate manifests, compatibility checking, selection strategies, combine modes, the `dojo ensemble` / `dojo ensemble candidates` commands, and the `ensemble_outputs:` block. Cross-run and snapshot ensembling are candidate-source patterns, not separate ensemble algorithms.

Concept

Ensembling is **prediction-space** multi-model output processing:

```
candidate discovery
candidate compatibility validation
candidate scoring
ensemble selection
ensemble construction
ensemble evaluation
artifact bundling
canonical result export
```

It is not a separate training algorithm. There are no `cross_run_ensemble` or `snapshot_ensemble` ensemble algorithm types — those are candidate sources for the one prediction-space ensemble pipeline. Weight-space ensembles (model soup, SWA, EMA) are deferred — see `appendix-deferred-features.md`. `torchensemble` is dropped as a dependency; Bagging / Boosting / Fusion / Adversarial / FastGeometric strategies are not ported.

Commands

- `dojo ensemble` — ensemble evaluation / inference against a candidate manifest or candidate-discovery sources.
- `dojo ensemble candidates` — artifact-inspection / manifest-writing command. Does not initialize experiment logging.

Cross-run ensembling works via `dojo ensemble` with `run_dir_glob` or explicit `run_dir` candidates. Snapshot ensembling against a brand-new training run is `task.type: snapshot_ensemble` (see `02-cli-and-task-types.md`). Re-running ensembling against a historical training run is a plain `dojo ensemble` invocation with a `run_dir` candidate, or a `run_dir_glob` source that matches one or more run directories.

Candidate discovery

Candidate discovery in the initial implementation is **limited to these explicit source types**:

- `explicit` — inline candidate list.
- `run_dir_glob` — Dojo training-run directories.
- `checkpoint_glob` — raw checkpoint files.
- `result_uri_glob` — cached result / prediction directories.
- `manifest` — pre-built candidate manifest.

Broad automatic registry-based cross-run discovery is deferred — see `appendix-deferred-features.md`.

For `task.type: snapshot_ensemble`, candidates are not drawn from these explicit sources: the just-finished run's cycle-end snapshot checkpoints (under `<training_outputs.dir>/checkpoints/`) are an **implicit** candidate source, discovered internally. They are ordinary `checkpoint`-kind candidates, and `ensemble.selection.strategy` chooses among them with the normal strategies — `all` to ensemble every cycle snapshot, `top_k` or `greedy_forward_selection` to use a subset. There is no snapshot-specific selection strategy.

Manifest output is JSON (not Parquet) under `ensemble_outputs.manifests.dir`, default `{ensemble_outputs.dir}/ensemble_m`
Every `dojo ensemble` run writes a manifest for provenance, even when member prediction rows are not materialized into `ensemble_results/`. The manifest records discovered candidates, compatibility status, exclusion reasons when available, selected members, assigned `ensemble_member_id` values, source result selectors / URIs, semantic compatibility status, member-provenance hashes, combine config, and selection config. Candidate-audit metadata is useful for inspection, but `ensemble_hash` is derived from the selected-ensemble identity block (selected members + combine + selection), not from every discovered candidate.

To write a shared manifest outside a normal run directory:

```
dojo ensemble candidates experiment=ifcb/candidate_search \
  ensemble_outputs.manifests.dir=./shared_manifests
```

Candidate source types and artifact kinds

`sources[].type` says how Dojo discovers candidates. Candidate `kind` says what artifact each discovered candidate represents.

Canonical source enum:

```
explicit
run_dir_glob
checkpoint_glob
result_uri_glob
manifest
```

Candidate artifact kinds:

- `run_dir` — Dojo training-run directory. Preferred for normal historical-run ensembling because Dojo can read resolved config, checkpoint metadata, validation metrics, and cached result artifacts.
- `checkpoint` — raw model checkpoint file. Requires enough config metadata to rebuild the model; Dojo may need to run validation / inference to produce comparable per-model metrics.
- `result_uri` — cached predictions / result rows. Useful when model files are unavailable or inference should not be rerun.
- `exported_model` — portable `.pt` / `.onnx` export with enough config metadata to run inference if needed.

Each member-level row carries an `ensemble_member_id` union column equal to the member's `checkpoint_hash` (checkpoint member) or `model_id` (exported model member). Combined ensemble rows have `ensemble_result_scope=ensemble` and no `ensemble_member_id`. See `06-results-artifacts-and-metadata.md`.

Discovery example

```
ensemble:
  candidates:
```

```yaml
    sources:
      - type: run_dir_glob
        uri_glob: s3://dojo-runs/ifcb_species/*/
      - type: checkpoint_glob
        uri_glob: ./checkpoints/*.ckpt
        config_uri: ./configs/ensemble_member_base.yaml
      - type: result_uri_glob
        uri_glob: ./cached_predictions/*/results/
      - type: manifest
        manifest_uri: ./shared_manifests/ifcb_candidates.json
      - type: explicit
        candidates:
          - kind: run_dir
            uri: ./runs/resnet50_a
          - kind: checkpoint
            uri: ./checkpoints/convnext_b.ckpt
            config_uri: ./configs/convnext_b.yaml
          - kind: result_uri
            uri: ./runs/convnext_b/results
          - kind: exported_model
            uri: ./exports/efficientnet_d/model.pt
            config_uri: ./exports/efficientnet_d/config/resolved.yaml
    metadata_resolution:
      policy: cascade
      order:
        - resolved_config
        - result_metadata
        - checkpoint
        - exported_model
      drift_check:
        enabled: true
        compare: [resolved_config, checkpoint, result_metadata]
  target:
    split: val
    dataset_id: ifcb_species_v4   # use dataset_hash when dataset does not self-name
  source_policy: inference_as_needed
```

When `ensemble.source_policy` can run member inference (`inference_as_needed` or `force_inference`), the active command also requires a normal `data:` block describing the target dataset. The `ensemble.target` fields are the semantic target selector and cache-matching contract; they are not a replacement for the dataset backend configuration. For `strict_no_inference` runs that use only cached result rows, `data:` may be omitted when the cached result metadata supplies the target `dataset_hash` / `dataset_id` and split.

Authored config vs. runtime artifacts

Authored config describes candidate sources, target split, source policy, selection strategy, combine modes, and output-retention policy. It should not enumerate discovered compatible candidates, selected members, assigned `ensemble_member_id` values, measured cost fields, or row selectors unless the user is explicitly providing an `explicit` candidate source.

Resolved config fills defaults and concrete output paths. When `ensemble_outputs.dir_template` contains `{ensemble_id}`, that path resolves after candidate discovery, compatibility validation, selection, and selected-ensemble identity generation.

The ensemble manifest is the execution artifact that records what the run actually found and selected: all discovered candidates, compatible candidates, incompatible candidates with reasons when practical, selected members, source result selectors, hashes, cost metadata when known, and the selection / combine config snapshot.

Metadata resolution

Candidate source type controls discovery. `metadata_resolution` controls where Dojo reads candidate metadata after candidates are discovered. Metadata includes target schema, class mapping, preprocessing / transform config, model config, dataset identity, checkpoint provenance, and cached-result provenance.

Initial `metadata_resolution.policy` values:

- `cascade` — read metadata sources in `order` and use the first available authoritative value for each field. **Default.**
- `strict` — require all configured / available metadata sources for a candidate to agree on compatibility-critical fields.

Initial metadata source names:

- `resolved_config` — Dojo `config/resolved.yaml` or `resolved.json` from a run directory or explicit `config_uri`.
- `result_metadata` — metadata stored alongside cached result / prediction files.
- `checkpoint` — metadata embedded in or adjacent to a `.ckpt` member.
- `exported_model` — metadata embedded in or adjacent to a portable `.pt` / `.onnx` export.

`drift_check` is a validation pass, not a discovery mechanism. When enabled, Dojo compares the listed metadata sources if more than one is available for a candidate. Any mismatch in compatibility-critical fields is reported according to the policy; for the initial implementation, drift in target schema, class mapping, preprocessing, or model-output shape is an error.

Compatibility

Hard compatibility requirements for a prediction-space ensemble:

```
sample identity semantics
required input fields available
target schema
head names
head task types
class mappings
ordinal encoding / decoding rules
regression target units and scaling
output tensor shapes
output tensor meanings
```

Per-head compatibility — a candidate may be excluded from one head and included for another if multi-head compatibility allows.

Compatibility assessment uses a cascading metadata policy by default:

1. `config/resolved.json`
2. result metadata
3. checkpoint metadata
4. exported model metadata

Explicit per-metadata-source policies and drift checks across multiple metadata targets are supported. Cross-member fast-equality checks apply to semantic output contracts such as `target_schema_hash` and `class_mapping_hash`. `model_config_hash` and `preprocessing_hash` are member-provenance hashes: they validate a member's own artifacts and cached rows, and they support drift checks across metadata sources for the same candidate, but they are **not** required to match across ensemble members. See `06-results-artifacts-and-metadata.md` for hash field selection rules and the human-readable diff path on mismatch.

Member-specific preprocessing

Candidate models may have different preprocessing requirements (image size, normalization, crop, tabular feature transforms, etc.). Member preprocessing metadata travels with each member; the ensemble runner applies preprocess-

ing per-member when necessary, with an optional shared-preprocessing fast path when all members agree on input shape / normalization / etc.

This enables heterogeneous ensembles such as ViT@224 + ConvNeXt@320 + EfficientNet@384 against the same evaluation dataset.

`preprocessing_hash` equality is only a fast-path signal for shared preprocessing. A mismatch does not make members incompatible as long as the evaluation data provides each member's required input fields and the member's own preprocessing metadata is available. Drift within one candidate remains an error when multiple metadata sources disagree about that candidate's preprocessing contract.

Source policy

Controls behavior when candidate-result coverage is partial:

- `strict_no_inference` — refuse to run; only cached results are used.
- `inference_as_needed` — run inference for any missing rows.
- `force_inference` — re-run inference for every member ignoring caches.

Ensemble commands default to using existing result files as inputs when input dataset and output target match (`inference_as_needed`). Cached-result inputs must match the ensemble target by `dataset_hash`, or by explicit `dataset_id` plus `split` when the dataset self-names and a content hash is unavailable. Target schema, class mapping, output tensor shape, and output tensor meaning still govern cached-result compatibility. `model_config_hash` and `preprocessing_hash` validate cached rows against their own producing member when present; they do not need to match other members.

Selection strategies

Supported in the initial implementation:

- `all` — use every supplied candidate.
- `best_candidate` — select the single best candidate by the configured metric (useful as a baseline / control).
- `top_k` — top K candidates by validation metric.
- `greedy_forward_selection` — start with the best single candidate and iteratively add the candidate that most improves ensemble validation performance.

Snapshot ensembles (`task.type: snapshot_ensemble`) reuse these same strategies over the run's implicit cycle-snapshot candidates (see "Candidate discovery" above); there is no separate snapshot selection strategy.

Deferred selection / weighted strategies: see `appendix-deferred-features.md`.

Selection examples

```yaml
ensemble:
  selection:
    strategy: top_k
    k: 5
    metric: val/species/f1_macro
    mode: max
```

```yaml
ensemble:
  selection:
    strategy: greedy_forward_selection
    metric: val/species/f1_macro
    mode: max
    max_members: 8
    stop_if_no_improvement: true
```

Selection config is normal Dojo/Hydra-composed config and can be varied through the top-level `sweep:` block. For example, Dojo sweeps may vary `ensemble.selection.strategy`, `ensemble.selection.max_members`, or combine-mode settings to compare which candidate subsets contribute most to ensemble quality.

Candidate metadata may include optional cost fields such as inference latency, parameter count, FLOPs, and peak memory when known. Initial metrics / figures can compare quality against these costs; cost-aware selection objectives are an extension on top of the same manifest fields.

Combine modes

Classification:

- `probabilities_mean`
- `logits_mean`
- `majority_vote`

Regression:

- `prediction_mean`
- `prediction_median`

Ordinal:

- `ordinal_probabilities_mean`
- `ordinal_logits_mean`

Deferred combine modes

- weighted combine modes (weighted logits / probabilities / vote / mean);
- `prediction_trimmed_mean`.

Combine internals

`logits_mean` (classification):

```
ensemble_logits = mean(member_logits)
probabilities = softmax(ensemble_logits)
prediction_index = argmax(probabilities)
prediction_confidence = max(probabilities)
```

`probabilities_mean` (classification):

```
member_probabilities = [softmax(logits_1), softmax(logits_2), ...]
probabilities = mean(member_probabilities)
prediction_index = argmax(probabilities)
prediction_confidence = max(probabilities)
```

`probabilities_mean` is usually safer when combining heterogeneous models because it reduces sensitivity to different logit scales.

`majority_vote` (classification):

```
member_votes = [argmax(member_output_i) for each member]
prediction_index = the class with the most member votes
                   # ties broken deterministically by lowest class index
prediction_confidence = vote_count(prediction_index) / num_members
```

Hard voting needs only each member's `prediction_index`, which is why it is the one classification mode that combines from members that logged a prediction but no `logits` / `probabilities`. Ties are broken deterministically by **lowest class index** (no RNG).

Regression:

```
prediction_value = mean(member_predictions)
prediction_uncertainty = std(member_predictions)
```

Ordinal: average ordinal logits (or ordinal probabilities) then decode through the configured ordinal decoding rule (`model.heads.<name>.ordinal.decoding`). The ensemble artifact records the decoding rule used per ordinal head.

Using cached results correctly

Cached-result ensembling (`source_policy: strict_no_inference`, or `inference_as_needed` when all rows are present in cache) reads member predictions from existing result Parquet files instead of re-running inference. The combine math is identical; what changes is that each chosen combine mode imposes a column requirement on every member's cached results.

Required columns per combine mode (column definitions live in `06-results-artifacts-and-metadata.md`):

Combine mode	Required cached columns
`logits_mean` (classification)	`logits`
`probabilities_mean` (classification)	`probabilities` (or `logits` to derive)
`majority_vote` (classification)	`prediction_index`
`prediction_mean` / `prediction_median` (regression)	`prediction_value` (use `prediction_value_internal` if combining in transformed space)
`ordinal_logits_mean`	`ordinal_logits`
`ordinal_probabilities_mean`	`probabilities` (per-bin)

Every member must also carry the standard provenance columns (`sample_id`, `head_name`) and the partition keys needed to locate its rows; see `06-results-artifacts-and-metadata.md` for the full schema.

Notes and limitations:

- Cached rows are joined across members by `sample_id` at the configured target split. Members missing rows under `strict_no_inference` fail the run; under `inference_as_needed` the runner fills gaps by running that member's inference.
- A member whose cached results omit a column required by the chosen combine mode (e.g. only `prediction_index` logged, but the ensemble asks for `logits_mean`) cannot participate from cache; either switch combine mode, drop the member, or use `inference_as_needed` / `force_inference` to re-derive the needed columns.
- Compatibility checks (`target_schema_hash`, `class_mapping_hash`, ordinal encoding rule, regression units) apply identically to cached and fresh-inference ensembling.
- Per-member preprocessing differences do not matter for cached-result combine — predictions already reflect each member's preprocessing.
- For regression, choose `prediction_value` vs. `prediction_value_internal` deliberately and consistently across members; mixing external- and internal-space combines is not supported.

Ensemble run example

```yaml
data:
  backend: parquet_manifest
  manifest_uri: ./data/ifcb_holdout_manifest.parquet
  sample_id_column: roi_id
  image_uri_column: image_uri
  split_column: split
  targets:
    species:
      column: species_idx
      type: multiclass_classification
      class_names: ./data/species_classes.json
      missing_policy: error
    biovolume:
      column: biovolume_um3
      type: regression
      transform: log1p_standardize
      missing_policy: drop_sample
    quality_grade:
```

```yaml
      column: quality_grade_idx
      type: ordinal_classification
      class_names: ./data/quality_grade_classes.json
      missing_policy: drop_sample

ensemble:
  candidates:
    sources:
      - type: manifest
        manifest_uri: ./shared_manifests/ifcb_candidates.json
  target:
    split: holdout
    dataset_id: ifcb_species_holdout_v4
  source_policy: inference_as_needed
  selection:
    strategy: greedy_forward_selection
    metric: val/species/f1_macro
    mode: max
    max_members: 8
  inference:
    combine:
      classification: probabilities_mean
      regression: prediction_median
      ordinal: ordinal_probabilities_mean

ensemble_outputs:
  dir_template: "{experiment.name}/ensembles/{ensemble_id}"
  results:
    member_results:
      mode: none
```

With `source_policy: inference_as_needed`, Dojo first uses compatible cached member result rows from the candidate manifest, then runs inference for any selected member / sample rows that are missing. The `data:` block above is therefore required: it defines the target dataset to load for fresh member inference, while `ensemble.target` names the split and dataset identity used to validate cached rows.

`ensemble_outputs:` block

Peer to `training_outputs:` and `sweep_outputs:`. Sub-blocks: `results`, `export`, `metrics`, `figures`, `manifests`, `members`.

Two member-related keys are distinct: `members` configures the `ensemble_members/` directory of locally materialized member **artifact files** (checkpoints / exports, symlinked or cached), while `results.member_results.mode` controls whether selected member **result rows** are written into `ensemble_results/`. One is files, the other is rows.

Default on-disk sub-directories under the resolved `ensemble_outputs.dir`:

```
ensemble_outputs.dir/
  config/
  exports/
  metrics/
  ensemble_figures/
  ensemble_results/
  ensemble_manifests/
  ensemble_members/        # optional; only when materialized locally
```

Member materialization to `ensemble_members/` is only allowed for `dojo ensemble` runs. Local member files may be symlinked; remote member files may be cached through `storage` config and symlinked. Member files are never

re-uploaded by Dojo.

Ensemble metrics and figures should compare member best/final metrics against the resulting ensemble model.

For `task.type: snapshot_ensemble`, `ensemble_outputs.dir_template` defaults to match `training_outputs.dir_template` so training and ensemble outputs share one run directory. In that shared directory:

- training result rows are written under `results/` and ensemble result rows are written under `ensemble_results/`, so the two writers do not collide even when the top-level output directory is shared;
- row `stage` remains semantic provenance (`train_validation` vs. `ensemble_eval`), not the primary collision-avoidance mechanism;
- metrics / figures filenames include the producing block when namespacing is needed;
- when `--clobber` is used, each resolved physical directory is cleared **once** at startup, so the ensemble step does not delete training artifacts the training step just wrote.

Member result retention

`ensemble.source_policy` controls how member predictions are obtained for ensemble math. `ensemble_outputs.results.member` controls whether selected member prediction rows are retained under `ensemble_outputs.results.dir`.

Initial modes:

- `none` — default. Write combined ensemble rows and the always-written ensemble manifest only. Do not materialize member-level rows into `ensemble_results/`.
- `transcribe` — require source result rows for every selected member, validate dataset / split, semantic output compatibility, and member-provenance hashes, then rewrite those rows into canonical ensemble form with `stage=ensemble_eval`, `ensemble_result_scope=member`, `ensemble_id`, `ensemble_hash`, and `ensemble_member_id`. This mode does not run inference just to retain rows.
- `inference` — require `ensemble.source_policy: force_inference`; run selected member inference and write fresh member-level rows into `ensemble_results/`.

Retention applies only to selected ensemble members, not every discovered compatible candidate. With `mode: none`, member identity is still known at execution time from the selected-member manifest, but no member-level result rows or `ensemble_member_id` values are persisted in `ensemble_results/`.

Ensemble result records

Ensemble result rows use the standard result schema plus ensemble provenance columns.

- All rows written by the ensemble pipeline use `stage=ensemble_eval`.
- Combined ensemble rows use `ensemble_result_scope=ensemble` and leave `ensemble_member_id` null.
- Retained member rows use `ensemble_result_scope=member` and populate `ensemble_member_id` with the member's `checkpoint_hash` or `model_id`.
- Cached source rows that remain in their source result dataset keep their original `stage`; if transcribed into `ensemble_results/`, they are rewritten as ensemble-produced rows.

Default `ensemble_outputs.results.partition_by` is `[stage, record_type, ensemble_result_scope]`. `ensemble_member_id` remains available as a filter column and may be used as an opt-in partition key for member-only analysis outputs.

Cross-References

- `02-cli-and-task-types.md` — `dojo ensemble`, `dojo ensemble candidates`, `task.type: snapshot_ensemble`.
- `03-configuration.md` — placement of `ensemble:` and `ensemble_outputs:`, snapshot-ensemble directory sharing.
- `05-models-training-and-heads.md` — snapshot-cycle scheduler and checkpointing.
- `06-results-artifacts-and-metadata.md` — `ensemble_result_scope`, `ensemble_member_id`, `stage=ensemble_eval` rows, compatibility and member-provenance hashes, `ensemble_manifests/` JSON manifests, partitioning.
- `09-sweeps-and-batch-runs.md` — Dojo-prepared sweeps over ensemble selection / combine axes; sweeps as candidate-source feeders.

- `10-export.md` — `ensemble_outputs.export` and ensemble model artifacts.
- `appendix-deferred-features.md` — weight-space ensembles, weighted combine modes, `prediction_trimmed_mean`, registry-based candidate discovery.
- `glossary.md` — `ensemble_member_id`, `ensemble_id`, candidate / selection vocabulary.

09. Sweeps and Batch Runs

Purpose

Defines config-defined sweeps (Dojo-owned expansion over the Hydra Compose API), batch-run-style grid sweeps, deferred Bayesian / AutoML HPO, and the `sweep_outputs:` block. Sweeps can explore training hyperparameters, ensemble strategy / combine-mode combinations, random-seed sensitivity, or feed candidate manifests into a subsequent ensembling step. Bayesian / AutoML HPO is deferred.

Sweep preparation and manual execution

Dojo composes configs through the Hydra **Compose API** (`hydra.compose`), not `@hydra.main`, so there is no Hydra launcher, no Hydra-managed working directory, and no `chdir`. A sweep is prepared when `dojo sweep prepare` composes a config whose `sweep:` block defines axes (see "Sweep definition block" below). Dojo expands the cartesian product itself and owns runtime IDs, output-directory resolution, run-directory collision handling, and all canonical artifacts.

`dojo train` executes one concrete training run. It rejects authored / composed configs with an active `sweep.grid`; use `dojo sweep prepare` first, then run concrete jobs from the prepared sweep.

A sweep is prepared from either an experiment config that includes a `sweep:` block, or axes overridden onto `sweep.grid` from the command line (dash-free config overrides):

```
# the experiment config carries a sweep: block
dojo sweep prepare experiment=ifcb/sweep_lr_bs

# or supply axes as config overrides (Hydra list-value syntax)
dojo sweep prepare experiment=ifcb/experimentA \
  sweep.mode=grid \
  'sweep.grid.optimizer.lr=[1e-4,3e-4]' \
  'sweep.grid.training.batch_size=[32,64]'
```

There is no `-m` / `--multirun` flag and no CLI comma-list sweep shorthand; the `sweep:` block is the single sweep definition. No `+` is needed for `sweep.grid.*` overrides because `sweep.grid` is an open mapping in the root schema.

Process CWD never changes between runs. All Dojo paths are absolute or resolved relative to `output_root` (`03-configuration.md`), so there is no Hydra `os.getcwd()` footgun and no `_hydra/` coordination area — Dojo's resolved `training_outputs.dir`, `ensemble_outputs.dir`, and `sweep_outputs.dir` are the only output locations.

- `runtime.sweep_id` is generated once per sweep before concrete runs are expanded. `runtime.run_id` is generated once per concrete run after its sweep-axis values are known.
- Dojo generates all run IDs before any run starts and raises an error if collisions exist (see the resolution order below). Uniqueness comes from realized sweep values or the `{job_num}` sweep index (`sweep.active_run.index`), not from Hydra job metadata.

  [!NOTE] OmegaConf `${...}` interpolation is resolved by OmegaConf during composition. Dojo-owned output templates omit the `$` and use `{...}` patterns resolved by Dojo after composition, validation, and runtime-value generation (`03-configuration.md`). The two interpolation layers do not overlap.

  [!NOTE] **Deferred automated runners.** The initial execution mode is manual. Automated local sequential execution and Slurm / HPC queue submission are deferred to P4.16. Dojo owns expansion and artifact layout regardless of the later execution mechanism.

Initial sweep execution is manual:

```
dojo sweep prepare experiment=ifcb/experimentA \
  'sweep.grid.optimizer.lr=[1e-4,3e-4]'
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --index 0
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --run-id RUN_ID
dojo sweep status ./runs/ifcb/sweep_results/SWEEP_ID
dojo sweep report ./runs/ifcb/sweep_results/SWEEP_ID
```

`dojo sweep train` reads the sweep manifest, selects one training job by `--index` or `--run-id`, and invokes the same internal training path as `dojo train --resolved-config JOB_DIR/config/resolved.yaml`. Per-run status is written by the run itself; the central manifest remains the immutable prepared job index.

Sweep definition block

`sweep:` is the top-level algorithmic block for sweep generation. It is separate from `sweep_outputs:`, which controls sweep-level reporting artifacts.

```yaml
sweep:
  mode: grid
  conflict_policy: default
  execution:
    mode: manual
  grid:
    runtime.seed: [101, 102, 103]
    optimizer.lr: [1.0e-4, 3.0e-4]
    training.batch_size: [32, 64]
    model.image_input.backbone.architecture.name: [resnet50, convnext_tiny]
```

`sweep.mode` values:

- `grid` — functional; explicit cartesian product over parameter lists.

`bayesian` is deferred and **not a schema value**: authoring `sweep.mode: bayesian` fails validation as an out-of-enum value (see `appendix-deferred-features.md` P4.1).

`sweep.execution.mode` values:

- `manual` — functional initial mode. `dojo sweep prepare` writes the prepared sweep artifacts; users run concrete jobs explicitly.
- `local_sequential` — deferred to P4.16.
- `slurm` — deferred to P4.16.

Grid sweep syntax

Inside `sweep.grid`, each normalized key is a target config path and each value is a YAML list; the concrete runs are the cartesian product of those lists. Authored YAML may use flat dotted keys:

```yaml
sweep:
  grid:
    optimizer.lr: [1.0e-4, 3.0e-4]
```

CLI overrides naturally compose as nested mappings, e.g. `'sweep.grid.optimizer.lr=[1e-4,3e-4]'`. After Hydra composition and before validation, Dojo normalizes `sweep.grid` by flattening nested leaves under `sweep.grid` with dots, so that override becomes the target path `optimizer.lr`.

The `sweep:` block is the single sweep definition: there is no CLI comma-list shorthand and no normalization of axes scattered elsewhere in the config. After expansion, each concrete run receives ordinary resolved scalar config values at the target paths. `dojo train` rejects active `sweep.grid` values in authored / composed configs; `sweep.grid` is only expanded by `dojo sweep prepare`.

Commas, lists, and shell quoting

Comma handling differs between CLI overrides and config files:

- **CLI overrides** use Hydra's override grammar — Dojo relies on it for all `key=value` composition. The whole grammar is in play, with one exception, the *bare top-level comma*:
  - **Bare top-level comma = multirun, which Dojo rejects the direct use of.** In Hydra's grammar a comma that sits directly in the value (`optimizer.lr=1e-4,3e-4`) means "run once per listed value" — a *multirun sweep*. It is the only construct that requires Hydra's multirun machinery, and Dojo never runs *hydra* multirun: it composes branching/grids of single configs through the Compose API. So this comma styling override **errors during composition**. Sweeping is handled differently.
  - **Sweeps come only from the `sweep:` config block**.
  - **Commas inside a value are fine.** The rest of the grammar works normally, including commas that are structurally part of a value: list literals `[a,b]`, dict literals `{a:1,b:2}`, and Hydra-quoted strings `key='a,b'`. These are not top-level commas, so they never trigger multirun.

  The trap: `optimizer.lr=1e-4,3e-4` (top-level comma → errors) and `optimizer.lr=[1e-4,3e-4]` (list value → legal) look almost identical but mean opposite things. Author sweep axes as the bracketed form on `sweep.grid`, e.g. `'sweep.grid.optimizer.lr=[1e-4,3e-4]'`, never `optimizer.lr=1e-4,3e-4`.
- **Shell quoting:** single-quote any override token containing `[...]` or `{...}` so the shell does not glob- or brace-expand it before Hydra sees it (e.g. `'sweep.grid.training.batch_size=[32,64]'`). Tokens without brackets (`sweep.mode=grid`) need no quotes.
- **Config files** follow ordinary YAML: commas separate elements only inside flow collections (`[a, b]`, `{a: 1}`) and are literal characters in plain or quoted scalars. Hydra's comma-as-sweep grammar is a CLI-override behavior only — it never applies inside YAML, so `sweep.grid` axes are written as ordinary YAML lists.

Batch-run-style grid sweeps

A batch run is a grid sweep where the only intentional run-varying parameter is `runtime.seed`. This is useful for measuring model sensitivity to randomness while holding architecture, data, training parameters, and evaluation settings fixed.

```yaml
sweep:
  mode: grid
  grid:
    runtime.seed: [101, 102, 103, 104, 105]
```

Sweep reporting can then summarize metrics across seeds, for example with box plots / five-number summaries for `f1_macro` or per-class F1.

Sweep conflict policy

`sweep.conflict_policy` controls what happens when a swept target path also has a value in the non-sweep config:

- `default` — default. A `sweep.grid` entry for a target path overrides any value at that path elsewhere in the composed config, per concrete run.
- `strict` — the target path must be absent or null outside `sweep.grid`; any non-sweep value there raises a config-compilation error.

CLI config overrides supersede both the target config and `sweep:` definitions. Sweep axes supplied on the command line are written directly onto `sweep.grid` (e.g. `'sweep.grid.optimizer.lr=[1e-4,3e-4]'`), not inferred from comma-separated scalar overrides.

Active run metadata

`sweep.active_run` is generated only in resolved config artifacts and runtime config objects. It should not be written in source configs.

Example resolved-config fragment for one concrete run:

```yaml
sweep:
  mode: grid
  conflict_policy: default
```

```yaml
active_run:
  index: 3
  values:
    runtime.seed: 103
    optimizer.lr: 0.0001
    training.batch_size: 64
    model.image_input.backbone.architecture.name: convnext_tiny
```

Bayesian sweep (deferred)

Bayesian / AutoML HPO is deferred and **not represented in the schema**: `bayesian` is not a `sweep.mode` value and there is no `sweep.bayesian` block, so authoring either fails generic validation at load (out-of-enum value / unknown key). The eventual config shape — engine, metric, sampler, trial budget, and a per-parameter search space — is sketched in `appendix-deferred-features.md` (P4.1) and lands with the schema when the runtime is built.

Sweep preparation, training, status, and reporting order

Single-run `dojo train` uses the same run-level resolution rules as a prepared sweep job. If an authored / composed config has a non-empty `sweep.grid`, `dojo train` errors and points the user to `dojo sweep prepare`.

1. **Prepare the sweep with `dojo sweep prepare`**
    1. Compose the authored config and CLI overrides through Hydra.
    2. Validate `sweep.mode`, `sweep.grid`, and `sweep.execution`. `sweep.execution.mode: manual` is functional; deferred execution modes are not schema values and fail validation.
    3. Compute `sweep_hash` from stable sweep-definition inputs: base config, sweep mode, sweep axes / search params, and explicit sweep metadata. Exclude generated IDs, output paths, `output_root`, and all `*_outputs` blocks.
    4. Generate `runtime.sweep_id` once for the sweep: manual value as-is; template rendered from non-generated sweep / base config fields or already-computed hash sources such as `{coolname:sweep_hash}`; `{coolname:noseed}` as a fresh unseeded coolname; unset value falls back to `{coolname:sweep_hash}`. Bare `{coolname}` is deterministic from `runtime.seed` and is not the default.
    5. Expand the cartesian product into concrete jobs.
    6. For each job, apply that job's sweep-axis values, validate the concrete config, compute `config_hash`, generate `runtime.run_id` (default `{coolname:noseed}`; hash-seeded templates such as `{coolname:config_hash}` can render because `config_hash` already exists), and resolve per-run output directories in memory.
    7. Check all generated `run_id` values and resolved per-run output directories for collisions before writing any per-run artifacts. Collision errors name the affected sweep indices and suggest adding `{job_num}` or a realized sweep value such as `runtime.seed` to the run-id / directory template.
    8. Write per-run config artifacts:
        - `config/composed.yaml`
        - `config/resolved.yaml`
        - `config/resolved.json`
        - `config/cli.txt`
        - `config/overrides.txt`
        - `config/sweep_values.txt`
    9. Resolve `sweep_outputs.dir_template` to `sweep_outputs.dir` once for the sweep, then resolve sweep-output sub-block paths under it.
    10. Write sweep-level config artifacts under `sweep_outputs.dir/config/`:
    - `composed.yaml` — sweep-level composed config with `sweep.grid`;
    - `resolved.yaml` — sweep-level resolved config with finalized `runtime.sweep_id`, `sweep_hash`, and output paths, but no concrete `sweep.active_run`;
    - `resolved.json`;
    - `sweep_manifest.json` — immutable concrete job index;
    - `cli.txt`;
    - `overrides.txt`.
2. **Run one training job with `dojo sweep train`**
    1. Read `SWEEP_DIR/config/sweep_manifest.json`.

2. Select exactly one job by `--index` or `--run-id`.
3. Invoke the same internal path as `dojo train --resolved-config JOB_DIR/config/resolved.yaml`.
4. Use the same resolved-config guard as `dojo train`: if the selected run directory contains artifacts beyond prepared config / status files, require explicit `--resume` or `--clobber` behavior.
5. Write a simple per-run status file at `JOB_DIR/status.json`. `pending` is inferred from the manifest when the file does not exist. Written states are `initializing`, `training`, `exporting`, `done`, and `failed`.
6. Result rows include `run_id`, `config_hash`, and related run provenance. Sweep-produced result rows also include `sweep_id` and `sweep_hash`.

3. **Check status with `dojo sweep status`**
   1. `dojo sweep status SWEEP_DIR` lists all sweep indices, `run_id` values, key sweep values, and current status.
   2. `--index` or `--run-id` reports one job.
   3. Status is read live from the immutable manifest, per-run `status.json` files, and expected run artifacts. It does not mutate the manifest and does not write a sweep-level status cache.
   4. The command clearly reports whether all prepared jobs are `done`.
4. **Write sweep reports with `dojo sweep report`**
   1. Read `SWEEP_DIR/config/sweep_manifest.json`.
   2. Require all non-skipped manifest jobs to be `done`; otherwise list pending / running / failed jobs and exit without writing partial report artifacts.
   3. If `sweep_outputs.enabled: false`, exit cleanly after confirming reporting is disabled.
   4. Read each run's resolved config and metric artifacts from paths recorded in the manifest.
   5. Apply `sweep_outputs.collect`, write sweep-level metrics and figures, and promote / export sweep-level artifacts only if explicitly configured.

`sweep_outputs:` block

Peer to `training_outputs:` and `ensemble_outputs:`. Holds **sweep-level reporting** outputs.

Sub-blocks: `dir_template`, `enabled`, `collect`, `artifacts`, `metrics`, `figures`. There is **no** `results` sub-block — per-row data comes from the underlying per-job `training_outputs` / `ensemble_outputs`.

`sweep_outputs.enabled` defaults to `true`. When `false`, Dojo still prepares the sweep and concrete jobs may still run, but `dojo sweep report` skips sweep-level collection, summary metrics, aggregate figures, and sweep artifact promotion / export.

`sweep_outputs.collect` is a list of metric / artifact collection specs. Each item names what to collect from every concrete run and which per-run source to read. For metric specs, optional `mode` is `min` or `max` and controls sweep-level ranking / best-run selection for that collected metric:

```yaml
sweep_outputs:
  collect:
    - metric: val/species/f1_macro
      source: best
      mode: max
```

Initial `source` values:

- `best` — collect the value associated with the run's best checkpoint. By default this uses the run's `checkpointing.monitor` / `checkpointing.mode`; an explicit collect `mode` may be supplied for aggregation ranking.
- `last` — collect the final recorded value for the run.
- `all` — collect all recorded values for that metric across epochs / steps, for trend plots or post-hoc summaries.

Sweep reporting does not discover runs by scanning directories. During sweep preparation, Dojo writes `sweep_outputs.dir/config/sweep_manifest.json` as the immutable job index. The manifest records each concrete run's index, `run_id`, `config_hash`, realized sweep values, resolved `training_outputs.dir` / `ensemble_outputs.dir`, resolved config artifact paths, and per-run status path. `dojo sweep status` and `dojo sweep report` read the manifest, then read each run's `config/resolved.yaml`, `status.json`, and metric artifacts from those recorded locations.

`sweep_outputs.artifacts` is a report-time artifact promotion block. It does not create a separate "sweep model"; it copies or converts selected model artifacts from completed concrete jobs recorded in `sweep_manifest.json`, such as the best run's best checkpoint according to a configured collected metric.

Default on-disk sub-directories under the resolved `sweep_outputs.dir`:

```
sweep_outputs.dir/
  config/
    composed.yaml
    resolved.yaml
    resolved.json
    sweep_manifest.json
    cli.txt
    overrides.txt
  exports/
  metrics/
  figures/
```

Sweep ID

`runtime.sweep_id` may be manually set, rendered from a coolname source template such as `{coolname:sweep_hash}`, or fall back to `{coolname:sweep_hash}` when unset. See `06-results-artifacts-and-metadata.md` for the full ID / hash rules.

Result rows produced as part of a sweep carry both `sweep_id` and `sweep_hash` provenance columns, and may include `sweep_id` in their `partition_by` list.

Sweep output example

```yaml
runtime:
  sweep_id: "{coolname:sweep_hash}"

model:
  image_input:
    backbone:
      architecture:
        source: torchvision
        name: resnet50
      weights:
        source: library
        name: DEFAULT

training:
  batch_size: 32

optimizer:
  lr: 0.0003

sweep:
  mode: grid
  conflict_policy: default
  execution:
    mode: manual
  grid:
    model.image_input.backbone.architecture.name: [resnet50, efficientnet_b0, convnext_tiny]
    training.batch_size: [32, 64]
    optimizer.lr: [0.0003, 0.0001]
```

```yaml
output_root: ./runs

training_outputs:
  dir_template: >-
    {experiment.name}/sweep_runs/{model.image_input.backbone.architecture.name:slug}/bs{training.batch_size

sweep_outputs:
  dir_template: >-
    {experiment.name}/sweep_results/{runtime.sweep_id}
  enabled: true
  collect:
    - metric: val/species/f1_macro
      source: best
      mode: max
    - metric: val/species/f1_per_class/*
      source: best
      mode: max
  metrics:
    summary_csv: true
    summary_json: true
    summary_parquet: true
  figures:
    metric_rankings: true
    per_class_heatmap: true
```

Sweep report outputs include:

- compare best / final F1 across best / final epoch for model trainings;
- compare per-class F1 across best / final epoch;
- comparison metrics in CSV / JSON / Parquet;
- comparison figures.

Sweeps over ensembling

Sweeps can explore ensemble selection strategy / combine-mode combinations against a candidate manifest, via a `sweep:` block or `sweep.grid.*` overrides:

```
dojo sweep prepare experiment=ifcb/ensemble_search \
  sweep.mode=grid \
  'sweep.grid.ensemble.selection.strategy=[top_k,greedy_forward_selection]' \
  'sweep.grid.ensemble.inference.combine.classification=[probabilities_mean,logits_mean]'
```

Each prepared concrete ensemble job can then run through the lower-level command-specific replay path, for example `dojo ensemble --resolved-config JOB_DIR/config/resolved.yaml`. The `dojo sweep train` convenience wrapper is for training jobs. Sweep status and report commands still operate on the prepared sweep directory and summarize ensemble metrics across the swept axes.

Sweeps as candidate-source feeders

A training sweep may write to a shared candidate manifest directory that a subsequent `dojo ensemble` invocation reads:

```
dojo sweep prepare experiment=ifcb/sweep_for_ensembling
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --index 0
dojo sweep train ./runs/ifcb/sweep_results/SWEEP_ID --index 1
dojo sweep report ./runs/ifcb/sweep_results/SWEEP_ID
dojo ensemble candidates \
  experiment=ifcb/post_sweep_candidates \
  ensemble.candidates.sources.0.type=run_dir_glob \
```

```
  ensemble.candidates.sources.0.uri_glob=./runs/ifcb_sweep/*/ \
  ensemble_outputs.manifests.dir=./shared_manifests
dojo ensemble \
  experiment=ifcb/post_sweep_ensemble \
  ensemble.candidates.sources.0.type=manifest \
  ensemble.candidates.sources.0.manifest_uri=./shared_manifests/ifcb_post_sweep_candidates.json
```

This is the supported pattern for "use a sweep's outputs as ensemble candidates" — `dojo ensemble candidates` over an explicit run-directory glob, not a registry-driven discovery (deferred).

Cross-References

- `02-cli-and-task-types.md` — CLI architecture; `dojo sweep prepare`, `dojo sweep train`, `dojo sweep status`, and `dojo sweep report`.
- `03-configuration.md` — `output_root`, `*_outputs.dir_template` resolution, run / sweep directory collision handling.
- `06-results-artifacts-and-metadata.md` — `sweep_id`, `sweep_hash`, partition columns; `metrics/` and `figures/` sub-block semantics shared with `training_outputs` and `ensemble_outputs`.
- `08-ensembles.md` — `dojo ensemble candidates` for candidate manifests built from sweep outputs.
- `appendix-deferred-features.md` — Bayesian / AutoML HPO; registry-based candidate discovery.
- `glossary.md` — `sweep_id`, `sweep_hash` definitions.

10. Export

Purpose

Defines portable model export: artifact types, training / ensemble `export:` sub-blocks, sweep-level artifact promotion, the `dojo export` command, and export metadata.

Artifact types

Export artifact `type` values in config:

- `torchscript`
- `onnx`

There is no `pt` type. `.pt` is the **default filename extension** that TorchScript artifacts use; the `type` field names the artifact format, not its filename suffix.

State-dict-only export is not an initial-implementation artifact type. Pickled state dicts are training-internal artifacts produced by Lightning checkpointing (`.ckpt`); portable exports go through `torchscript` or `onnx`.

Configuration

Export is explicit. It is controlled by `training_outputs.export`, `ensemble_outputs.export`, `sweep_outputs.artifacts`, the `dojo export` command, or task-orchestration config. ONNX export is **not** a runtime training config item — it lives in the export config.

```yaml
training_outputs:
  export:
    enabled: true
    artifacts:
      - type: torchscript
        name: model.pt
        source: best_checkpoint
      - type: onnx
        name: model.onnx
        source: best_checkpoint
```

```
    opset: 18
    dynamic_axes: true
```

The same `export:` sub-block shape applies under `ensemble_outputs:`. Sweep-level report-time artifact promotion lives under `sweep_outputs.artifacts` and may reuse export artifact specs when it converts selected concrete-job checkpoints into portable models.

Meaning by output block:

- `training_outputs.export` exports model artifacts produced by the current training run, typically `source: best_checkpoint` or `source: last_checkpoint`.
- `ensemble_outputs.export` exports the selected ensemble described by the ensemble manifest.
- `sweep_outputs.artifacts` runs during `dojo sweep report`. It does not export a separate "sweep model"; it promotes or converts model artifacts from completed concrete jobs recorded in `sweep_manifest.json`, such as the best run's best checkpoint according to a configured collected metric.

`eval_outputs` has no `export:` sub-block in the initial implementation. `dojo infer` / `dojo eval` consume an existing model artifact and write results, metrics, figures, and `eval_manifest.json`; they do not create a new model artifact. Use `dojo export` to convert the evaluated checkpoint or model explicitly.

`dojo export`

Explicit conversion of checkpoints, ensemble manifests, or already-existing run output into portable artifacts.

`--checkpoint`, `--output`, `--type`, and `--ensemble-manifest` are command options, not config keys (see `02-cli-and-task-types.md`).

```
dojo export \
  --checkpoint s3://bucket/runs/run123/checkpoints/best.ckpt \
  --output s3://bucket/runs/run123/exports/model.pt \
  --type torchscript
```

Ensemble export:

```
dojo export \
  --ensemble-manifest s3://bucket/runs/run123/ensemble_manifests/manifest.json \
  --output s3://bucket/runs/run123/exports/snapshot_ensemble.pt \
  --type torchscript
```

`dojo export` does not decide which checkpoints belong in an ensemble. That decision belongs to `dojo ensemble`.

Export metadata

Exported artifacts include:

```
model architecture
backbone architecture source / name
backbone weights provenance
head definitions
ordinal encoding / decoding (per ordinal head)
objective_summary scorer spec
class names
class index mappings
normalization mean / std
image mode
resize policy
bucket definitions
tabular feature names / order
tabular categorical encoding specs and frozen vocabularies
tabular encoder config and resolved input / output dimensions
tabular normalization specs and resolved stats
```

tabular imputation (per-column strategy + frozen fill values)
tabular missing-indicator columns (when enabled)
model input names and implicit concatenation order
input shape
checkpoint weight URI
config_hash
model_config_hash
target_schema_hash
class_mapping_hash
preprocessing_hash
Dojo version

This metadata is the **portable inference contract** (`06-results-artifacts-and-metadata.md`) — the same schema embedded in training checkpoints under `checkpoint["dojo_inference_contract"]` — so `dojo infer` / `dojo eval` consume exports and checkpoints through one path.

If authored config used `weights.name: DEFAULT`, export metadata records the resolved concrete provider weight identity, not the `DEFAULT` alias.

For ONNX, metadata is also embedded into ONNX metadata properties when possible, with `metadata.json` written alongside `model.onnx`.

Aspect / size buckets and ONNX

For bucketed-resize models:

- CNN / ConvNeXt / ResNet exports often support dynamic H / W via `dynamic_axes: true`.
- ViT exports are typically more robust as **one ONNX file per bucket shape**. The `metadata.json` records bucket → ONNX file mapping:

```yaml
preprocessing:
  resize_policy: bucketed
  buckets:
    - name: square
      size: [224, 224]
      min_aspect: 0.75
      max_aspect: 1.33
      onnx_model: model_224x224.onnx
    - name: wide
      size: [224, 448]
      min_aspect: 1.33
      max_aspect: 3.0
      onnx_model: model_224x448.onnx
```

Exports directory

Exports always land under `<*_outputs.dir>/exports/`. See `06-results-artifacts-and-metadata.md` for the per-block artifact layout. Snapshot-ensemble runs share one `exports/` directory across the training and ensemble phases; per-artifact `name` values disambiguate.

Cross-References

- `02-cli-and-task-types.md` — `dojo export` command.
- `03-configuration.md` — export-block and sweep-artifact placement.
- `05-models-training-and-heads.md` — supervised model composition (export source).
- `06-results-artifacts-and-metadata.md` — `exports/` sub-directory, `model_id` / `model_hash`, compatibility-hash field selection.
- `08-ensembles.md` — exported ensemble artifacts.
- `11-dependencies.md` — `onnx` optional extra.

- `glossary.md` — `model_id`, `model_hash` definitions.

11. Dependencies

Purpose

Defines the lightweight base install plus optional extras layout. Distinguishes core dependencies (config / schema / storage / result reading) from extras gated by functional area.

Principles

- Base install supports config validation, inspection, storage / result access, schema handling, and non-checkpoint artifact / metadata introspection **without requiring Torch**. Inspecting Lightning `.ckpt` files requires the Torch stack.
- Training, SSL, ONNX, and IFCB support live behind extras. Logging sinks other than `local` (Aim, MLflow) are deferred and not registered sink types; metrics and figures are recorded locally for the foreseeable future. The `aim` extra is commented out (undeliverable on current Python) and no `mlflow` extra is currently declared.
- `amplify-db-utils` and `amplify-storage-utils` are core dependencies.

Base dependencies

The two `amplify-*` packages are base dependencies hosted on the WHOIGit GitHub organization and installed as direct git references (unpinned, tracking the default branch). `ifcbkit` is also installed from a WHOIGit direct git reference, but only through the optional `ifcb` extra. `allow-direct-references = true` is set in `pyproject.toml`.

```
dependencies = [
  "pydantic",
  "pydantic-settings",
  "hydra-core",
  "omegaconf",
  "pyarrow",
  "duckdb",
  "amplify-storage-utils @ git+https://github.com/WHOIGit/amplify-storage-utils.git",
  "amplify-db-utils @ git+https://github.com/WHOIGit/amplify-db-utils.git",
  "typer",
  "rich",
  "coolname",
  "humanize",
  "tqdm",
  "imagesize",
]
```

Optional extras

```
[project.optional-dependencies]
train = [
  "torch",
  "torchvision",
  "lightning",
  "torchmetrics[visual]",
  "numpy",
  "pandas",
  "pillow",
]

timm = ["timm"]
```

```
ssl = ["lightly"]

ifcb = ["ifcbkit @ git+https://github.com/WHOIGit/ifcbkit.git"]

repr_eval = [
  "umap-learn",
  "hdbscan",
  "scikit-learn",
]

# aim extra is defined but the sink is deferred (not a registered sink
# type) and it is currently undeliverable on Python 3.13/3.14 (aimrocks
# has no wheel), so it is commented out and excluded from `all`.
# aim = ["aim"]
onnx = ["onnx", "onnxruntime-gpu"]

all = [
  "image_classifier_dojo[train,timm,ssl,ifcb,repr_eval,onnx]",
]

dev = [
  "pytest",
  "pytest-cov",
  "ruff",
  "mypy",
  "pre-commit",
]
```

Notes on the current `pyproject.toml` state:

- `torchmetrics[visual]` pulls the extra needed for figure/visualization metrics.
- `imagesize` supports header-only image dimension inspection for `dojo inspect dataset --dimensions` without requiring Torch.
- `ifcb` installs `ifcbkit` from git (no `[s3]` extra, to avoid its conflicting pinned `amplify-storage-utils` reference).
- `onnx` uses `onnxruntime-gpu`.
- The `mlflow` extra and a standalone `s3` extra are **not currently declared**. MLflow remains a deferred logger sink (not a registered sink type; see `appendix-deferred-features.md`); its extra can be added when the sink is built. S3 capability rides along with the git `amplify-storage-utils` dependency rather than a separate extra.

Extra purpose summary

- `train` — Torch stack required to run training / inference and to inspect Lightning `.ckpt` checkpoint files.
- `timm` — first-class `model.image_input.backbone.architecture.source: timm` support. **Not deferred**; gated by this extra and raises a clear runtime error when missing.
- `ssl` — Lightly only. SSL DINOv2 ViT backbones come from the `timm` extra; install both for SSL.
- `ifcb` — `ifcbkit` (from git) for the `ifcb_bins` dataset backend. Usable outside SSL (supervised IFCB training, holdout eval, inspect). Installed without `ifcbkit`'s own `[s3]` extra to avoid its pinned `amplify-storage-utils` reference conflicting with the base git dependency; S3 access still comes through the base `amplify-storage-utils` install.
- `repr_eval` — UMAP, HDBSCAN, and scikit-learn for representation evaluation (projections, clustering, linear / ridge probes, baseline metrics). Usable against supervised encoders, not SSL-only.
- `aim` — Aim logger sink, deferred (**not a registered sink type** — see `appendix-deferred-features.md`). For the foreseeable future only the `local` sink is functional; metrics and figures are recorded locally. The `aim` extra is additionally undeliverable on current Python (`aimrocks` ships no 3.13/3.14 wheel) and is commented out in `pyproject.toml`.
- `mlflow` — MLflow logger sink, deferred (**not a registered sink type** — see `appendix-deferred-features.md`).

No extra is currently declared in `pyproject.toml`; add it when the sink is built.

- `onnx` — ONNX export and runtime (`onnxruntime-gpu`).
- *(no standalone `s3` extra)* — S3 storage capability comes through the base `amplify-storage-utils` git dependency rather than a separate extra.
- `all` — convenience meta-extra that pulls every functional extra above (`train,timm,ssl,ifcb,repr_eval,onnx`).
- `dev` — testing and developer tooling.

Common install recipes

```
# Schema / config inspection / result reading only
pip install image_classifier_dojo

# Supervised training
pip install image_classifier_dojo[train]

# Supervised + timm backbones
pip install image_classifier_dojo[train,timm]

# SSL DINOv2 with representation evaluation
pip install image_classifier_dojo[train,timm,ssl,repr_eval]

# IFCB bins (S3 access comes through the base amplify-storage-utils dep)
pip install image_classifier_dojo[train,ifcb]

# All functional extras
pip install image_classifier_dojo[all]

# Local development
pip install -e .[all,dev]
```

Dropped / removed dependencies

- `torchensemble` — dropped. Bagging / Boosting / Fusion / Adversarial / FastGeometric strategies are not ported. See `08-ensembles.md`.
- `pyifcb` — dropped in favor of `ifcbkit`.
- `h5py` / `tables` and the `hdf` extra — dropped. HDF-derived result exports are deferred; see `appendix-deferred-features.md`

Cross-References

- `01-goals-and-scope.md` — non-goals and deferred features.
- `04-data-and-storage.md` — `ifcb` extra and S3 through the base `amplify-storage-utils` dependency.
- `05-models-training-and-heads.md` — `train` and `timm` extras.
- `06-results-artifacts-and-metadata.md` — `aim`, `mlflow` extras.
- `07-ssl-and-representation-eval.md` — `ssl` and `repr_eval` extras.
- `10-export.md` — `onnx` extra.
- `appendix-deferred-features.md` — Aim runtime, MLflow runtime, HDF, WebDataset.

12. Validation, Testing, and Preflight

Purpose

Defines layered validation (Pydantic, `dojo inspect config`, dataset preflight, runtime validation), `runtime.preflight` controls, the strict-schema policy for deferred features (they are absent from the schema and fail generic validation), and the functional-feature testing scope.

Layered validation

1. **Pydantic static schema validation.** Catches structural and type errors at config load.
2. **`dojo inspect config`.** Composition, validation, rendered paths, output-tree preview, run / sweep directory collision warnings, enabled-output / deferred-feature reporting. Default: offline schema-and-local feasibility. Opt-in `--check-remote` flag for remote-availability HEAD / etag checks (see `02-cli-and-task-types.md`).
3. **`dojo inspect dataset`** / training preflight. Manifest and target checks, missing-target reporting, sample-drop summarization, and required frozen-stats cache availability. Default missing-target policy is `error` for all heads. Preflight consumes frozen stats; it does not compute or repair missing cache aspects.
4. **Runtime validation.** Checkpoints, models, schemas, tensor shapes, exports.

`runtime.preflight`

Preflight controls live under `runtime.preflight`. Dataset checks include:

- `empty_train_classes`
- `empty_eval_classes`
- `missing_required_targets`
- `no_remaining_valid_target_labels`
- `non_contiguous_class_indices`
- `imbalance_ratio_gt`

`imbalance_ratio_gt` means:

`max_class_count / min_nonzero_class_count > configured_threshold`

Defaults to a warning, not an error.

Missing-label preflight follows each target's `data.targets.<target>.missing_policy`:

- `error` — missing labels in active supervised / eval splits fail preflight.
- `drop_sample` — report the number of rows dropped for that target and fail if the active split becomes empty or an enabled objective / scorer has zero valid labels.
- `mask_objective` — report the number of rows masked for that target and fail if an enabled objective / scorer has zero valid labels in an active training or eval split.

```yaml
runtime:
  seed: 123
  precision: bf16-mixed
  num_workers: 8
  fast_dev_run: false
  autobatch:
    enabled: true
    mode: binsearch
  preflight:
    enabled: true
    checks:
      empty_train_classes: error
      empty_eval_classes: warn
      missing_required_targets: error
      no_remaining_valid_target_labels: error
      non_contiguous_class_indices: error
      imbalance_ratio_gt:
        severity: warn
        threshold: 20.0
```

Runtime controls

Supported `runtime` keys: `seed`, `fast_dev_run`, `precision`, `num_workers`, `autobatch`, `preflight`, `run_id`, `sweep_id`.

`autobatch` runs a pre-training batch-size search to size the batch to the current device/GPU before training proper begins — a thin wrapper over Lightning's `Tuner.scale_batch_size`. It is off unless `autobatch.enabled: true`; `mode` is `binsearch` (default) or `power`. The found batch size feeds `training.batch_size` for the run.

Early stopping is a training-loop behavior under `training:`, not `runtime:` (see `05-models-training-and-heads.md`).

Policy for deferred features

Deferred features are **absent from the schema**, not stubbed. The Pydantic config models are strict (`extra="forbid"`), and enum / discriminated-union fields list only implemented values, so configuring a deferred feature fails generic validation at config load — exactly as a typo or nonsense key would. There are no reserved-but-inert config slots and no runtime `NotImplementedError` stubs.

Deferred features therefore carry **no per-feature test obligation**. The whole class is covered by a small number of generic strict-schema tests:

1. an unknown key anywhere in the tree is rejected (`extra="forbid"`);
2. an out-of-enum value / unknown discriminated-union tag is rejected, and the error lists the implemented values.

The generic validation error names the offending key or value only; it does **not** advertise the feature as planned. The deferred roadmap lives in `appendix-deferred-features.md` and `13-workplan.md`, never in the running schema. When a deferred feature is built, its value is added to the schema and gains real functional tests; nothing feature-specific needs to be deleted first.

The one validation message that stays curated is for **invalid combinations of implemented features** (incompatible head / loss pairs, bad objective references, invalid dataset columns); those are real current contract and keep their specific messages.

Two deferred entries are not config tokens and so are handled in kind: deprecated-package removal (P4.8) is a cleanup milestone with a no-imports verification obligation, and the `majority_vote` probability-mass tie-break (P4.12) is an enhancement to functional behavior. Neither is a stub.

See `appendix-deferred-features.md` for the full deferred backlog.

Functional features get full functional tests

When the relevant extra is installed:

- `model.image_input.backbone.architecture.source: timm` (functional);
- `model.image_input.backbone.architecture.source: torchvision`;
- `model.image_input.backbone.weights.source: checkpoint`;
- `local` logger sink (the only functional sink);
- `dino_v2` SSL via Lightly;
- `ifcb_bins` dataset backend (with `[ifcb]`);
- UMAP, t-SNE, HDBSCAN, regression / ordinal / classification probes (with `[repr_eval]`);
- ONNX export (with `[onnx]`);
- S3 storage (via the base `amplify-storage-utils` dependency);
- snapshot ensembles and prediction-space ensembles (selection strategies `all`, `best_candidate`, `top_k`, `greedy_forward_selection`; combine modes per `08-ensembles.md`; snapshot ensembles select over the run's implicit cycle-snapshot source).

The `local` sink is the only functional logging sink, so functional logging tests exercise `local` alone. Multi-sink composition is supported structurally (`CompositeExperimentLogger`), but `aim` and `mlflow` are not registered sink types — a config naming either fails schema validation at load.

Test scope by area

- **Config tests** — Hydra composition; Pydantic validation success and failure; CLI override validation; invalid head / loss combinations; invalid objective references; invalid dataset columns; invalid logger sink configs; invalid ensemble configs; `dojo init` materialization, dependency-closure copying, fixture-data copying, and non-clobber / clobber behavior.

- **Dataset tests** — CSV / Parquet / `parquet_images` / `ifcb_bins` backends; multi-head targets; target `missing_policy` reporting and preflight behavior; tabular feature extraction; categorical vocabulary fitting and frozen one-hot lookup; `sample_id` / `uri` / `bin_id` / `bin_uri` propagation; mocked storage resolver for S3-style paths.
- **Storage tests** — local amplify-backed storage; cache resolver behavior; S3 optional-import behavior; `open_bytes` / `localize` / `write_bytes` / `exists`.
- **Transform tests** — letterbox, aspect / size buckets, foreground-aware crop, grayscale repeat-to-3, `normalize`, `rotate`, flips, DINOv2 multi-view transform, extreme aspect ratios, small native resolutions.
- **Model tests** — torchvision, timm (with `[timm]`), checkpoint inspection gated by `[train]`, and checkpoint backbones; freeze policies; head construction; multi-head model composition; embedding adapter; numeric and one-hot categorical tabular input concatenation.
- **Objective / loss tests** — loss / metric compatibility per head type; metric registry canonical names / params / output names; objective shorthand; `mask_objective` valid-label masking; weighted total loss; resolved `objective_summary` serialization.
- **Supervised training smoke tests** — minimal-fixture train + validate + canonical result writing.
- **SSL training smoke tests** — DINOv2 functional smoke test.
- **Representation evaluation tests** — embeddings, projections, clustering, probes (with `[repr_eval]`).
- **Ensemble tests** — candidate discovery; manifest writing; `dojo ensemble candidates`; supported selection strategies and combine modes; cached-result and live-inference paths.
- **Export tests** — TorchScript and ONNX exports; metadata embedding; holdout-eval scorer reconstruction from the embedded inference contract.
- **Logging tests** — `local` functional; configs naming `aim` or `mlflow` sinks rejected at schema validation.

Test fixtures

Three tiers:

- **Tier 1 — committed fixtures (`tests/fixtures/`).** Minimal, deterministic data used by unit and integration tests. Parquet manifests with **image bytes inlined** (`parquet_images` mode). Tracked via git LFS. Fixture parquet content is added separately, by the developer; this design doc only specifies the format and pipeline.
- **Tier 2 — local development fixture.** Not committed. Lives at `/home/sbatchelder/Projects/ifcbNN/datasets/minis` or similar; pointed at via env var (e.g. `DOJO_DEV_DATASET_ROOT`).
- **Tier 3 — real example dataset.** Not committed; fetched at runtime. Hugging Face dataset `sbatchelder/NES-plankton-classifier-2022-dataset`.

Integration tests consume Tier 1 fixtures exclusively.

Cross-References

- `02-cli-and-task-types.md` — `dojo inspect config`, `dojo inspect dataset`, remote-validation flag.
- `03-configuration.md` — `runtime:` block placement.
- `04-data-and-storage.md` — dataset preflight checks.
- `06-results-artifacts-and-metadata.md` — logging-sink behavior.
- `appendix-deferred-features.md` — deferred-feature backlog (absent from the schema) and the P4.8 cleanup milestone.
- `11-dependencies.md` — extras gate which functional tests run.
- `glossary.md` — preflight / runtime vocabulary.

13. Workplan

Purpose

Describes the order of work for the new `src/dojo` implementation. This is a clean-break refactor rather than a line-by-line port from `src/dojo_deprecated/`; the deprecated package is reference material, not the implementation plan.

The work is ordered to prove shared contracts early, then layer broader capabilities on top once config, storage, results, and identity hashing are stable.

Priority 1 — Minimal viable end-to-end thin slice

Goal: one supervised single-head training run, configured through Hydra / Pydantic, reading a Parquet train / val dataset, writing canonical tall-Parquet results with a `_metadata.json` sidecar.

This slice is deliberately the thinnest path that touches every architectural boundary exactly once. It proves the contracts that everything else depends on, and surfaces integration problems with `amplify-db-utils` / `amplify-storage-utils` while the blast radius is still small.

Layer	Minimal inclusion
Config	Hydra composition + Pydantic root schema for a minimal supervised run (`experiment`, `task.type: supervised`, `runtime`, `storage`, `data`, `model`, `objectives`, `training`, `output_root`, `training_outputs`)
CLI	`dojo train` and `dojo inspect config` only
Data	`parquet_manifest` backend + shared sample contract (`sample_id`, `uri`, `split`, one target); `parquet_images` fixture to avoid path-resolution combinatorics
Model	`torchvision` backbone + single `multiclass_classification` head + one `cross_entropy` objective; no tabular, adapter, or multi-head path
Training	Supervised LightningModule, AdamW, best-k checkpointing, `local` logger sink only
Storage	`amplify-storage-utils` resolver behind the Dojo storage interface; `output_root` / `dir_template` resolution
Results	Canonical tall-Parquet writer via `amplify-db-utils`: `sample_metadata` + `classification_output` record types, provenance columns, `config_hash` / `dataset_hash` / `checkpoint_hash`, `_metadata.json` sidecar

Explicitly excluded from the slice: SSL, ensembling, sweeps, export, multi-head, tabular input, representation evaluation, `ifcb_bins`, `timm`, ONNX, and all non-local logging.

Acceptance criteria:

- `dojo inspect config` composes, validates, renders output paths, and reports enabled outputs.
- `dojo train` runs on the `parquet_images` fixture and produces a resolved run directory with `config/`, `checkpoints/`, `results/`, and `metrics/`.
- Result Parquet round-trips through `amplify-db-utils`, and a reader can filter by `stage` / `record_type`; the `_metadata.json` sidecar validates against the schema.
- Hashes are deterministic and reproducible across two runs of the same config.
- One integration test (`test_train_supervised.py`) plus unit tests for the config schema, result writer, and hashing canonicalizer are green in CI.

Why this gates the rest: if the `amplify-db-utils` result contract, the storage interface, or the hash / identity scheme need rework, it is far cheaper to find out here than after heads, objectives, ensembling, and sweeps are layered on top.

Priority 2 — Core supervised platform

P2.1 Config / CLI / storage foundation

- Full config tree and Pydantic contract.
- `dojo init` for materializing packaged configs and optional fixture data into a local editable project.
- `dojo inspect dataset`, folding in old manifest-making behavior and acting as the cheapest gate against bad data.
- `dojo inspect backbone` and `dojo inspect checkpoint`.
- Composite logger abstraction: `local` functional; Aim and MLflow deferred (not registered sink types).

P2.2 Dataset backends

- `csv_manifest`, `parquet_manifest`, and `parquet_images`.
- Shared record contract.
- Preflight checks: `empty_train_classes`, `non_contiguous_class_indices`, `imbalance_ratio_gt`.
- `dojo inspect dataset` aspect flags (tiered manifest / header / decode passes, terminal chart rendering) and the `dataset_hash`-keyed stats cache producing the frozen normalization / tabular / target / class-count / bit-depth / dimension / bin-length values consumed at config resolution, plus the separate `dataset_content_hash` on full passes.
- Class-name derivation from dataset-level feature metadata, as a third source behind the P1 `label_index_column` / `label_name_column` (`04-data-and-storage.md`). When neither target column carries readable names, read the index → label mapping from the dataset's own schema metadata — e.g. a Hugging Face `ClassLabel` feature (`features["label"].int2str(...)`), or equivalent sidecar / Arrow field metadata — and fold the resolved mapping into the frozen class mapping in the stats cache. Not all inputs expose this (plain Parquet, CSV manifests), so it stays an optional enrichment, never required.

P2.3 Model composition

- Backbone registry: `torchvision` architecture functional, `timm` architecture gated by the `timm` extra; `weights.source` `none` / `library` / `checkpoint` initialization all functional.
- Head registry with target validation.
- Objectives binding heads to losses, metrics, and weights.
- Multi-head / multi-objective normalization.
- Embedding adapter.
- Supervised transfer learning from a checkpoint.

P2.4 Transforms

- Transform builder.
- Letterbox, aspect buckets, foreground crop, grayscale, `normalize`.
- Per-step `train_only` flag and the derived, resolved-only `inference_pipeline` consumed by non-train stages, export, and `preprocessing_hash`.
- `aspect_bucket` assignment as a working-manifest `aspect_bucket` column (derived from cached dimensions × scheme) and the `batch_aspect_buckets` bucket-aware sampler yielding size-homogeneous batches.
- Sampler factory: `class_balanced` and `weighted` samplers reading frozen class counts, composing with `batch_aspect_buckets` (bucket grouping outer, class weighting within bucket).
- Record scale metadata columns.

P2.5 Results hardening

- Full record-type taxonomy.
- Ground-truth columns on `classification_output` rows (the sample's true `target_index` / `target_name` alongside the `prediction_*` columns), so a row is self-scoring without joining back to `sample_metadata` / the source manifest. This is the ground-truth (`y_true`) half of the `target` vs `prediction` pairing; P1 writes predictions only and drops the old target identifier column (rows are scoped by `head_name`, and the head → target link lives in `_metadata.json`). Settle the readable-suffix asymmetry then (`prediction_label` vs `target_name`).

- Per-head `head_hash` for join-free cross-run querying: a content hash over a head's resolved config (target, `num_classes`, `class_mapping`, network), finer-grained than the whole-schema `target_schema_hash`. Lets rows from the same head configuration be grouped across runs without a denormalized name column.
- Partitioning.
- External / internal column convention.
- Clean per-epoch metrics CSV: one merged row per epoch instead of separate train and validation rows at the same step (a Lightning `CSVLogger` quirk — the `on_train_epoch_end` / `on_validation_epoch_end` hooks flush as separate `log_metrics` calls). P1 logs epoch-only (no per-batch step rows) but leaves the train/val row split as-is.
- Compatibility-hash extractors implementing the pinned `target_schema_hash`, `class_mapping_hash`, `model_config_hash`, and `preprocessing_hash` source field lists from `06-results-artifacts-and-metadata.md`.

P2.6 Inference and holdout evaluation

- `dojo infer predictions` and `dojo infer embeddings`.
- `dojo eval holdout`.
- `eval_outputs` directory handling, canonical result writing, and the always-written `eval_manifest.json`.
- Checkpoint / export loading through the embedded portable inference contract, without depending on the producing run's `resolved.yaml`.
- Holdout scorer reconstruction from `objective_summary` and `target_schema`.
- Result rows with `stage=infer` / `stage=holdout_eval`, including canonical provenance, embedding, and prediction columns.

Priority 3 — Major capability layers

P3.1 Export

- TorchScript and ONNX.
- Export metadata.
- Bucket-aware ONNX.

Needed for downstream serving and for ensembling exported models.

P3.2 Prediction-space ensembling

- Candidate discovery with explicit sources only.
- Compatibility checking.
- Selection strategies.
- Combine modes.
- Cached-result path.
- `task.type: snapshot_ensemble`.

P3.3 Sweeps

- Dojo-owned sweep expansion over the Hydra Compose API (no `@hydra.main`, no `-m`).
- Top-level `sweep:` block: grid sweeps and batch-run-style seed sweeps.
- `dojo sweep prepare`, manual per-job training, `dojo sweep status`, and `dojo sweep report`.
- `sweep_outputs` reporting.

P3.4 SSL and representation evaluation

- DINOv2 via Lightly.
- `representation_eval`: probes, projections, clustering, diagnostics.
- Standalone and training-integrated execution.

P3.5 IFCB bins

- `ifcbkit` integration for the `ifcb_bins` backend across train / eval / infer paths, including SSL where applicable.
- Custom dataset and dataloaders for IFCB bins.

P3.6 Tabular input

- `model.tabular_input`: selected logical feature columns, input-stream name (`name`, default `tabular`), and tabular encoder. Excluded from the P1 slice and P2.3 model composition; layered in here.
- `model.image_input.name` names the image input stream and defaults to `image`; `model.image_input.backbone.architectu` remains the architecture selector.
- When image and tabular inputs are both enabled, the supervised compositor concatenates embeddings implicitly in canonical order: image first, tabular second. Learned post-concat capacity belongs in `embedding_adapter`.
- `transforms.tabular`: numeric normalization specs / statistics, missing-value imputation (per-column strategy, frozen train-split fill values, optional numeric missing indicators), one-hot categorical encoding with frozen train-split vocabularies and explicit unknown / missing tokens, and train-only augmentations such as `random_missing`.
- Resolved tabular preprocessing state persisted in the config artifact, exported with portable models (`10-export.md`), and exercised by the `preprocessing_hash` / `model_config_hash` extractors from P2.5 (`model.tabular_input.enabled: false` leaves the tabular-input sub-block disabled).

Priority 4 — Deferred backlog

These features are lower priority. Deferred features are **absent from the strict schema** until promoted into active work, so configuring one fails generic validation — there are no `NotImplementedError` stubs or reserved slots. Cleanup and enhancement items name their verification obligation instead. See `appendix-deferred-features.md`.

- P4.1 Bayesian sweeps.
- P4.2 Multilabel support: one classifier head can emit several outputs from the list of possible outputs.
- P4.3 Aim runtime logging.
- P4.4 MLflow runtime logging.
- P4.5 Non-DINOv2 SSL: SimCLR, VICReg, PMSN, original DINO.
- P4.6 Weight-space ensembles (P4.6a), weighted combine modes (P4.6b), and `prediction_trimmed_mean` (P4.6c).
- P4.7 HDF / HDF5 result exports.
- P4.8 Deprecated package `src/dojo_deprecated/` removal.
- P4.9 Distributional and count regression heads (`distributional_regression`, `count_regression`).
- P4.10 WebDataset.
- P4.11 Registry-based ensemble candidate discovery.
- P4.12 `majority_vote` probability-mass tie-break: when members carry `probabilities`, break vote ties by highest summed member probability across the tied classes (falling back to lowest class index). An enhancement to the functional lowest-index tie-break, not a deferred config token, so it carries no obligation and has no `appendix-deferred-features.md` entry.
- P4.13 Tabular-only model schema.
- P4.14 Expanded tabular encoder families beyond `identity`, `linear`, and `mlp` (`tab_transformer`, `ft_transformer`, `tabnet`, `embedding_bag`, `wide_and_deep`).
- P4.15 `dojo init --wizard` interactive config/project questionnaire.
- P4.16 Automated sweep execution runners: local sequential execution and Slurm / HPC queue submission. Initial execution mode is `manual` via `sweep.execution.mode`.

Historical Notes

`src/dojo_deprecated/` has already been moved out of the new package path. Treat it as reference material for behavior and edge cases, not as the structure to port directly.

Likely useful reference areas:

- `dojo_deprecated/multiclass/` for supervised training cues, per-class counting, and callback behavior.
- `dojo_deprecated/multilabel/` as a historical example of multi-head multiclass behavior; do not port it as multilabel support.
- `dojo_deprecated/selfsupervised/` for SSL implementation cues.
- `dojo_deprecated/tools/dataset_lists_from_folder.py` for manifest inspection behavior that now belongs under `dojo inspect dataset`.
- `dojo_deprecated/schemas/core.py` for old config-field intent, while the new Pydantic schemas remain the contract.

- `dojo_deprecated/multiclass/callbacks.py` for checkpointing and training-loop behavior worth preserving.

Deprecated code should not receive new features. Once the new `src/dojo` implementation covers everything through P4.7 and nothing (code or test) imports from `src/dojo_deprecated/`, the deprecated package can be deleted (the P4.8 cleanup milestone).

Cross-References

- `01-goals-and-scope.md` — scope and deferred backlog.
- `02-cli-and-task-types.md` — CLI surface.
- `03-configuration.md` — top-level config tree.
- `06-results-artifacts-and-metadata.md` — result schemas, sidecar `_metadata.json`, hashes, and artifact layout.
- `07-ssl-and-representation-eval.md` — SSL and representation evaluation.
- `08-ensembles.md` — prediction-space ensembling.
- `09-sweeps-and-batch-runs.md` — grid sweeps, batch-run-style seed sweeps, and sweep outputs.
- `11-dependencies.md` — base install and extras.
- `12-validation-testing-and-preflight.md` — validation and strict-schema deferral policy.
- `appendix-deferred-features.md` — deferred-feature backlog (absent from the strict schema) and the P4.8 cleanup milestone.

Appendix — Deferred Features

Purpose

Lists features that are intentionally out of scope for the initial implementation but preserved as backlog items. This is a **roadmap, not a contract**: nothing here is reflected in the running schema or code until it is built. Per `12-validation-testing-and-preflight.md`, deferred features are simply **absent from the strict (`extra="forbid"`) schema**, so configuring one fails generic validation at load — there are no reserved config slots, no runtime `NotImplementedError` stubs, and no per-feature tests. Because nothing in code references this list, keep it in sync with reality by hand. Each entry records its future config-token shape (so the destination is documented) and the obligations that *do* exist when it is promoted; the P4.8 cleanup milestone names a verification obligation instead.

How deferral works

A deferred feature's config token (enum value, discriminated-union tag, or block) is **not added** to the Pydantic schema. Because the schema is strict (`extra="forbid"`) and enums list only implemented values, authoring the token fails generic validation at config load — the same failure a typo produces. The error names the offending key/value only; it does not mention this backlog.

Deferred features therefore carry **no per-feature tests**. The class is covered generically (one test that unknown keys are rejected, one that out-of-enum values are rejected). When a feature is promoted, its token is added to the schema and gains real functional tests; there is no stub test to delete first.

Deferred features

P4.1 Bayesian sweeps

- Initial sweeps are config-defined grid sweeps with Dojo-owned expansion (grid / explicit value lists). See `09-sweeps-and-batch-runs.md`.
- Future config shape: a `bayesian` value for `sweep.mode` plus a `sweep.bayesian` block (engine, metric, sampler, trial budget, search params). Neither is in the schema initially, so `sweep.mode: bayesian` fails as an out-of-enum value and a `sweep.bayesian` block fails as an unknown key.
- When promoted: add the `bayesian` mode and `sweep.bayesian` schema, then real functional tests for expansion and trial orchestration.

P4.2 Multilabel support

- Future config shape: `multilabel_classification` as a `model.heads.<name>.type` value.
- Meaning: one classifier head emits several binary labels from a single target/output vector. This is different from multi-head multiclass, where each target has its own `multiclass_classification` head.
- Current status: `multilabel_classification` is **not** a member of the head-type union, so authoring it fails as an unknown discriminated-union tag. Multi-head multiclass is functional and does not depend on it.
- When promoted: add the head type, its result record type, and real functional training / inference tests.

P4.3 Aim logger sink

- Future config shape: `aim` as a `training_outputs.logging.sinks[].type` value. `aim` is **not** a registered sink type, so authoring it fails as an unknown discriminated-union tag.
- Rationale: for the foreseeable future the project records metrics and figures **locally only** (the `local` sink). Aim was previously planned as functional but is deferred; the `aim` dependency also has no installable build on current Python (`aimrocks` ships no 3.13/3.14 wheel), which reinforces deferring it.
- Extra: `aim` (commented out in `pyproject.toml` while undeliverable on current Python).
- When promoted: register the sink type and add functional logging tests.

P4.4 MLflow logger sink

- Future config shape: `mlflow` as a `training_outputs.logging.sinks[].type` value. `mlflow` is **not** a registered sink type, so authoring it fails as an unknown discriminated-union tag.
- Extra: `mlflow` — **not currently declared** in `pyproject.toml`; add the extra when the sink is built.
- When promoted: register the sink type and add functional logging tests.

P4.5 Non-DINOv2 SSL methods

- Future config shape: `simclr`, `vicreg`, `pmsn`, `dino` as `ssl.method` values (each with its own method config). None are schema members, so authoring any of them fails as an out-of-enum value.
- The functional method is `dino_v2` via Lightly.
- When promoted: add the method and its config, then functional SSL tests.

P4.6a Weight-space ensembles

- Model soup, greedy soup, uniform soup, SWA, EMA.
- These produce a single exported model artifact (not a prediction-space combine), so they need new schema (a different ensemble paradigm) that is not present initially; authoring it fails generic validation.
- When promoted: design the schema and add functional tests per strategy.

P4.6b Weighted ensemble combine modes

- Future combine values: weighted_logits_mean, weighted_probabilities_mean, weighted_vote, weighted_prediction_mean, weighted_ordinal_logits_mean, weighted_ordinal_probabilities_mean (plus the per-member weight plumbing they need). None are schema members, so authoring any fails as an out-of-enum value.
- When promoted: add the modes and the weight source, then functional tests.

P4.6c `prediction_trimmed_mean`

- Sort member predictions, drop configured low / high extremes, average the rest.
- Future config shape: `prediction_trimmed_mean` as a regression combine value. Not a schema member, so authoring it fails as an out-of-enum value.
- When promoted: add the mode (and its trim fraction) and a functional test.

P4.7 HDF / HDF5 derived result exports

- `.h5` metrics rollups, `results.h5`. Would re-introduce an `hdf` extra and `h5py` / `tables` dependencies (currently dropped).

- Future config shape: an `hdf` export format value. Not a schema member, so authoring it fails as an out-of-enum value.
- When promoted: re-add the extra, the format, and functional export tests.

P4.8 Deprecated package removal

- Cleanup milestone, not a deferred feature.
- Remove `src/dojo_deprecated/` once the new `src/dojo` implementation covers everything through P4.7 and nothing imports from it, per `13-workplan.md`.
- Verification obligation: no imports from `src/dojo_deprecated/`, no tests depending on it, and any remaining useful behavior has either been reimplemented in `src/dojo` or intentionally dropped.

P4.9 Distributional and count regression heads

- Future config shape: `distributional_regression` (emits distribution parameters, e.g. `gaussian`, `negative_binomial`) and `count_regression` (emits count / rate parameters) as `model.heads.<name>.type` values.
- Deferred together because neither has a result record type in the initial implementation: `06-results-artifacts-and-metada` defines `regression_output` (single value + uncertainty) but no `distributional_output` / `count_output` for multi-parameter distribution or count / rate heads. Their dedicated losses (`gaussian_nll`, `negative_binomial_nll`, `poisson_nll`) are deferred with them; plain `regression` (`mse`, `mae`, `huber`, `smooth_l1`, `quantile`) is the only functional regression head type.
- Neither is a member of the head-type union, so authoring either fails as an unknown discriminated-union tag. When a head is promoted, add its result record type, losses, and functional inference tests.

P4.10 WebDataset backend

- Future config shape: `webdataset` as a `data.backend` value. Not a schema member, so authoring it fails as an out-of-enum value.
- When promoted: add the backend and its datamodule, plus functional dataset tests.

P4.11 Registry-based ensemble candidate discovery

- Broad automatic registry-based cross-run candidate discovery.
- Note: cross-run ensembling itself is **functional** via explicit candidate sources (`explicit`, `run_dir_glob`, `checkpoint_glob`, `result_uri_glob`, `manifest`). Only the registry-driven discovery is deferred. The term "cross-run ensemble" is no longer used as a distinct mode (see `08-ensembles.md`).
- Future config shape: a `registry` candidate-source type. Not a schema member, so authoring it fails as an unknown discriminated-union tag.
- When promoted: add the source type and functional discovery tests.

P4.12 `majority_vote` probability-mass tie-break

Intentionally has no entry here: it is an enhancement to the functional lowest-index tie-break, not a deferred config token, so it carries no obligation. Defined in `13-workplan.md` (P4.12).

P4.13 Tabular-only model schema

- Initial tabular support is image-backed supervised modeling with optional tabular features: an image backbone remains required, and tabular features may be concatenated implicitly after image embeddings.
- Tabular-only modeling is deferred because it requires a broader schema change: explicit model input enablement, image-free data validation, image-free preprocessing / result metadata, export metadata without image input shape, and updated preflight rules.
- Because `model.image_input.backbone` is required initially, configuring a tabular-only model fails generic validation (a required field is missing). When promoted, relax the requirement and add functional schema / runtime tests.

P4.14 Expanded tabular encoder families

- Initial `model.tabular_input.encoder.type` values are intentionally small: `identity`, `linear`, and `mlp`.
- Potential future encoder families: `tab_transformer`, `ft_transformer`, `tabnet`, `embedding_bag`, and `wide_and_deep`.
- These are deferred because each adds nontrivial schema, dependency, preprocessing, export, and compatibility-hash surface area.
- None are members of the encoder-type enum, so authoring any fails as an out-of-enum value. When an encoder family is promoted, add functional tests for its schema, construction, hashing, export metadata, and inference behavior.

P4.15 `dojo init --wizard`

- Interactive project/config questionnaire that writes a local config tree and optionally fixture data.
- Deferred because the initial `dojo init` should stay deterministic and template-based: copy packaged configs, materialize dependency closures, and optionally materialize a small fixture dataset.
- Not a config token but a CLI flag: the `--wizard` option simply does not exist initially, so passing it is a standard unknown-option error. When promoted, add CLI interaction tests for questionnaire branching, generated files, and non-clobber behavior.

P4.16 Automated sweep execution runners

- Initial sweep execution is manual: `dojo sweep prepare` expands the sweep and writes per-run resolved configs; users run concrete jobs explicitly, for example with `dojo sweep train SWEEP_DIR --index N` or the lower-level `dojo train --resolved-config ...` path.
- Initial `sweep.execution.mode` value: `manual` (the only schema member).
- Deferred execution modes (not yet schema members, so authoring either fails as an out-of-enum value):
  - `local_sequential` — run prepared sweep jobs sequentially on the local machine.
  - `slurm` — submit prepared sweep jobs to a Slurm / HPC queue.
- Deferred because runner orchestration adds scheduling, retry, log collection, cluster-specific configuration, and concurrency behavior beyond the initial sweep artifact contract. Initial `dojo sweep status` reads live per-run status files and does not persist a sweep-level status cache.
- When promoted: add integration tests for local sequential execution and unit / contract tests for Slurm script generation, submission dry-runs, manifest status updates, and failure recovery behavior.

Cross-References

- `01-goals-and-scope.md` — scope and non-goals.
- `08-ensembles.md` — explicit candidate sources for the not-deferred parts of cross-run ensembling; deferred selection / combine modes.
- `09-sweeps-and-batch-runs.md` — config-defined grid sweeps vs. Bayesian HPO.
- `11-dependencies.md` — extras list and dropped dependencies.
- `12-validation-testing-and-preflight.md` — strict-schema deferral policy in full.
- `glossary.md` — terminology.

Appendix — Proposed Repository Structure

Purpose

Concrete proposed target directory layout. The tree is derived from the canonical decisions in `03-configuration.md`, `04-data-and-storage.md`, `05-models-training-and-heads.md`, `06-results-artifacts-and-metadata.md`, `07-ssl-and-representation-eval.md`, `08-ensembles.md`, `09-sweeps-and-batch-runs.md`, `10-export.md`, `11-dependencies.md`, `12-validation-testing-and-preflight.md`, and `13-workplan.md`. It is populated according to the priority order in the workplan; modules shown here may be added incrementally and remain unimplemented until their priority is reached.

This is intent, not contract. Implementers may collapse, split, or rename leaf modules to fit emergent code shape; the *areas of responsibility* shown here are what must be preserved.

Top-level layout

```
image-classifier-dojo/
  pyproject.toml
  README.md
  Dockerfile
  LICENSE

  configs/
  src/dojo/
  src/dojo_deprecated/       # reference during refactor; removal governed by 13-workplan.md. No new code to
  tests/
  DESIGN-DOC/
```

configs/

The repository-level `configs/` tree is the editable development copy of the packaged config resources. The installable package also carries a read-only copy under `src/dojo/configs/`; `dojo init` materializes selected packaged configs into a user's local `./configs` tree.

```
configs/
  config.yaml                # top-level Hydra entrypoint

  experiment/                # Hydra experiment group; selected via experiment=<name>
    ifcb/
      baseline_resnet50.yaml
      experimentA.yaml
      dino_v2_ssl.yaml
      convnext_snapshot.yaml
      transfer_from_ssl.yaml
      ensemble_from_runs.yaml
      ensemble_from_cached_results.yaml
      representation_eval_standalone.yaml
    sweeps/                  # experiment configs that define a sweep: block
      grid_lr_wd.yaml
      batch_seed_5.yaml
      ensemble_axis_sweep.yaml

  data/
    csv_local.yaml
    csv_s3.yaml
    parquet_manifest.yaml
    parquet_images.yaml
    ifcb_bins.yaml

  transforms/
    supervised_default.yaml
    letterbox_square.yaml
    aspect_bucket.yaml
    ssl_dino_v2_plankton.yaml

  backbone/
    torchvision/
      resnet50.yaml
```

```
    efficientnet_b0.yaml
    inception_v3.yaml
    vit_b_16.yaml
  timm/                    # functional; gated by [timm] extra
    convnext_tiny.yaml
    vit_small_patch16_224.yaml
    efficientnet_b0.yaml
  checkpoint/
    from_local_checkpoint.yaml
    from_s3_checkpoint.yaml

heads/
  single_classification.yaml
  multihead_species_quality.yaml
  classification_regression_ordinal.yaml

ssl/
  dino_v2.yaml

representation_eval/
  knn.yaml
  linear_probe.yaml
  diagnostics_default.yaml
  standalone_default.yaml

optimizer/
  adamw.yaml
  sgd.yaml

scheduler/
  cosine.yaml
  cosine_warm_restarts.yaml  # snapshot-cycle scheduler
  step.yaml
  none.yaml

loss/
  cross_entropy.yaml
  weighted_cross_entropy.yaml
  class_balanced_effective_number.yaml
  focal.yaml
  label_smoothing.yaml
  regression_huber.yaml
  ordinal_coral.yaml
  ordinal_corn.yaml

runtime/
  default.yaml
  fast_dev_run.yaml
  preflight_strict.yaml

storage/
  local_only.yaml
  s3.yaml

sweep/
  disabled.yaml
```

```
    grid.yaml
    batch_seed_5.yaml              # grid sweep over runtime.seed only

  training_outputs/
    default.yaml
    sweep_aware.yaml

  ensemble_outputs/
    default.yaml
    snapshot_shared_with_training.yaml

  sweep_outputs/
    default.yaml

  eval_outputs/
    default.yaml               # standalone eval/infer run dir
    colocate_with_source.yaml  # dir_template uses {source_run_dir}

  ensemble/
    disabled.yaml
    snapshot_select_all.yaml   # snapshot_ensemble: strategy all over implicit cycle snapshots
    top_k.yaml
    greedy_forward_selection.yaml
    cached_results_only.yaml   # source_policy: strict_no_inference

  export/
    torchscript_model.yaml
    torchscript_ensemble_model.yaml
    onnx_model.yaml

  logging/
    local.yaml                 # only registered sink
```

No `task/` config group — `task.type` is set in the experiment config. No `configs/hydra/` group — Dojo uses the Hydra Compose API, not `@hydra.main`, so there is no Hydra run / sweep / `chdir` config to set.

src/dojo/

```
src/dojo/
  __init__.py

  cli/
    __init__.py
    main.py                    # Typer app entrypoint; composes config, dispatches
    init.py                    # dojo init: materialize configs + optional fixture data
    train.py                   # all task.type values: supervised, ssl, snapshot_ensemble
    eval.py                    # dojo eval, dojo eval holdout, dojo eval representation
    infer.py                   # dojo infer, dojo infer predictions, dojo infer embeddings
    ensemble.py                # dojo ensemble, dojo ensemble candidates
    sweep.py                   # dojo sweep prepare|train|status|report
    export.py                  # dojo export
    inspect.py                 # dojo inspect config|dataset|backbone|checkpoint

  configs/                     # packaged read-only config resources; mirrors the repository configs/ tree a
    config.yaml
    experiment/
```

```
  data/
  transforms/
  backbone/
  ...                       # same groups as the repository configs/ tree (no training/ group; training p

example_data/               # tiny packaged fixture dataset for dojo init --data

config_schemas/             # Pydantic; single source of truth
  __init__.py
  root.py
  experiment.py
  task.py                   # task.type union
  runtime.py                # runtime + preflight
  storage.py
  data.py
  transforms.py
  model.py                  # image_input, tabular_input, embedding_adapter, heads
  backbones.py
  heads.py
  objectives.py
  losses.py
  optimizers.py
  schedulers.py
  checkpointing.py
  ssl.py
  representation_eval.py
  ensemble.py
  sweep.py
  outputs.py                # training_outputs, ensemble_outputs, sweep_outputs, eval_outputs
  logging.py
  export.py
  validation.py             # cross-cutting validators
  hashing.py                # canonicalizer + per-field hash rules

data/
  __init__.py

  datamodules/
    __init__.py
    base.py
    csv.py
    parquet.py               # serves both parquet_manifest and parquet_images backends
    ifcb_bins.py

  datasets/
    __init__.py
    csv_image_dataset.py
    parquet_image_dataset.py
    ifcb_bins_dataset.py

  record_schemas/           # Pydantic for persisted records
    __init__.py
    sample.py
    target.py
    result.py
    embedding.py
```

```
    batch.py                # hot-path batches stay as dicts; this is the schema spec
    ensemble_member.py

  transforms/
    __init__.py
    builder.py
    letterbox.py
    aspect_bucket.py
    foreground_crop.py
    grayscale.py
    normalization.py
    crop.py
    blur.py
    noise.py
    rotation.py

  samplers/
    __init__.py
    factory.py
    class_balanced.py
    batch_aspect_buckets.py
    weighted.py

  manifests/                # dataset manifest IO + dojo inspect dataset
    __init__.py
    io.py
    directory_scan.py
    summary.py              # used by dojo inspect dataset
    validation.py

storage/
  __init__.py
  resolver.py                # output_root + dir_template + path resolution
  amplify.py                 # wrapper around amplify-storage-utils
  cache.py                   # local read-through cache (when configured)

models/
  __init__.py

  backbones/
    __init__.py
    base.py
    registry.py
    torchvision.py
    timm.py                 # gated by [timm] extra
    weights.py              # none/library/checkpoint initialization
    feature_extractor.py
    freeze.py               # apply training.freeze.backbone policies

  heads/
    __init__.py
    base.py
    classification.py
    regression.py
    ordinal.py              # ordinal_classification semantics
    multihead.py
```

```
    projection.py              # SSL projection / prediction heads

  tabular_input/
    __init__.py
    encoders.py
    normalization.py

  embedding_adapter/
    __init__.py
    identity.py
    linear.py
    mlp.py

  compositors/
    __init__.py
    supervised.py              # backbone + tabular_input + implicit concat + adapter + head(s)
    ssl.py                     # SSL composition (encoder + projection)
    snapshot_ensemble.py       # composition for task.type: snapshot_ensemble

tasks/
  __init__.py

  supervised/
    __init__.py
    module.py                  # LightningModule
    objectives.py
    metrics.py
    step_outputs.py

  ssl/
    __init__.py
    dino_v2.py                 # functional
    lightly_backbones.py
    lightly_heads.py
    lightly_losses.py
    eval_callbacks.py          # schedules representation_eval during SSL training

  snapshot_ensemble/
    __init__.py
    module.py                  # supervised module + cycle-end snapshots
    cycle_scheduler.py
    cycle_checkpointing.py

  representation_eval/         # was eval/; supports SSL + supervised checkpoints
    __init__.py
    runner.py                  # dojo eval representation entrypoint
    knn.py
    linear_probe.py
    embeddings.py
    diagnostics.py             # augmentation consistency, retrieval, etc.
    clustering.py
    projections.py
    outlier.py
    scheduling.py              # during-training scheduling helpers

losses/
```

```
  __init__.py
  classification.py
  regression.py
  ordinal.py                    # CORAL, CORN
  class_balanced.py
  focal.py
  factory.py

metrics/
  __init__.py
  classification.py
  regression.py
  ordinal.py
  multihead.py
  calibration.py
  confusion.py                  # computes confusion matrix + derived metrics
  representation_eval.py        # knn / linear-probe / diagnostic metrics

training/
  __init__.py
  trainer_factory.py
  callbacks.py
  checkpointing.py
  resume.py
  early_stopping.py

ensemble/
  __init__.py
  runner.py                     # dojo ensemble entrypoint
  candidates.py                 # dojo ensemble candidates entrypoint
  discovery.py                  # explicit, manifest, run_dir_glob, checkpoint_glob, result_uri_glob, impli
  compatibility.py              # hashes + cascading metadata policy + drift check
  selection.py                  # all, best_candidate, top_k, greedy_forward
  combine.py                    # logits_mean, probabilities_mean, majority_vote, prediction_mean, predicti
  cached_results.py             # offline ensembling from result Parquet
  manifest.py                   # JSON manifest IO
  bundle.py                     # ensemble artifact bundling
  preprocessing.py              # member-specific + shared fast path

inference/
  __init__.py
  inferencer.py
  outputs.py
  preprocessing.py
  batch_writer.py

export/
  __init__.py
  torchscript.py
  onnx.py
  metadata.py
  bucketing.py                  # ONNX + aspect_bucket shapes

results/
  __init__.py
  schemas.py                    # canonical tall-Parquet schema
```

```
  writers.py                    # multi-sink result writer via amplify-db-utils
  parquet.py                    # primary functional backend via amplify-db-utils
  partitioning.py               # split / stage / epoch keys
  metadata.py                   # _metadata.json sidecar
  # No csv.py as primary - Parquet is canonical; CSV optional

figures/                        # renderer behind the figures: output block; PNG / SVG / HTML
  __init__.py
  base.py                       # figure-spec dispatch from *_outputs.figures
  training_curves.py
  confusion_matrix.py           # heatmap; matrix data computed in metrics/confusion.py
  calibration.py
  projections.py                # UMAP / t-SNE / PCA scatter
  ensemble_comparison.py

artifacts/
  __init__.py
  paths.py                      # run-artifact layout within an already-resolved run dir (output_root + di
  manifest.py                   # run-level artifact manifest
  metrics.py
  checkpoints.py
  hashing.py                    # checkpoint_hash + filename convention

logging/
  __init__.py
  base.py
  factory.py                    # multi-sink composition
  local.py                      # functional (only registered sink)

runtime/
  __init__.py
  preflight.py                  # runtime.preflight checks
  controls.py                   # runtime.* behavior knobs
  seeding.py
  device.py
  autobatch.py

hydra/
  __init__.py
  compose.py                    # Hydra Compose API wrapper (no @hydra.main)
  resolvers.py                  # custom OmegaConf resolvers
  sweep.py                      # Dojo sweep preparation + sweep_id / manifest wiring

utils/
  __init__.py
  import_utils.py
  random.py
  distributed.py
  torch_utils.py
  serialization.py

patches/
  __init__.py
  model_summary_with_grad.py
```

src/dojo_deprecated/

The pre-refactor package, already moved out of the new `src/dojo` path. Importable for reference during the clean-break refactor, deleted once the new `src/dojo` implementation covers everything through P4.7 and nothing imports from it, per `13-workplan.md`. No code added here.

tests/

```
tests/
  fixtures/
    configs/
    images/
    manifests/
    checkpoints/
    parquet/
    ifcb_bins/
    cached_results/              # for offline ensemble tests

  unit/
    config_schemas/
    runtime/
    data/
    storage/
    transforms/
    models/
    heads/
    objectives/
    losses/
    metrics/
    logging/
    results/
    export/
    ensemble/
    representation_eval/
    hashing/

  integration/
    test_train_supervised.py
    test_train_ssl_dino_v2.py
    test_train_snapshot_ensemble.py
    test_representation_eval_standalone.py
    test_representation_eval_during_ssl.py
    test_supervised_holdout_eval.py
    test_inspect_config.py
    test_inspect_dataset.py
    test_inspect_backbone.py
    test_inspect_checkpoint.py
    test_infer_predictions.py
    test_infer_embeddings.py
    test_ensemble_from_run_dirs.py
    test_ensemble_from_cached_results.py
    test_ensemble_candidates_manifest.py
    test_export_torchscript.py
    test_export_onnx.py
    test_sweep_expansion_config.py
    test_sweep_outputs.py
```

Notes on key directories

config_schemas/

Pydantic is the single source of truth for the config tree (03-configuration.md). The CLI loads Hydra-composed configs and validates them through config_schemas.root. The models are strict (extra="forbid") and enums list only implemented values, so unknown keys and out-of-enum / unknown-tag values — including any deferred feature — fail generic validation at load; that behavior is covered by generic tests in tests/unit/config_schemas/, not per-feature stub tests. Hashing rules and the canonical-JSON canonicalizer both live in config_schemas/hashing.py, next to the schemas they consume, so field-level hash inclusion rules and the canonicalizer stay co-located with the field definitions. artifacts/hashing.py (checkpoint_hash + filename) imports that canonicalizer.

data/record_schemas/

Pydantic schemas for persisted records (samples, targets, results, embeddings, ensemble-member rows). Hot-path DataLoader batches stay as dicts/dataclasses internally; record_schemas describe the on-disk shape written by results/writers.py.

storage/

Thin wrapper around amplify-storage-utils. The wrapper exists so output-path resolution (output_root, dir_template, dir, and the --resolved-config replay guard / --clobber handling) is a project-level concern not pushed into the library. It is top-level because storage is shared by data loading, result writing, checkpointing, export, and artifact inspection.

models/compositors/

Compositors assemble model parts:

image_input backbone + optional tabular_input encoder + implicit input concatenation + optional embedding ac

Tasks (tasks/supervised, tasks/ssl, tasks/snapshot_ensemble) train composited models. The compositor for task.type: snapshot_ensemble is a supervised compositor plus the cycle-end snapshot machinery.

ensemble/

The ensemble package is intentionally *not* organized around ensemble algorithm types. There is one prediction-space pipeline; candidate source, selection strategy, and combine mode are the axes that vary. cached_results.py is the offline-from-Parquet path described in 08-ensembles.md; it does not duplicate logic from combine.py.

runtime/

Process-level concerns separated from training-loop concerns: preflight checks, seeding, device selection, autobatch, fast-dev-run wiring. preflight.py is invoked from CLI entrypoints before any heavy work, per 12-validation-testing-and-preflight.md.

figures/

Renderer behind the configurable *_outputs.figures block. Per 06-results-artifacts-and-metadata.md, the two artifact classes stay separate: numeric aggregates and confusion-matrix **data** are written to metrics/ (computed in metrics/, persisted via artifacts/metrics.py), while figures/ owns rendered **images** (PNG / SVG / HTML) — training curves, confusion-matrix heatmaps, projection scatter, calibration, and ensemble comparison. A confusion matrix is therefore not a results/ row; results/ is the per-row tall-Parquet writer only.

tasks/representation_eval/

representation_eval is the package name — ssl_eval does not exist in the V2 tree. The runner supports both standalone use (dojo eval representation) and during-training scheduling (via eval_callbacks.py in the SSL task and scheduling.py helpers callable from the supervised task).

Cross-References

- `03-configuration.md` — config tree the `config_schemas/` package validates; `outputs/` resolution semantics.
- `04-data-and-storage.md` — dataset backends plus top-level `amplify-storage-utils` integration in `storage/`.
- `05-models-training-and-heads.md` — what lives under `models/`, `losses/`, `metrics/`, `training/`, and `tasks/supervised/`.
- `06-results-artifacts-and-metadata.md` — schema implemented by `results/`, `artifacts/`, and `data/record_schemas/`.
- `07-ssl-and-representation-eval.md` — what lives under `tasks/ssl/` and `tasks/representation_eval/`.
- `08-ensembles.md` — package layout under `src/dojo/ensemble/`.
- `09-sweeps-and-batch-runs.md` — Compose API + Dojo sweep expansion under `hydra/` and `sweep_outputs/`.
- `10-export.md` — `export/` package contents.
- `11-dependencies.md` — extras that gate `models/backbones/timm.py`, `tasks/ssl/`, IFCB, etc.
- `12-validation-testing-and-preflight.md` — `runtime/preflight.py` and the `tests/` tree.
- `13-workplan.md` — priority order for populating this tree.
- `appendix-deferred-features.md` — deferred features absent from the strict schema until promoted into active work.

Glossary

Purpose

Single source of truth for terminology used across the design doc. Defines config keys, identifiers, hashes, record taxonomies, and architectural concepts referenced by every other file. When in doubt, the terms below win.

Identifiers and hashes

- **`run_id`** — Identifier for one `dojo train` / `dojo eval` / `dojo ensemble` invocation. Either manually set, rendered from a template, or generated from the fresh unseeded `{coolname:noseed}` default. There is no `run_hash`.
- **`config_id` / `config_hash`** — `config_hash` is a deterministic hash of the resolved config excluding runtime-resolved values, output paths, `output_root`, and the `*_outputs` blocks. `config_id` is either manually set or generated from `{coolname:config_hash}` after `config_hash` is computed.
- **`dataset_id` / `dataset_hash`** — `dataset_hash` is a cheap, always-available identity that never reads image pixels or tabular cell payloads: manifest identity / canonical manifest projection or URI plus size/etag/last-modified plus backend type; declared logical tabular column names / bindings are included when present. Falls back to URI-only hashing with `dataset_hash_provenance: uri_only` when size/etag are unavailable. `dataset_id` is only present when the manifest provides a self-name; there is no generated fallback.
- **`dataset_content_hash`** — a separate, optional true hash over sample content: image bytes plus declared tabular feature values when present, recorded only when `dojo inspect dataset` runs a content pass (`--content-hash` / `--normalization`). It is integrity / drift verification and is never folded into `dataset_hash`, which must stay stable regardless of whether a content pass ran.
- **`checkpoint_hash`** — SHA-256 of the `.ckpt` file bytes. The first 6 hex characters appear in the checkpoint filename (`{stem}.{first6_hex}.{ext}`). There is no `checkpoint_id`.
- **`model_id` / `model_hash`** — Applies to exported portable model artifacts (`.pt` / `.onnx`). `model_hash` is SHA-256 of the exported file bytes; `model_id` is either manual or generated from `{coolname:model_hash}`.
- **`ensemble_id` / `ensemble_hash`** — `ensemble_hash` is canonical hash of the selected-ensemble identity block (selected members + selection + combine config), excluding candidate-audit metadata; `ensemble_id` is either manual or generated from `{coolname:ensemble_hash}`.
- **`sweep_id` / `sweep_hash`** — `sweep_hash` is canonical hash of the sweep definition (base config + axes + value lists), excluding runtime-resolved values and output paths. `sweep_id` may be a template render or fallback to `{coolname:sweep_hash}`.
- **`ensemble_member_id`** — Union column for ensemble member-level result rows. Equals the member's `checkpoint_hash` (for checkpoint members) or `model_id` (for exported-model members). Populated only when `ensemble_result_scope=member`.
- **`ensemble_result_scope`** — Ensemble result-row scope. `member` means a retained selected-member prediction row; `ensemble` means the combined ensemble prediction row.

- **Compatibility hashes** — `target_schema_hash`, `class_mapping_hash`, `model_config_hash`, `preprocessing_hash`. Both the hash and the source sub-block are stored in `_metadata.json` so consumers can fast-compare on the hash and slow-compare via human-readable diff on mismatch. The canonical source field lists live in `06-results-artifacts-and-metadata.md`.
- **Inference contract** — Portable, self-describing bundle embedded in both training checkpoints (`checkpoint["dojo_inferen`
and exported models: buildable `model_config`, resolved `inference_pipeline` + frozen preprocessing stats, per-head class maps, target schema with frozen target transforms, resolved `objective_summary` for holdout scoring, and the four compatibility hashes. `dojo infer` / `dojo eval` rebuild a model from it without the producing run's `resolved.yaml`. Full definition in `06-results-artifacts-and-metadata.md`.

Coolname seed sources

`{coolname:<source>}` generates a deterministic coolname from the named resolved value, usually a hash such as `config_hash`, `model_hash`, `ensemble_hash`, or `sweep_hash`. The seed material includes the source label as well as the value, so identical raw hash strings in different domains do not imply identical names. `{coolname}` without a source is deterministic from resolved `runtime.seed`; use `{coolname:noseed}` for a fresh unseeded name per invocation. Defaults use explicit sources where determinism matters and `{coolname:noseed}` where uniqueness matters.

Storage and truncation

Full hashes are recorded by default (e.g. full SHA-256 hex). The only standard truncation is the **first 6 hex characters** embedded in checkpoint filenames. Result-row provenance columns carry both `*_id` and `*_hash` when both exist.

Top-level config groups

- **`experiment`** — Experiment-level metadata (name, etc.).
- **`task`** — Carries `task.type` which selects training behavior: `supervised`, `ssl`, or `snapshot_ensemble`.
- **`runtime`** — Process behavior: `seed`, `precision`, `num_workers`, `fast_dev_run`, `autobatch`, `preflight`, and the `run_id` / `sweep_id` templates.
- **`storage`** — Storage resolver config (local cache dir, etc.). Peer to `runtime`.
- **`data`** — Dataset backend + targets.
- **`transforms`** — Input preprocessing: image transform pipeline plus tabular preprocessing. Config resolution derives the non-train `inference_pipeline` from it.
- **`model`** — `image_input`, `tabular_input`, `embedding_adapter`, `heads`. `model.image_input.backbone` holds the image backbone config. `backbone.architecture` describes the module shape; `backbone.weights` describes initialization. Image and tabular embeddings concatenate implicitly when both inputs are enabled.
- **`objectives`** — Bind heads to losses, weights, metrics.
- **`ssl`** — SSL framework selector (functional method: `dino_v2` via Lightly).
- **`representation_eval`** — Replaces the old `ssl_eval`; applicable to any task that produces image embeddings, supervised or SSL.
- **`training`**, **`optimizer`**, **`scheduler`**, **`checkpointing`** — Top-level training config blocks.
- **`ensemble`** — Ensemble selection/combine/candidate config for the `dojo ensemble` family and `task.type: snapshot_ensemble`.
- **`sweep`** — Sweep-generation config. `sweep.mode: grid` is functional; `sweep.mode: bayesian` is deferred and not a schema value. Sweep axes normalize into this block before concrete runs are expanded by `dojo sweep prepare`. Initial `sweep.execution.mode` is `manual`; automated local sequential and Slurm execution are deferred.
- **`output_root`** — Single filepath string used as the base for rendered `*_outputs.dir_template` values and bare-relative `*_outputs.dir` values.
- **`training_outputs`**, **`ensemble_outputs`**, **`sweep_outputs`**, **`eval_outputs`** — Peer output-config blocks (see `03-configuration.md`). `eval_outputs` is the home for `dojo infer` / `dojo eval` rows, metrics, and the always-written `eval_manifest.json`.

Sweep terminology

- **Grid sweep** — A sweep with `sweep.mode: grid`, where each entry in `sweep.grid` maps a target config path to a YAML list of values. The concrete runs are the cartesian product of those lists.

- **Batch run** — A batch-run-style grid sweep where the only intentional run-varying parameter is `runtime.seed`. Used to measure sensitivity to random initialization, data order, and other seeded behavior while holding model / training settings fixed.
- **Bayesian sweep** — A deferred `sweep.mode: bayesian` for search engines such as Optuna. Not represented in the schema initially; authoring it fails validation. See `appendix-deferred-features.md` P4.1.
- `sweep.active_run` — Generated resolved-config metadata for one concrete run in a sweep. It records the realized sweep-axis values for that run and is not written in source configs.
- `sweep_manifest.json` — Immutable job index written by `dojo sweep prepare` under `sweep_outputs.dir/config/`. It records each concrete job's index, `run_id`, realized sweep values, output dirs, and resolved config / status paths. It is not a mutable status cache.

Path resolution

For any `dir` key:

- Absolute paths (`/folder`) are used as provided.
- `./folder` is resolved relative to the process CWD.
- Bare relative paths (`folder`) are resolved against the parent object's resolved `dir`. For top-level `*_outputs.dir`, the parent base is `output_root`. For sub-block values (e.g. `results.dir` under `training_outputs`), the parent base is `training_outputs.dir`.

`dir_template` uses Dojo-owned `{...}` template syntax with an **open** token set: any dotted resolved-config path (e.g. `{experiment.name}`, `{runtime.run_id}`, `{training.batch_size}`) plus a fixed set of special tokens (`{coolname:<source>}`, `{coolname:noseed}`, `{coolname}`, `{timestamp}`, `{job_num}`, `{ensemble_id}`, `{source_run_dir}`, `{dataset_id}`). Tokens may carry a `:spec` suffix — the custom `:slug` filesystem-safe formatter or any standard Python format spec (e.g. `{model.image_input.backbone.architecture.name:slug}`, `{training.batch_size:03}`). Resolved after config composition, validation, and runtime-value generation. The full rules live in `03-configuration.md`.

Result taxonomy

- `split` — Source dataset split: `train`, `val`, `test`, `unlabeled`, `holdout`.
- `stage` — Process that produced a result row: `train_validation`, `holdout_eval`, `infer`, `representation_eval`, `ensemble_eval`.
- `record_type` — What columns are populated on a row: `sample_metadata`, `embedding`, `classification_output`, `regression_output`, `ordinal_output`, `nearest_neighbor`, `knn_prediction`, `classification_probe_prediction`, `regression_probe_prediction`, `ordinal_probe_prediction`, `cluster_assignment`, `projection`, `outlier_score`, `diagnostic`.
- `head_name` — Logical head identifier on output rows.
- `embedding_kind` — `image_embedding`, `tabular_embedding`, `fused_input_embedding`, `head_input_embedding`.

Class and label vocabulary

Two axes, kept distinct throughout configs, code, and outputs:

- **class** — the *category* axis: the taxonomy of possible categories, which is sample-independent. Terms on this axis use `class`.
- **label** — the *per-sample assignment* axis: the category assigned to a given sample. Terms on this axis use `label`.

Within each axis, a bare or `_index` name is the integer form and a `_name` name is the human-readable string. In training, `target` and `prediction` default to integer indices.

- `label_index_column` — Data-config column (`data.targets.<target>`) holding each sample's integer class index.
- `label_name_column` — Data-config column holding each sample's class name (string). At least one of `label_index_column` / `label_name_column` is required per target. When only names are given, indices are assigned by sorting the distinct names (ASCII order).
- `num_classes` — Head output width (`model.heads.<head>.num_classes`): the count of categories the head projects to.

- `classes` — `_metadata.json` per-head ordered list of class names, indexed by class index.
- `class_mapping` — `_metadata.json` per-head mapping of class index (string key, natural numeric order) → class name. Same vocabulary as the `class_mapping_hash` compatibility dimension.

Classification prediction columns on `classification_output` rows:

- `prediction_index` — Predicted class index (argmax).
- `prediction_label` — Predicted class name, resolved through `class_mapping`; null when no name source is available.
- `prediction_confidence` — Probability of the predicted class.

Missing target policies

- `error` — Missing labels for the target fail preflight in active supervised / eval splits.
- `drop_sample` — Samples missing the target are removed when that target is required by an enabled objective or eval scorer.
- `mask_objective` — Samples missing the target remain in the batch, but losses and metrics for objectives / scorers using that target ignore those rows. Other objectives can still use labels present on the same sample.

External vs. internal target/prediction columns

- `target` / `prediction_value` are **external** (original units).
- `target_internal` / `prediction_value_internal` are **model-space** (post `target_transform`).
- When no `target_transform` is configured, writers may either populate both identically or leave the `_internal` columns null; the behavior is recorded in `_metadata.json`.

Task types (`task.type`)

- `supervised` — Standard supervised training, single- or multi-head.
- `ssl` — Self-supervised training. Functional method: `dino_v2` via Lightly.
- `snapshot_ensemble` — Convenience type that runs supervised training with a snapshot-cycle scheduler and then ensembles the snapshot checkpoints, sharing one run directory.

`inference`, `eval`, `ensemble`, and `export` are **commands**, not `task.type` values; the command (not `task.type`) selects which config blocks apply (`02-cli-and-task-types.md`).

Head types

- `multiclass_classification`
- `binary_classification`
- `multilabel_classification` (deferred — see appendix P4.2)
- `regression`
- `ordinal_classification` (not `ordinal_regression`; the head predicts a discrete ordered bin)
- `distributional_regression` (deferred — see appendix P4.9)
- `count_regression` (deferred — see appendix P4.9)

Backbone sources and weights

`model.image_input.backbone.architecture.source`: `torchvision`, `timm`. `timm` is functional, gated by the `timm` extra. Checkpoint loading is `model.image_input.backbone.weights.source: checkpoint`, not an architecture source. Lightly is **not** a public backbone source; SSL configs use `ssl.framework: lightly` but still select a backbone architecture through one of the public architecture sources.

Cross-References

- `03-configuration.md` — canonical config shape and path-resolution semantics.
- `06-results-artifacts-and-metadata.md` — full IDs/Hashes table and result schemas.
- `02-cli-and-task-types.md` — CLI surface and `task.type` semantics.